

Глава 8. Масштабируемость и избыточность локальной сети

Цели главы:

- Объяснить необходимость иерархии и модульности сети;
- Рассказать о технологиях, применяемых в иерархической сети на уровнях доступа, агрегации и ядра;
- Обозначить основные проблемы при организации резервирования на уровне L2 и рассказать об основных протоколах, применяемых для избежания этих проблем (STP, LACP);
- Продемонстрировать необходимость резервирования на уровне L3 и ознакомить с командами конфигурирования протокола VRRP на оборудовании Eltex.

8.1. Иерархические сети

8.1.1. Особенности планирования сетей различного масштаба

Строительство сетей может осуществляться либо структурно, либо хаотически. Как показывает практика, большинство сетей малого масштаба (сети SOHO¹ и малого бизнеса) строятся без предварительного планирования и предназначены для решения сиюминутных бизнес-задач без средне- и долгосрочного планирования как самого бизнеса, так и обслуживающей его инфраструктуры. Очень часто такие сети состоят вообще из одного сетевого устройства, комбинирующего в себе коммутатор, маршрутизатор и точку доступа.

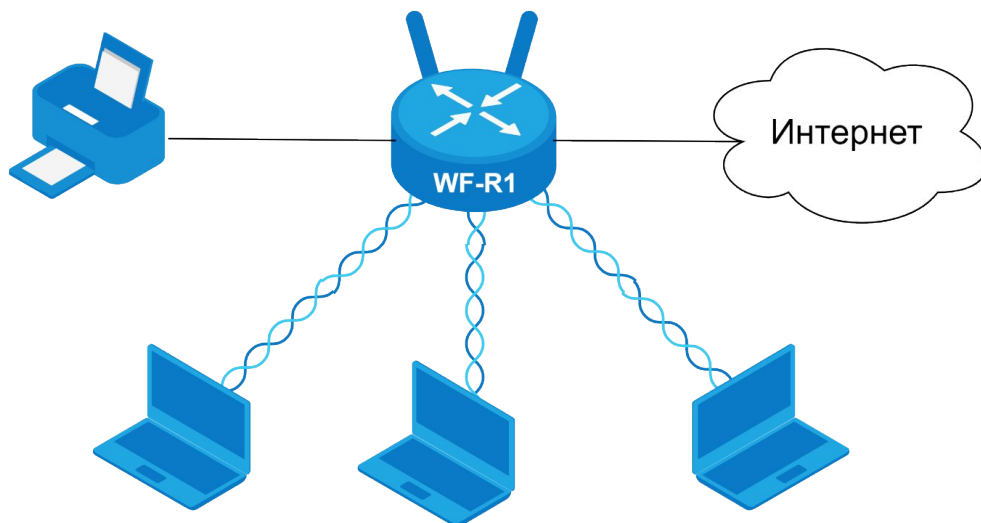


Рисунок 8.1.1.1. – Сеть масштаба «малый офис – домашний офис».

Владельцы и эксплуатанты таких сетей постоянно сталкиваются как с проблемами внедрения новых сервисов (например, IP-телефонии), так и с проблемами безопасности, которые так и останутся нерешенными из-за несоответствия инфраструктуры новым задачам и вызовам.

Самой большой проблемой этих сетей является то, что при отказе единственного сетевого устройства все оконечные устройства теряют связность как между собой, так и с глобальной сетью, а значит, и со всеми сетевыми сервисами.

¹ Small office & Home office – сети для дома и малого офиса

Сети среднего бизнеса довольно часто строятся по остаточному принципу и развиваются хаотически, что также обусловлено недостатками в планировании бизнеса. Обычно они «эволюционируют» из сетей малого бизнеса путем дозакупки одного или нескольких коммутаторов.

Так как при планировании таких сетей изначально отсутствует структурированный подход и все сетевые устройства работают на одном уровне, такие сети еще называют «**плоскими**» (*flat networks*).

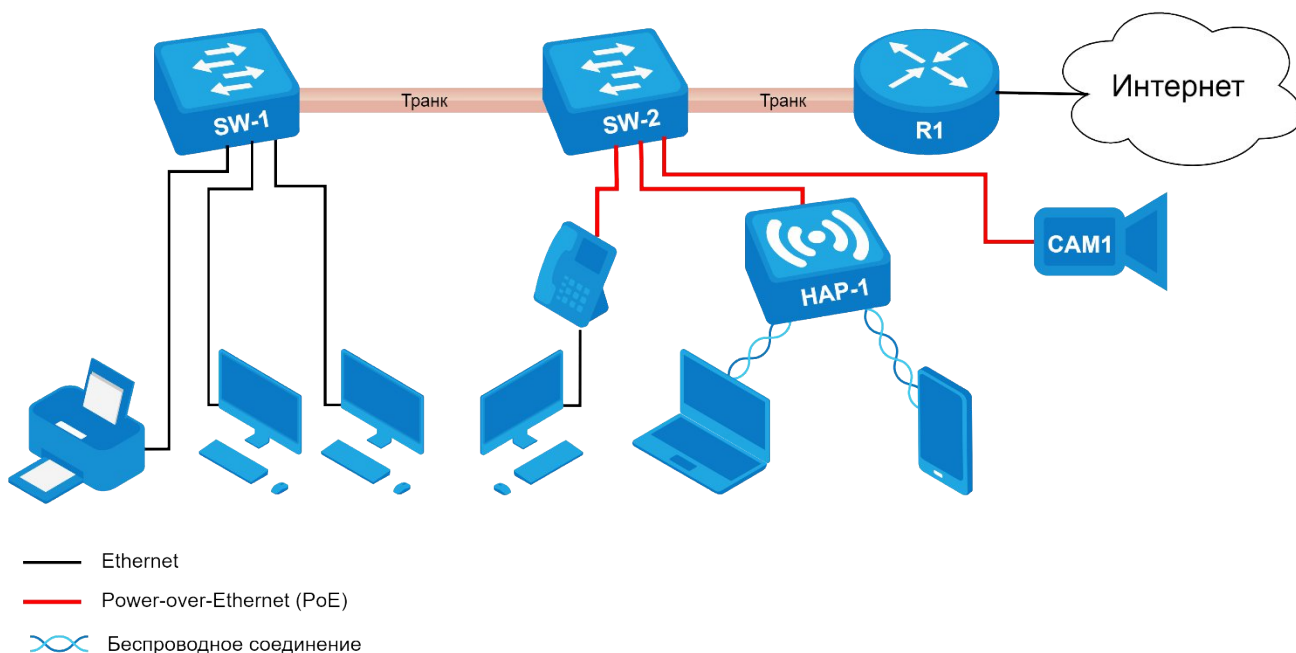


Рисунок 8.1.1.2. – Сеть предприятия масштаба среднего бизнеса

При таком методе организации сети увеличиваются широковещательные домены, даже если используются виртуальные сети (VLAN), так как растет количество устройств. Для беспроводного подключения довольно часто используются автономные точки доступа (*Heavyweight Access Point, HAP*), которые настраиваются вручную. Сильно осложняется внедрение сервисов, которые критически необходимы для бизнеса такого масштаба, но не предусматривались при проектировании инфраструктуры, например, IP-телефонии или цифрового видеонаблюдения.

В целом, дальнейший рост такой неструктурированной сети приводит к ухудшению (деградации) сервисов по следующим причинам:

1. большое количество широковещательных кадров в кабельной среде;
2. неоптимальная загрузка каналов передачи данных и невозможность приоритизации трафика для критичных приложений;
3. недостаточная гибкость в настройке беспроводной сети (каналы, SSID, общая зашумленность эфира и т.д.).

Вышеперечисленные недостатки становятся особенно заметными и начинают отрицательно влиять на бизнес владельца сети при подключении 100-110 конечных устройств. На этом «рубеже» у организации-владельца сети обычно формируется понимание следующего:

1. масштабов рынка и необходимого количества персонала на ближайшую и

среднесрочную перспективу;

2. географического расположения бизнеса: например, будет ли все производство сконцентрировано в одном месте, или будет несколько производственных баз в разных регионах, сколько офисов необходимо подключить к сети и т.д.;

3. состава сервисов, которые будут необходимы для бизнеса, и методов их внедрения (локально или «из облака»);

4. требований к кабельному и беспроводному сегментам сети.

При всём возможном многообразии вариантов ответов на вышеназванные вопросы основные требования к сети чаще всего сводятся к следующим:

1. подключенные оконечные устройства должны получать запланированные для них сервисы не менее определенного количества времени в день, месяц, год (доступность сети);

2. сеть должна обеспечивать оптимальное прохождение трафика, в том числе приоритетного (качество обслуживания, QoS);

3. сеть должна быть спроектирована таким образом, чтобы текущие пользователи получали по крайней мере текущий уровень сервисов при подключении новых пользователей (масштабируемость);

4. сеть должна поддерживать подключение и управление устройствами в рамках существующих или вновь создаваемых административных единиц предприятия (модульность);

5. отказ отдельного сетевого устройства или канала передачи данных должен затрагивать как можно меньшее количество пользователей и желательно не приводить к прерыванию критически важных сервисов (резервирование и избыточность).

Принципы проектирования сетей с иерархией сетевых устройств (иерархических сетей) являются одинаковыми для всех сетей от 100 и более подключенных устройств и направлены на достижение оптимальных характеристик работы сети, снижение перегрузок, уменьшение доменов отказа и т.д.

8.1.2. Специализация сетевых устройств

Все сетевые устройства обладают определенным набором характеристик, которые могут значительно отличаться даже внутри группы продуктов одного производителя.

Так, например, количество портов у одного коммутатора может разниться от 8 до 52, а пропускная способность каждого порта может варьироваться от 100 Мб/с до 400 Гб/с. Кроме того, значительно могут отличаться и функции, поддерживаемые устройствами.

Можно ли использовать в сети только устройства одной модели с максимально возможными характеристиками? Да, это возможно, но будет крайне экономически невыгодно, так как большая часть возможностей сетевого устройства будет попросту простаивать. Например, коммутатор с портами пропускной способностью 10 Гб/с бессмысленно использовать для подключения устройств с сетевыми интерфейсами 100Мб/с. С другой стороны, в центре обработки данных скорость 10 Гб/с приемлема для

подключения одиночных серверов, но уже крайне недостаточна для подключения «лезвийных» (*blade*) систем и организации межстоечной коммутации.

Кроме экономических существуют еще и чисто технические требования к различным группам сетевых устройств. К примеру, одни устройства требуют питания по сетевому кабелю (функция PoE), а другие нет. Соответственно, владелец сети должен использовать два вида коммутаторов: с функцией PoE и без нее. В некоторых сетях маршрутизатор может выполнять функции межсетевого экрана (МСЭ), а в некоторых для этого может быть предусмотрено отдельное устройство.

Исходя из совокупности факторов как технической, так и экономической природы, сетевые устройства можно условно разделить на несколько групп:

1. Устройства, предназначенные для подключения оконечных устройств. К ним относятся коммутаторы, точки беспроводного доступа и IP-телефоны с функцией коммутатора. Эти устройства образуют так называемый «уровень доступа» (*access layer*), так как именно они дают возможность подключения (доступа) в сеть.

2. Устройства, предназначенные для коммутации трафика устройств уровня доступа. Эти устройства собирают и распределяют трафик оконечных устройств, поэтому образуют так называемый *уровень агрегации* (от англ. *aggregation² layer*). Обычно это многоуровневые коммутаторы с более высокой пропускной способностью портов, чем у их «коллег» на уровне доступа.

3. Устройства, предназначенные для коммутации трафика от нескольких локальных сетей под управлением одной организации, таких как сети отдельных зданий, образующих единый комплекс (кампус), сеть центра обработки данных и т.д. Такие устройства называют «устройствами уровня ядра».

4. Устройства, которые служат для подключения локальной сети к сети Интернет или филиалам посредством технологий WAN. Такие устройства образуют подсистему «границы сети» (*network border*) и чаще всего представлены маршрутизаторами, маршрутизаторами с функцией МСЭ или отдельными устройствами МСЭ.

8.1.3. Коммутаторы уровня доступа

Коммутаторы уровня доступа предназначены для непосредственного подключения компьютеров, принтеров и других стационарных устройств посредством структурированной кабельной системы (СКС).

На уровне доступа происходит подключение оконечных устройств, трафик от которых должен коммутироваться в двух направлениях:

1. в горизонтальном: устройствам в той же локальной сети или в VLAN;
2. в вертикальном: в другие локальные сети, в VLAN или в Интернет.

Так как устройства уровня доступа обычно изображают внизу схем топологий, каналы, предназначенные для «вертикальной» передачи данных, называют «восходящими» или «аплинками» (от англ. *uplink*).

Аплинк – канал передачи данных, предназначенный для подключения коммутатора к ответному коммутатору более высокого уровня.

² В некоторых источниках используется также термин *distribution layer* (уровень распределения).

Нетрудно предположить, что этот же канал на устройстве более высокого уровня будет называться «нисходящим» (downlink).

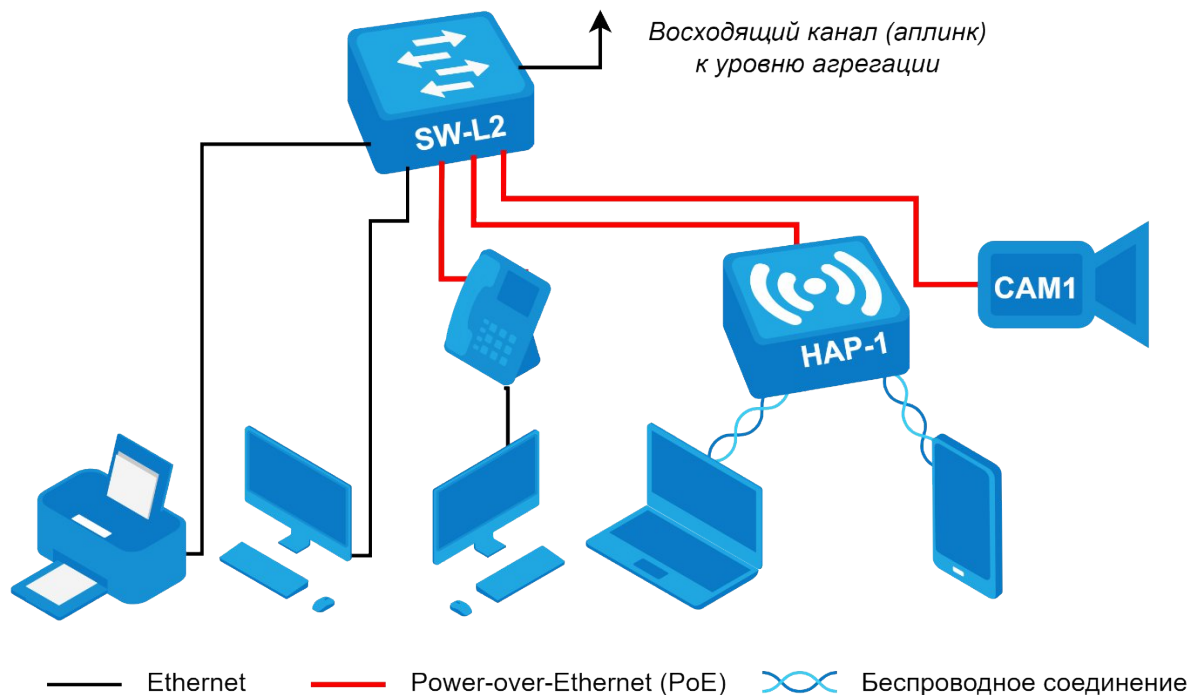


Рисунок 8.1.3.1. – Коммутатор уровня доступа и сформированный им сегмент сети

Рассмотрим сегмент сети, образованный коммутатором L2, на рисунке 8.1.3.1. В нем могут присутствовать несколько VLAN:

- VLAN данных, в которой будет передаваться трафик от стационарных персональных компьютеров;
- VLAN принтеров;
- VLAN камер видеонаблюдения;
- VLAN голосового трафика (voice VLAN), в которой будет передаваться трафик от телефона;
- Д) гостевая VLAN, к которой подключаются ноутбук и мобильный телефон через точку доступа;
- Е) VLAN для трафика управления беспроводными сетевыми устройствами (в нашем случае коммутатором);
- Ж) VLAN для трафика управления беспроводной локальной сетью (отдельная VLAN для точек доступа).

Кроме того, так как в сети есть высокоприоритетные устройства, такие как телефон и камера видеонаблюдения, желательно, чтобы коммутатор поддерживал технологии приоритизации трафика (Quality of Service, QoS). Также желательно, чтобы параметры приоритизации трафика назначались автоматически по протоколу LLDP-MED.

8.1.4. Требования к коммутаторам уровня доступа

Исходя из вышеперечисленного, мы можем вывести следующие требования для коммутаторов уровня доступа:

1. Расположение в специализированных аппаратных стойках и шкафах не далее 100м от штепселя подключения оконечного устройства, чтобы минимизировать потери сигнала от затухания в витой паре;
2. Наличие управляемого функционала коммутации с настройкой таблицы MAC-адресов;
3. Наличие медных или оптических интерфейсов с большей полосой пропускания, чем у основной массы интерфейсов для организации;
4. Наличие PoE для питания IP-телефонов, точек доступа, камер видеонаблюдения и других устройств, поддерживающих PoE;
5. Поддержка VLAN для сегментации ширококвещательных доменов;
6. Поддержка технологий приоритизации трафика (QoS) и автоопределения медиа-устройств (*Link Layer Discovery Protocol – Media Endpoint Discovery, LLDP-MED*) для назначения приоритета критичным типам трафика (голос, видео и т.д.);
7. Поддержка технологий управления массовым (ширококвещательным и многоадресным) трафиком: управление штормами (*StormControl*), обнаружение петель (*LoopbackDetection*), функция отслеживания многоадресных групп (*IGMP Snooping*), функция отслеживания DHCP (*DHCP Snooping*);
8. Поддержка объединения нескольких интерфейсов в единый канал передачи данных (*Link Aggregation Group, LAG*).

8.1.5. Уровень агрегации

При соединении коммутаторов доступа друг с другом транковыми каналами наблюдается довольно быстрое разрастание ширококвещательных доменов. На практике, если сеть предназначается для подключения более 100 устройств, рекомендуется для коммутации трафика между коммутаторами уровня доступа использовать коммутатор с портами большей пропускной способности, а также с функцией маршрутизации для обеспечения более быстрой пересылки пакетов между VLAN. То есть такой коммутатор предоставляет доступ к остальной части сети за счет коммутации, маршрутизации и политики сетевого доступа.

Так как такого рода коммутаторы предназначены для объединения потоков трафика от коммутаторов уровня доступа, их называют **коммутаторами агрегации**.

Для оптимизации потоков трафика рекомендуется организовывать связность коммутаторов доступа только через коммутатор агрегации.

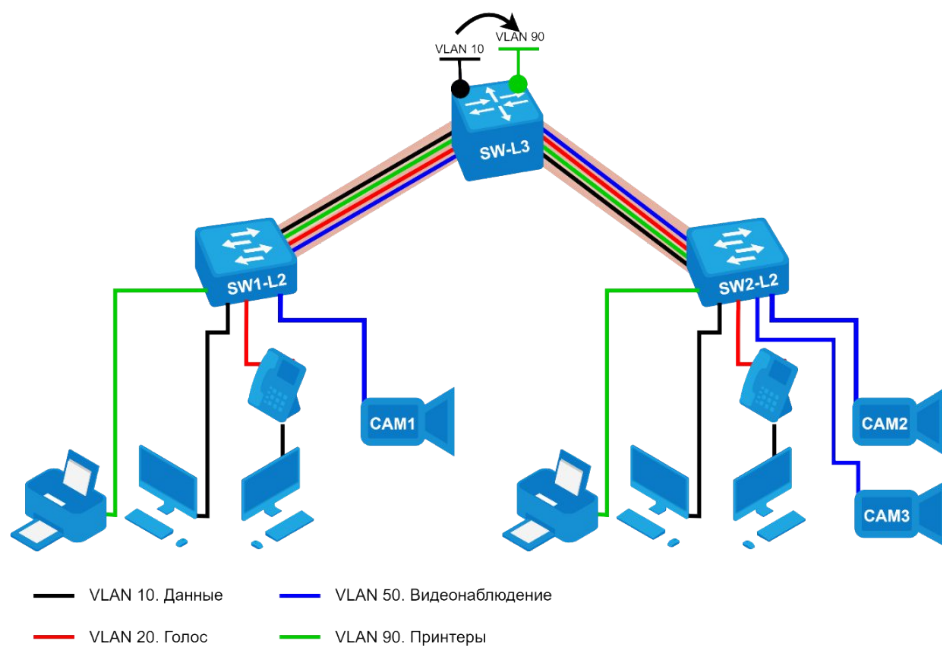


Рисунок 8.1.5.1. – Маршрутизация трафика между VLAN на уровне агрегации

Рассмотрим ситуацию на рисунке 8.1.5.1. Оконечные устройства сгруппированы в 4 VLAN: данных, голоса, видеонаблюдения и принтеров. Таким образом разделяется разнородный трафик, и появляется возможность его приоритизации. Если необходимо передать трафик из одной VLAN в другую, например, распечатать файл с компьютера, находящегося в VLAN данных, на принтере, находящемся в VLAN, применяется маршрутизация (подробнее о маршрутизации с применением коммутаторов уровня L3 см. Главу 4.).

Для передачи данных в подсистему «граница сети» также используется маршрутизация на уровне L3, причем чаще всего динамическая. Следовательно, коммутаторы уровня агрегации должны сочетать поддержку технологий как уровня L2, так и L3.

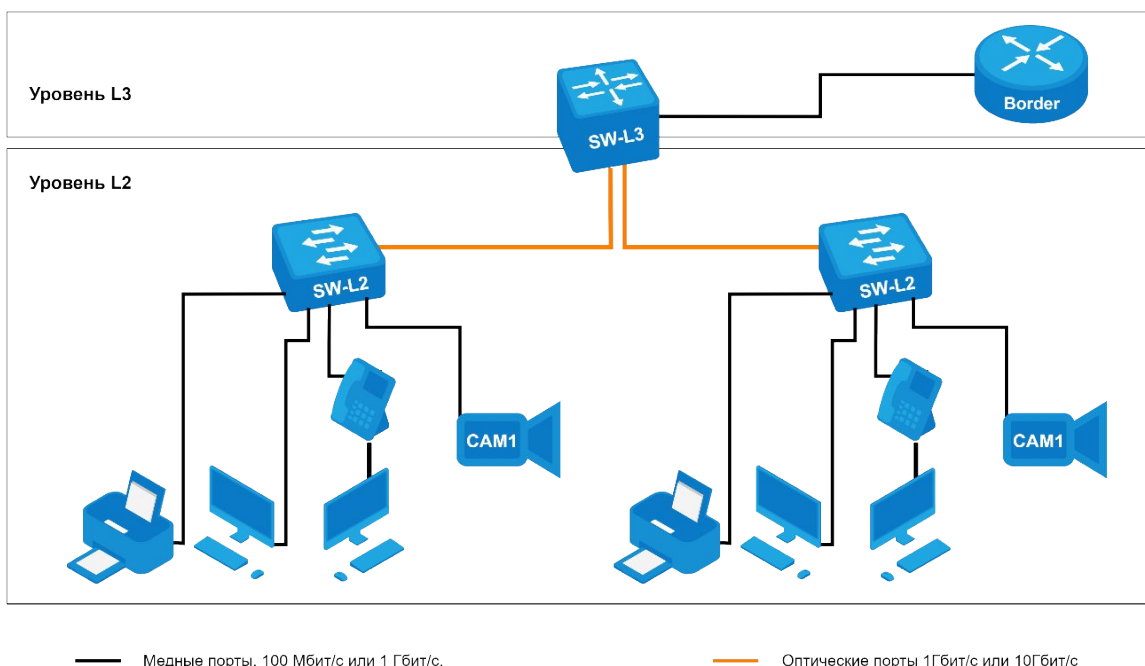


Рисунок 8.1.5.2. – Использование технологий уровней L2 и L3 в коммутаторах агрегации

Логичным будет предположить, что на уровне агрегации для коммутатора уже совершенно не критично наличие функции PoE, так как к нему подключаются сетевые устройства, которые PoE не поддерживают (коммутаторы, маршрутизатор, межсетевой экран (МСЭ) и т.д.).

Так как на коммутаторе агрегации терминируется широковещательный трафик всех VLAN, логичным представляется конфигурировать на нем либо сервер DHCP, либо ретранслятор DHCP (*DCHP Relay*). То же можно сказать и о ретрансляции DNS-запросов.

Коммутатор агрегации является важным звеном в передаче так называемого «критичного трафика» (например, голос и видео) от устройств на уровне доступа за пределы сети. Следовательно, поддержка технологий QoS является обязательной для данных коммутаторов.

8.1.6. Требования к коммутаторам уровня агрегации

1. Наличие управляемого функционала коммутации с настройкой таблицы MAC-адресов;
2. Поддержка статической и динамической маршрутизации трафика между различными VLAN;
3. Наличие медных или оптических интерфейсов с полосой пропускания, аналогичной полосе пропускания аплинков, подключаемых коммутаторов доступа, а также контроллеров беспроводной сети или межсетевых экранов;
4. Наличие оптических портов с большей полосой пропускания для подключения к уровню ядра;
5. Поддержка технологий приоритизации трафика (QoS) для передачи критичных разновидностей трафика (голос, видео и т.д.) за пределы сети;
6. Поддержка протокола STP для исключения возможности возникновения петель путем отключения избыточных путей для трафика на уровне L2;
7. Поддержка объединения нескольких интерфейсов в единый канал передачи данных (*Link Aggregation Group, LAG*);
8. Стекирование или агрегация интерфейсов на нескольких коммутаторах (*MLAG*).

8.1.7. Уровень ядра

Коммутаторы уровня ядра предназначены для коммутации и маршрутизации трафика, поступающего от коммутаторов уровня агрегации.

Основная задача коммутаторов данного уровня – работать максимально быстро и обеспечивать максимальную доступность (время работы без сбоев), так как этот компонент является наиболее критичным во всей сети. При возникновении сбоев (как аппаратных, так и сбоев конфигурации) без сетевой связности могут оставаться целые сегменты сети, в каждом из которых может быть несколько сотен конечных устройств.

Рассмотрим применение коммутатора ядра на рисунке 8.1.7.1. Коммутатор CORE (от англ. *ядро*) используется для подключения двух коммутаторов агрегации BLDG1 и BLDG2, которые, в свою очередь, организуют сети Здания 1 и Здания 2 соответственно. К ним подключаются коммутаторы уровня доступа SWx-L2. Вся кампусная сеть

подключена к Интернет-провайдеру посредством пограничного маршрутизатора Border. Для беспроводного подключения используются управляемые точки беспроводного доступа (*lightweight access point*) LAP1 и LAP2, управляемые контроллером беспроводной сети (*Wireless LAN Controller*) WLC.

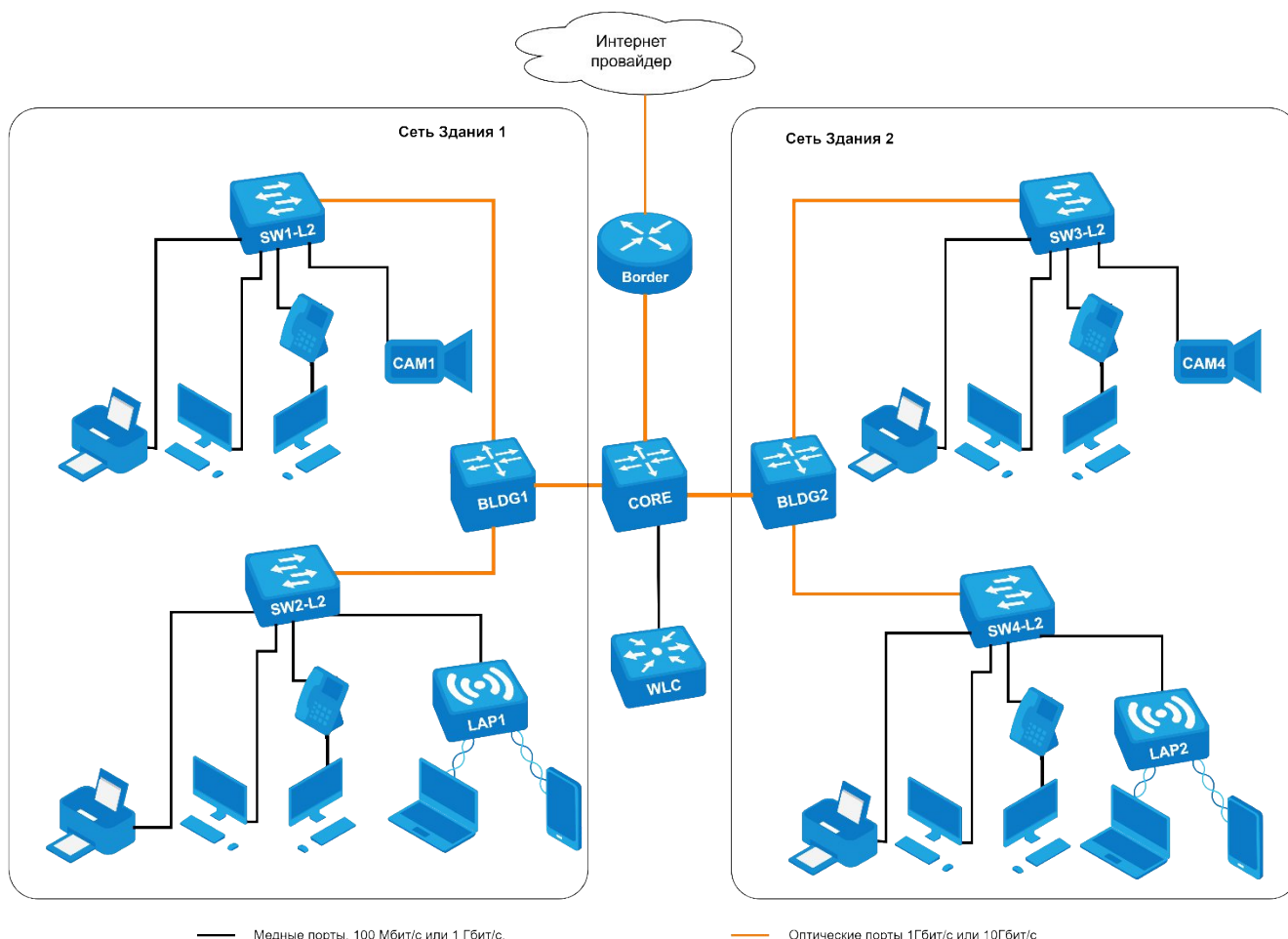


Рисунок 8.1.7.1. – Сеть группы зданий (кампусная сеть) с уровнями доступа, агрегации и ядра

Сеть, изображенная на рисунке 8.1.7.1., называется иерархической, так как все устройства в ней выполняют определенные функции в зависимости от «уровня». Данный термин не отражает взаимоотношения подчинения и зависимости.

Иерархическая организация сети позволяет планировать и строить сеть по модульному принципу: все сети соответствуют административному делению предприятия-эксплуатанта сети, ни одна из них не является транзитной, то есть отказ таких сетей не приводит к отказу сетей соседних административных единиц (отделов, управлений и т.д.). Также иерархический подход к построению сети предприятия облегчает процесс планирования обновления сети и процесс проектирования новых сегментов сети.

8.1.8. Требования к коммутаторам уровня ядра

К коммутаторам уровня ядра предъявляются следующие требования:

1. Повышенная общая пропускная способность коммутации;
2. Наличие портов пропускной способностью от 10 Гб/с для подключения коммутаторов агрегации и подсистемы границы сети;

3. Повышенная (от 99,99% времени) доступность устройств;
4. Поддержка протоколов динамической маршрутизации как внутреннего (*OSPF, IS-IS*), так и внешнего шлюза (*BGP*).

8.1.9. Домен отказа

В процессе проектирования и построения сети необходимо обращать внимание не только на функции, которые выполняет устройство, но также и на последствия, которые могут возникнуть при его выходе из строя, потере связности или отказе какой-либо функции (например, подсистемы *PoE*), также из-за возникновения обстоятельств, не связанных напрямую с данным сетевым устройством, например, при отключении электропитания в здании.

Группа устройств, на которые может повлиять отказ сетевого устройства или сетевого сервиса, составляет так называемый «домен отказа». Чем больше потенциальный домен отказа, тем больше внимания необходимо уделять для его сокращения или устранения.

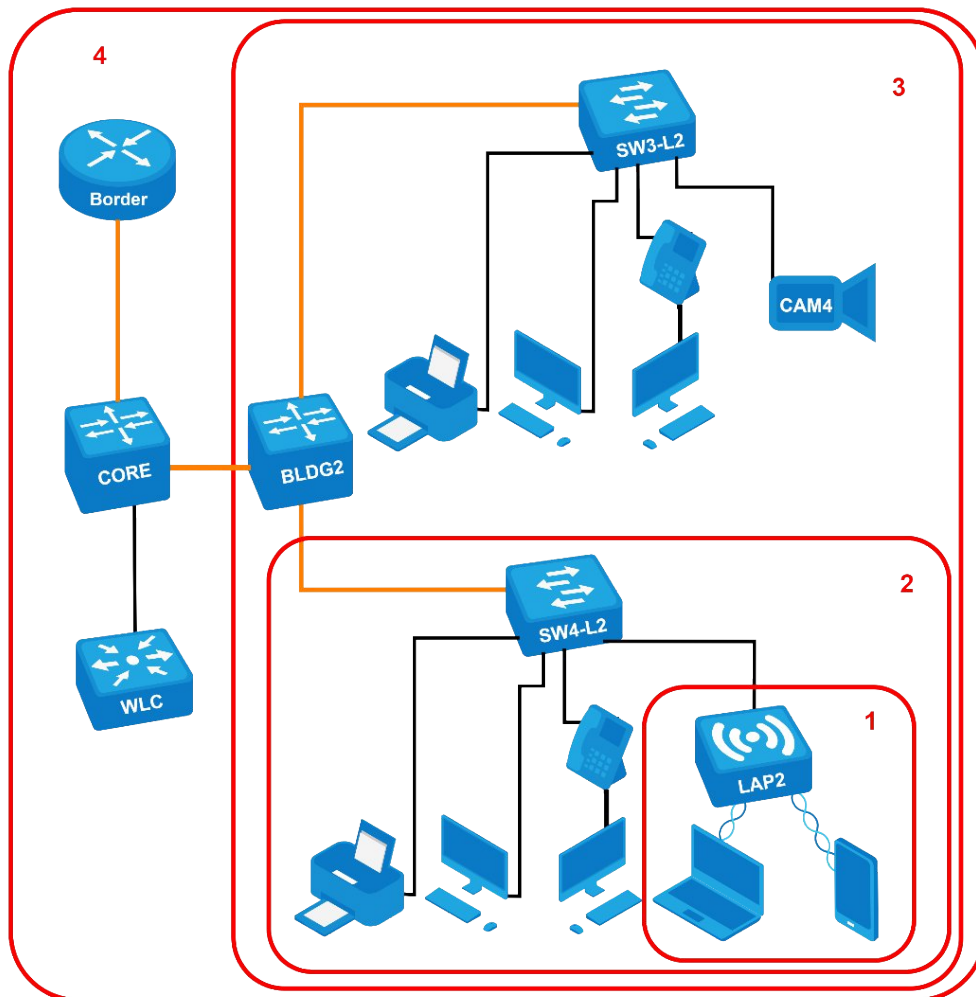


Рисунок 8.1.9.1. – Домены отказа точки доступа (1), коммутатора доступа (2), коммутатора агрегации (3) и коммутатора ядра (4)

Домен отказа (от англ. *fault domain*) – это совокупность сетевых и оконечных устройств, которые могут потерять доступ к сервисам сети при отказе определенного устройства.

На рисунке 8.1.9.1. обозначены различные варианты доменов отказа. Например,

при отказе точки доступа LAP2 (домен 1) без подключения к сети остаются два устройства. При отказе коммутатора доступа (домен 2) без подключения к сети остаются все оконечные устройства в данном сегменте сети. При отказе коммутатора агрегации BLDG2 (домен 3) сегменты доступа потеряют связность друг с другом и с внешними сетями, но внутри сегмента связность не потеряется, то есть устройства внутри одной и той же VLAN в пределах коммутатора продолжают передачу трафика друг другу. При отказе коммутатора ядра CORE (домен 4) сеть здания 2 продолжит функционировать, но потеряет связность с другими сетями предприятия и Интернетом. Также произойдет отказ беспроводной сети, так как точки доступа потеряют связность с контроллером беспроводной сети WLC. К такому же результату приведет выход из строя самого контроллера беспроводной сети WLC.

Домены отказа можно рассчитать не только для сетевых устройств, но и для каналов передачи данных между ними. Так, например, повреждение оптического кабеля между коммутаторами SW4-L2 и BLDG2 для устройств, подключенных к коммутатору доступа, будет равносильно отказу коммутатора агрегации, то есть вышедший из строя кабель приведет к таким же экономическим последствиям, что и вышедший из строя коммутатор, хотя стоимость последнего может в несколько сотен раз превышать стоимость кабеля. Следовательно, при планировании доменов отказа необходимо уделять пристальное внимание не только устройствам, но и каналам связи между ними.

Основная цель управления доменами отказа (fault domain management) – это их сокращение или, по возможности, устранение.

8.1.10. Масштабируемость сети

Наиболее простыми решениями по уменьшению потерь при отказе сегмента сети являются следующие:

1. Дублирование сетевых устройств;
2. Дублирование каналов передачи данных между устройствами.

Несмотря на кажущуюся очевидность таких подходов, реализация их в чистом виде может привести к ухудшению (деградации) или полному прекращению доступа к сетевым сервисам.

Например, подключение двух каналов от коммутатора доступа к коммутатору агрегации может привести не к увеличению пропускной способности, а к широкополосному или одноадресному шторму и блокировке сети.

Внедрение двух маршрутизаторов на границе сети без конфигурации соответствующих протоколов может привести либо к простоя одного из маршрутизаторов, либо к большим «накладным расходам» при пересылке трафика.

Способность сети к расширению без ухудшения доступа текущих пользователей к сетевым сервисам называют **масштабируемостью сети**. **Масштабируемость сети (network scalability)** – способность сети обеспечивать доступ текущих пользователей к потребляемым сервисам на том же или лучшем уровне при увеличении количества устройств в сети.

Масштабируемость, как и все прочие свойства вычислительных сетей, диктуется экономическими предпочтениями организации-эксплуатанта сети. В сетях малого и

среднего бизнеса масштабируемость не всегда является приоритетом, так как сама возможность расширения организации не всегда рассматривается владельцами как необходимое условие ведения бизнеса.

Сети крупных предприятий, особенно изначально строящиеся с применением иерархической архитектуры, обладают более высокой способностью к масштабированию. При этом устройства с более «высоких уровней» иерархической сети владелец всегда может использовать на более «низких» уровнях, таким образом обеспечивая не только техническую устойчивость, но и защиту инвестиций в IT.

Одним из примеров обеспечения масштабируемости сети является применение коммутатора ядра. Без его использования расширение сети ограничивается ёмкостью портов коммутаторов агрегации, которые пользователь может «забронировать» для подключения к другим коммутаторам агрегации.

Рассмотрим уже знакомый нам пример сети группы зданий, но без коммутатора ядра.

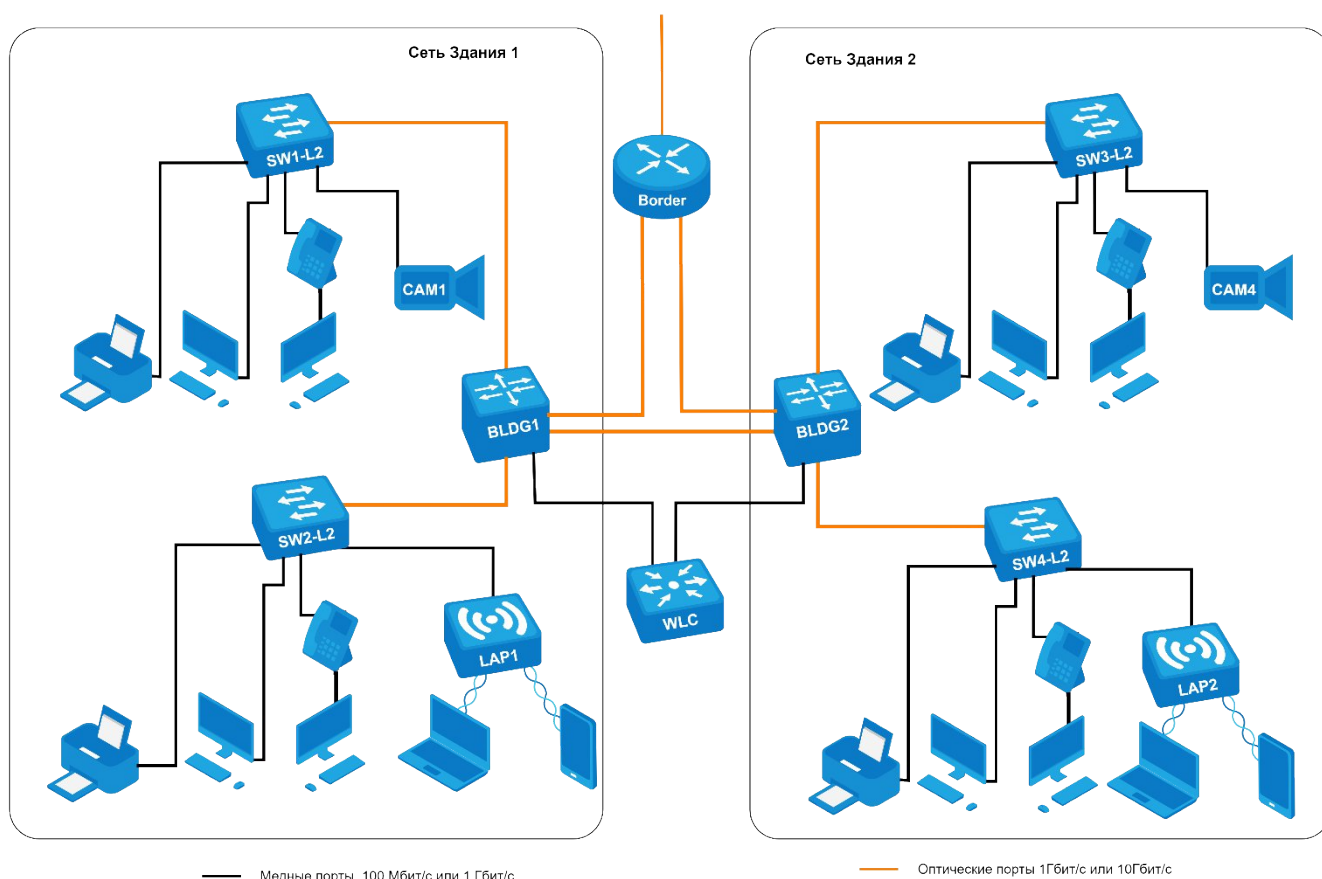


Рисунок 8.1.10.1. – Сеть группы зданий без коммутатора ядра

Во-первых, увеличилось количество каналов передачи данных между коммутаторами агрегации. Во-вторых, увеличилось количество занятых портов на маршрутизаторе Border и контроллере беспроводной сети WLC. При этом наличие дополнительных портов на этих устройствах уже не очевидно (обычно маршрутизатор «располагает» от 2 до 4 портов L3), что ставит под сомнение возможность подключения к данной сети уже третьего здания, не говоря о дальнейшем расширении.

Возможности масштабирования на уровне доступа обычно ограничены количеством портов коммутаторов доступа. В этой связи хорошим «подспорьем» для масштабируемости сети может служить переход на беспроводные сети как основной

метод получения сетевых сервисов. В этом случае при подключении к имеющемуся порту не одного оконечного устройства, а точки доступа, можно подключить к сети не одного, а несколько пользователей (разумеется, при соблюдении всех правил планирования радиозоферы, который является разделяемой средой и коллизийным доменом).

8.1.11. Резервирование и избыточность

Резервирование является главным инструментом борьбы с доменами отказа в сети. **Резервирование** – увеличение количества сетевых устройств или каналов передачи данных для устранения или уменьшения доменов отказа.

Избыточность – наличие в сети более одного устройства в каждом сегменте сети. То есть наличие двух коммутаторов доступа в одной сети не означает присутствие избыточности, а наличие двух коммутаторов агрегации в одном сетевом сегменте означает.

Как уже говорилось выше, резервирование не может производиться путем простого увеличения количества сетевых устройств или каналов передачи данных, практически всегда оно требует внедрения соответствующих технологий, направленных на обеспечение работоспособности сети.

Например, наличие всего одного аплинка между коммутатором доступа и коммутатором агрегации составляет домен отказа. Но если мы увеличим количество аплинков до двух, это не только не приведет к увеличению надежности сети, но и наоборот, создаст петлю с последующим развитием сетевого шторма, который со своей стороны приведет к отказу обоих аплинков.

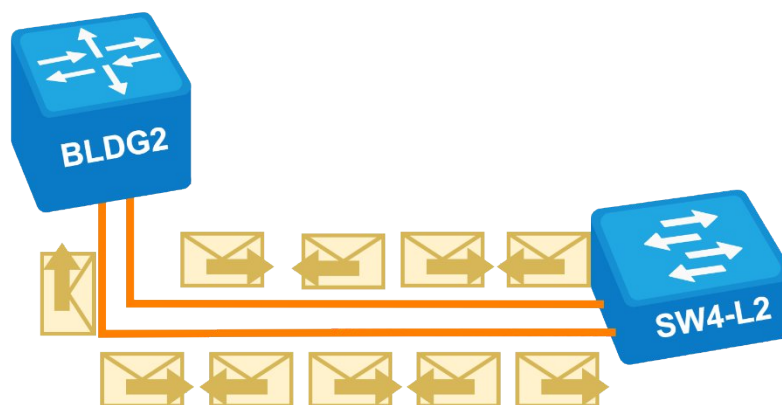


Рисунок 8.1.11.1. — Образование петли между коммутаторами в результате неверного резервирования

Ниже будут рассмотрены некоторые примеры резервирования на уровнях L1, L2 и L3.

8.1.12. Резервирование точек доступа

С ростом популярности беспроводных сетей одним из самых востребованных методов обеспечения надежности подключения устройств к сети становится резервирование беспроводных соединений и точек доступа.

При этом необходимо не просто увеличивать количество точек доступа, но и осуществлять правильное радиопланирование, так как две точки доступа, работающие в

одном радиоканале, будут создавать интерференции, которые приведут к взаимному гашению сигнала и ухудшению качества передачи информации.

На рисунке 8.1.12.1. показана ситуация правильного подхода к резервированию беспроводного подключения. Компьютеры изначально подключаются к неавтономной точке доступа LAP-2, но после ее выхода из строя они теряют сетевое подключение. Однако, так как они находятся одновременно в зоне покрытия точки LAP-1, оба компьютера за короткий срок могут восстановить подключение к сети.

Важным фактором является правильная конфигурация покрытия: точки работают в разных каналах радиочастотного диапазона (1 и 11), избегая взаимных интерференций.

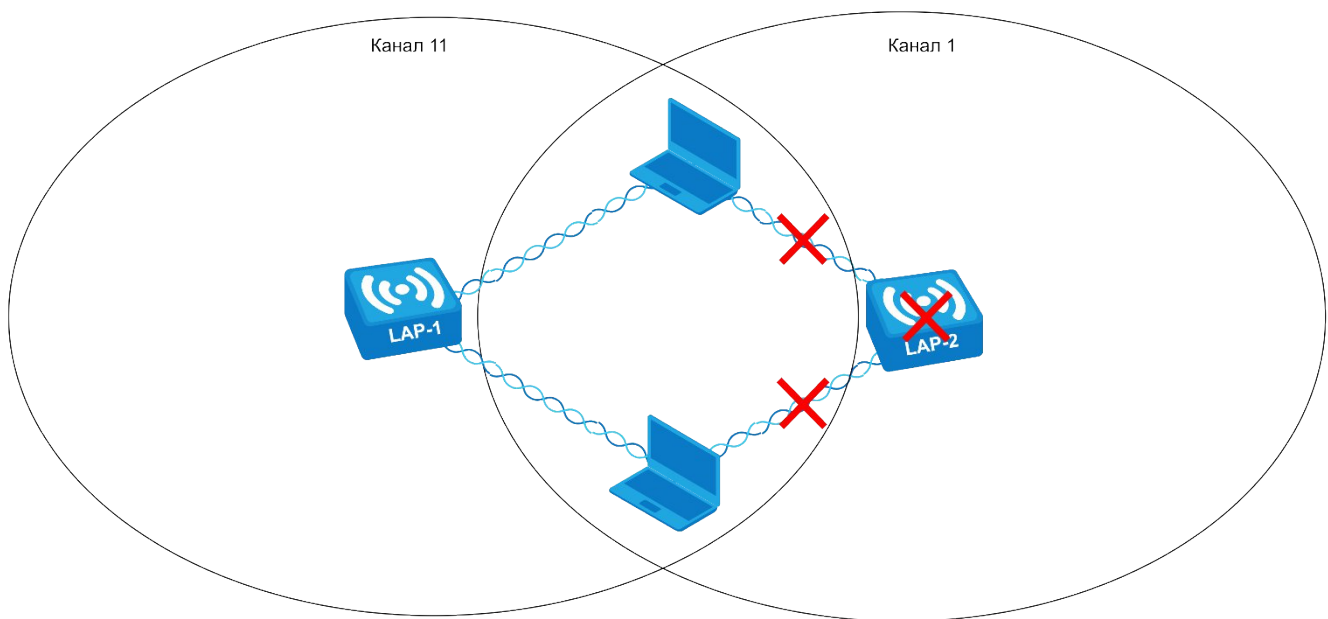


Рисунок 8.1.12.1. – Резервирование точек беспроводного доступа

8.1.13. Резервирование на уровне доступа

Коммутатор доступа, к сожалению, не поддается резервированию, так как не все оконечные устройства комплектуются дополнительными сетевыми картами. Однако можно предпринять меры для его более продолжительной работы, например, резервировать источники питания. Некоторые коммутаторы, кроме обычного источника питания переменного тока, имеют дополнительный разъем для подключения аккумуляторной батареи, которая берет на себя функции источника питания при отключении сети переменного тока.

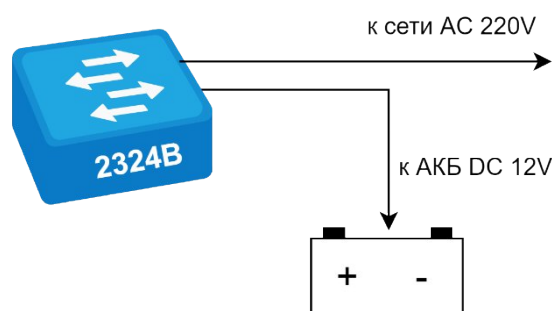


Рисунок 8.1.13.1. – Резервирование источников питания коммутатора доступа

Приведенное выше утверждение не относится к уровню доступа сетей центров

обработки данных: в них оконечными устройствами являются серверы, которые в обязательном порядке комплектуются двумя сетевыми интерфейсами для максимального облегчения резервирования и обеспечения высокого уровня доступности.

8.1.14. Резервирование на уровне агрегации

Резервирование на уровне агрегации осуществляется по следующим направлениям:

1. Дублирование коммутаторов агрегации;
2. Логическое объединение дублированных устройств для облегчения управления (стекирование или межкоммутаторная агрегация каналов – MLAG);
3. Дубликация каналов передачи данных как в направлении уровня ядра (аплинков), так и в направлении уровня доступа (даунлинков);
4. Резервирование блоков питания.

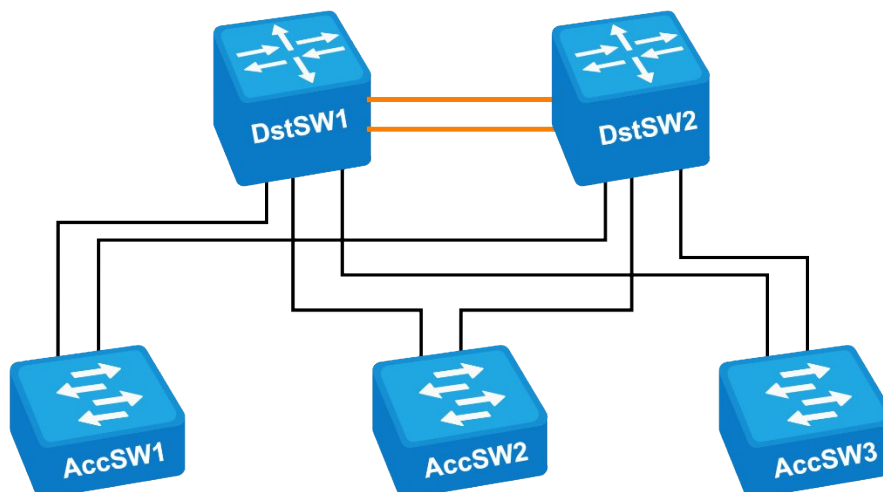


Рисунок 8.1.14.1. — Резервирование на уровне агрегации. Физическая топология

Рассмотрим физическую топологию осуществленного таким образом резервирования на Рис. 8.1.14.1.

Три коммутатора доступа AccSW1-3 подключены к двум коммутаторам агрегации DstSW1-2, причем каждый коммутатор доступа подключен одним каналом к каждому коммутатору агрегации. Сами коммутаторы агрегации соединены двумя каналами передачи данных. По начальному замыслу такая организация сети приведет к тому, что при выходе из строя одного коммутатора агрегации второй будет продолжать коммутировать трафик, поступающий с уровня доступа.

Нетрудно заметить, что такая топология содержит множественные петли, которые приведут к образованию штормов, а значит сеть будет неработоспособна.

Для снижения вероятности возникновения штормов необходимо произвести два действия:

1. Объединить коммутаторы агрегации в стек (от англ. *stack* – *стог, штабель*);
2. Объединить каналы коммутации в логические группы (*Link Aggregation Group, LAG*).

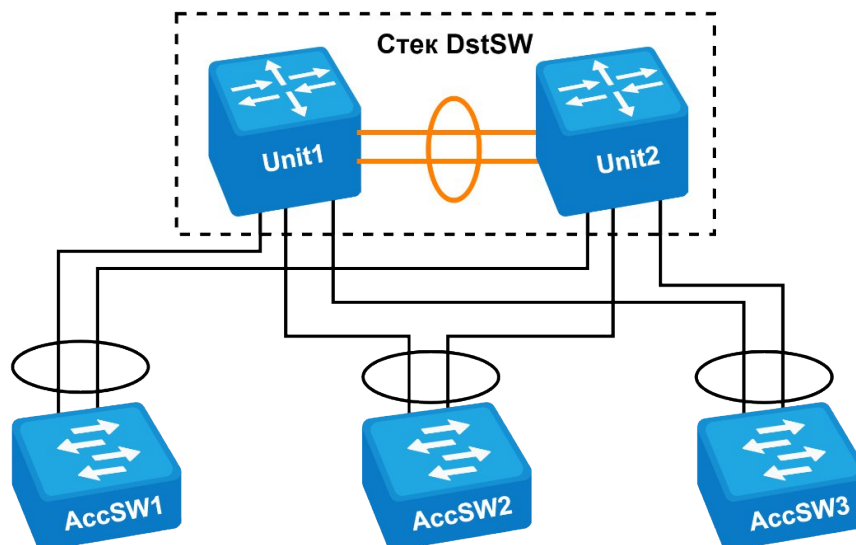


Рисунок 8.1.14.2. – Стекирование коммутаторов и объединение каналов для резервирования на уровне агрегации

При стекировании коммутаторы объединяются в одно логическое устройство с одним MAC-адресом и одним IP-адресом управления. Внутри стека коммутаторы называются «секциями» или «юнитами». Для прочих сетевых устройств и для администратора стек коммутаторов агрегации выглядит как единый коммутатор с расширенным количеством портов.

Логическая топология уровня агрегации с применением стекирования будет выглядеть так, как показано на рисунке 8.1.14.3.

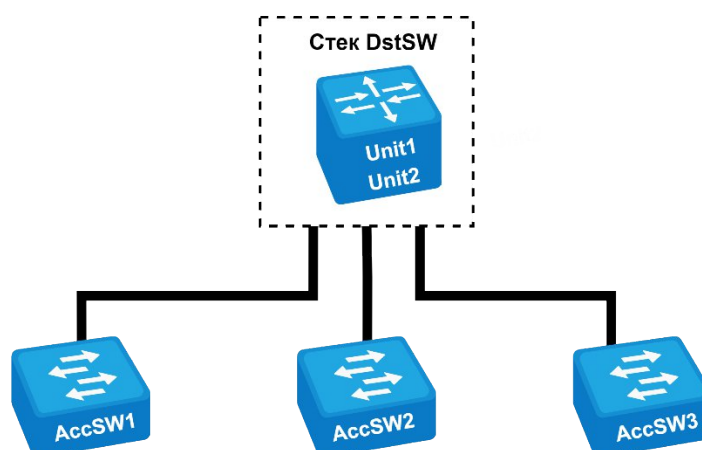


Рисунок 8.1.14.3. – Логическая топология сети с объединенными каналами передачи данных и стекированием на уровне агрегации

8.1.15. Резервирование и избыточность на уровне ядра

На уровне ядра резервирование осуществляется аналогично уровню агрегации:

1. Дубликация коммутаторов ядра;
2. Логическое объединение дублицированных устройств для облегчения управления (MLAG);
3. Дубликация каналов передачи данных как в направлении уровня агрегации, так и в направлении подсистемы границы сети (маршрутизаторы, межсетевые экраны, и

т.д.);

4. Резервирование блоков питания.

Вместо стекирования на уровне ядра используется другая технология объединения коммутаторов в единое виртуальное устройство – *Multi-Chassis Link Aggregation (MLAG)*³. В отличие от стека, где один юнит перехватывает управление всеми остальными, в технологии MLAG коммутаторы координируют свои действия только в разрезе агрегации каналов, образуя виртуальную группу каналов (*Virtual Port Channel, VPC*). В остальном они продолжают функционировать как независимые устройства, что необходимо, например, для резервирования протоколов маршрутизации.

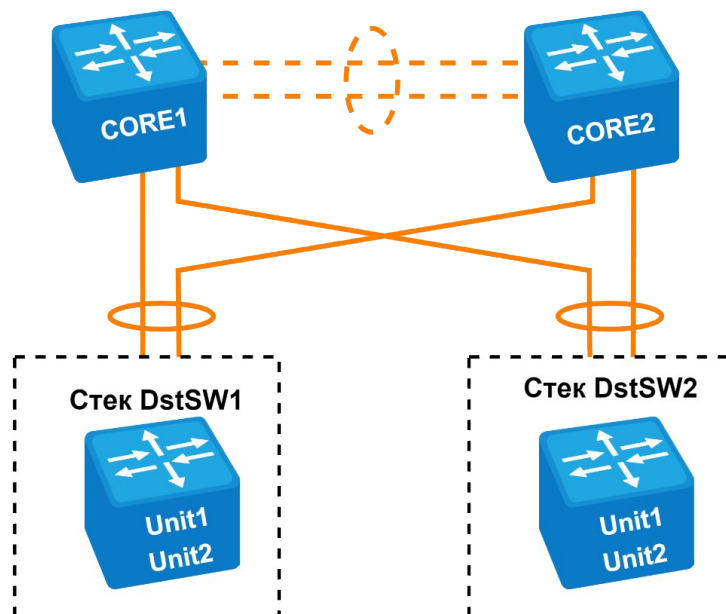


Рисунок 8.1.15.1. – Подключение стеков агрегации к коммутаторам ядра резервными каналами связи

На рисунке 8.1.15.1. изображена физическая топология подключения стекированных коммутаторов агрегации к коммутаторам ядра. Логическая же топология с «точки зрения» коммутаторов агрегации выглядит следующим образом:

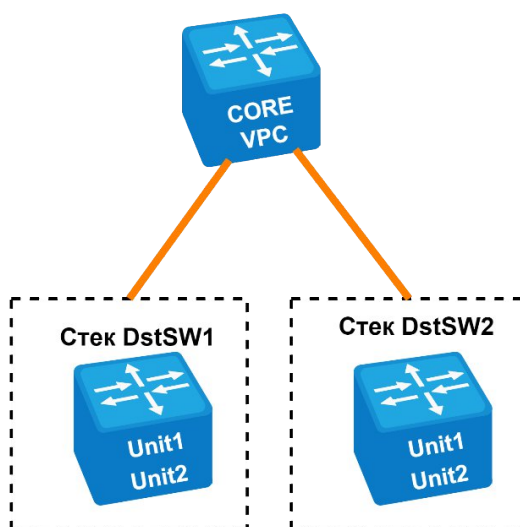


Рисунок 8.1.15.2. – Подключение стекированных коммутаторов агрегации к ядру. Логическая топология

³ Подробно в данном курсе не рассматривается.

8.1.16. Иерархическая сеть с резервированием

Рассмотрим сеть на рисунке 8.1.16.1., в которой зарезервированы основные компоненты.

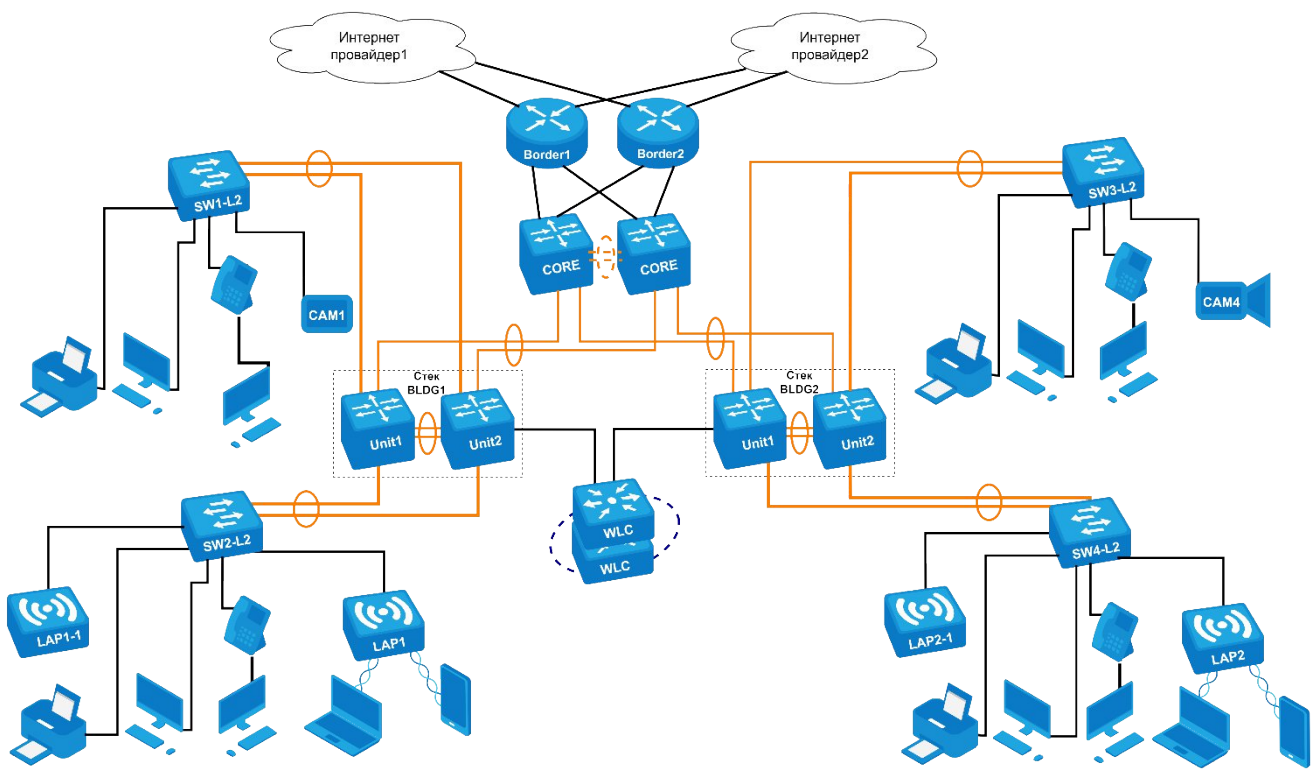


Рисунок 8.1.16.1. – Иерархическая сеть с резервированием

На уровне доступа в двух подсетях зарезервированы точки доступа, на которых должны быть настроены неперекрывающиеся каналы Wi-Fi-диапазона. Это сделано для того, чтобы при выходе из строя одной из точек доступа оконечные устройства не теряли подключение к сети.

Каждый коммутатор уровня доступа подключен объединенными каналами к коммутаторам уровня агрегации, которые собраны в стеки. Стек необходим для сохранения сетевых сервисов для устройств уровня доступа при выходе из строя другого коммутатора в стеке.

Каждый стек подключен двумя объединенными каналами передачи данных к обоим коммутаторам ядра.

К коммутаторам ядра подключена подсистема границы сети, представленная двумя маршрутизаторами Border1 и Border2. Маршрутизаторы используются как для резервирования, так и для балансировки нагрузки пересылки трафика в другие сети посредством сети Интернет. В нашем примере подключение осуществляется по наиболее защищенному от отказов способу: подключение к двум разным Интернет-провайдерам по резервным каналам связи (*dual-homed multihoming*).

Последним из резервных компонентов, который можно отметить на этой схеме, является кластер контроллеров беспроводной сети (*WLC cluster*). Кластер состоит из двух или более аппаратных контроллеров, которые работают под единым управлением и подключены к разным стекам коммутаторов уровня агрегации.

Далее в этой главе мы рассмотрим главные протоколы, необходимые для

резервирования и масштабирования сети:

1. Протокол Spanning Tree, необходимый для избежания возникновения петель на уровне доступа и агрегации;
2. Протокол LACP, необходимый для объединения нескольких физических каналов передачи данных в один для достижения отказоустойчивости и повышения пропускной способности;
3. Протокол VRRP, необходимый для обеспечения резервирования шлюза по умолчанию, и применяемых как на уровне агрегации, так и в подсистеме границы сети.

8.2. Протокол связующего дерева (Spanning Tree Protocol, STP)

8.2.1. Проблемы при образовании петель на уровне L2

При построении сетей на уровне L2 нередко ситуации, когда между коммутаторами образуются так называемые петли.

Рассмотрим пример топологии, где компьютер PC-A с MAC-адресом AA:AA:AA:AA:AA:AA отправляет кадр компьютеру PC-B с MAC-адресом BB:BB:BB:BB:BB:BB по сети, состоящей из коммутаторов SW-1, SW-2, SW-3 и SW-4. В целях достижения максимальной ясности предположим, что таблицы MAC-адресов на всех коммутаторах являются незаполненными, то есть в них не содержится изученных адресов.

На первом этапе кадр от PC-A приходит на коммутатор SW-1, и тот изучает MAC-адрес компьютера на порту Gi0/1.

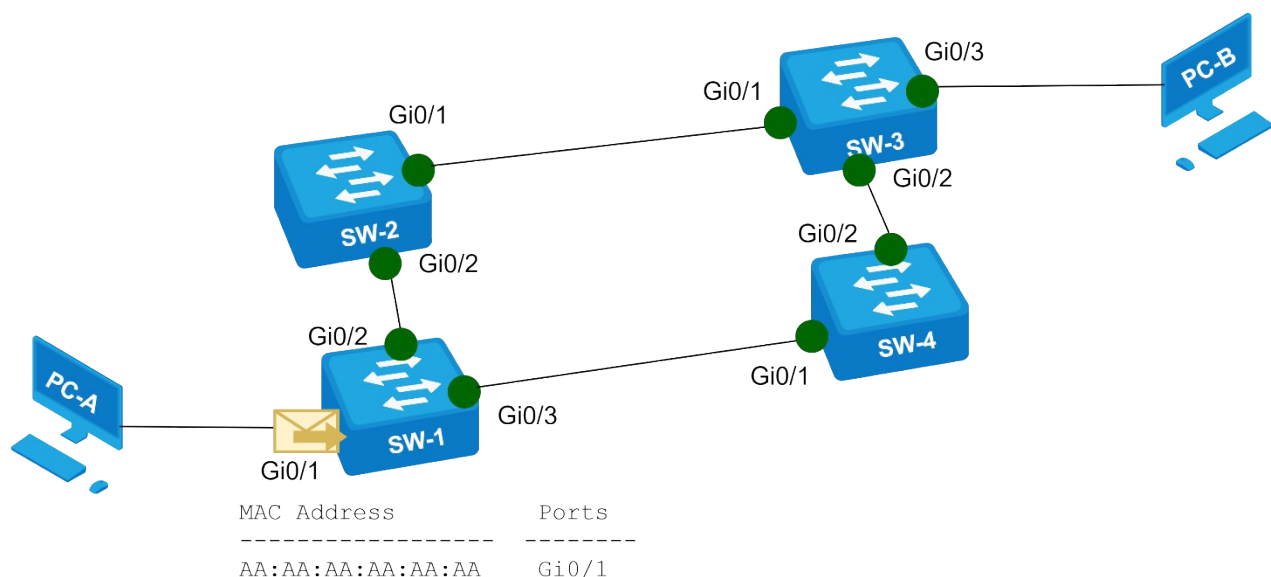


Рисунок 8.2.1.1. – Кадр поступает на SW-1

Вспомним, как коммутатор принимает решение о пересылке кадра:

1. При поступлении кадра на порт коммутатор записывает MAC-адрес источника в таблицу коммутации (изучает MAC-адрес);
2. Если MAC-адрес назначения изучен в таблице коммутации на интерфейсе, кадр копируется в этот интерфейс;

3. Если MAC-адрес назначения не изучен ни одним интерфейсом, то коммутатор копирует его во все интерфейсы, кроме того, на который кадр был получен. Такой метод рассылки называется «одноадресная рассылка на неизвестный адрес» (*unknown unicast*);

4. При изучении MAC-адреса, уже существующего в таблице коммутации на другом порту, коммутатор переписывает таблицу коммутации.

В нашем случае кадр будет скопирован в порты Gi0/3 и Gi0/2 с последующим получением коммутаторами SW-4 и SW-2 соответственно.

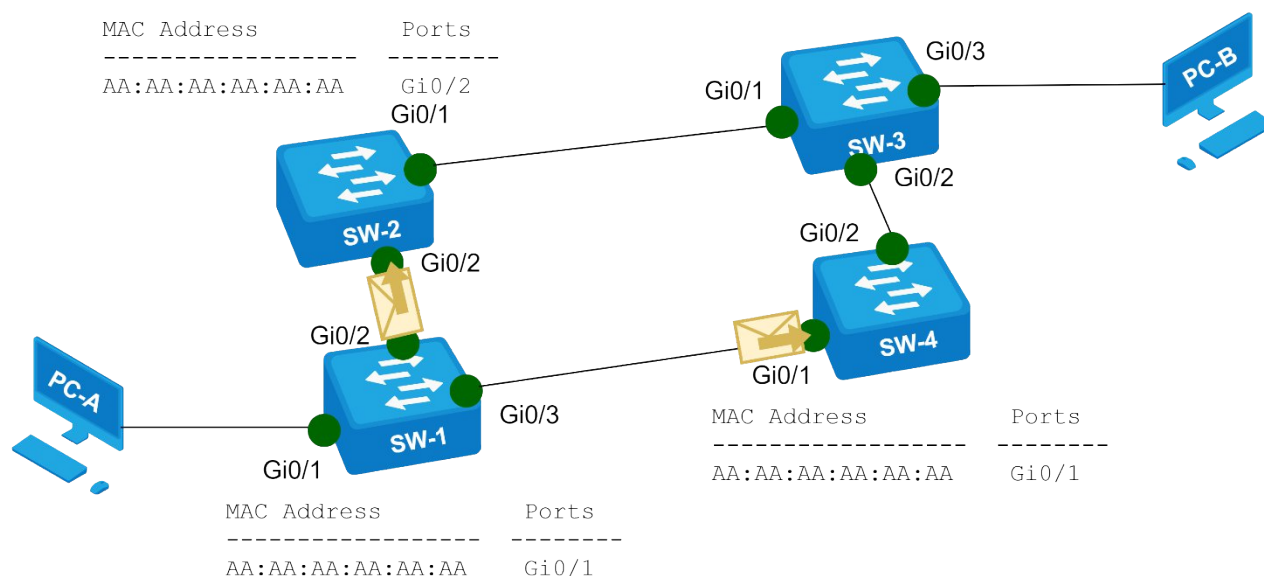


Рисунок 8.2.1.2. – Кадр копируется на порты и приходит на SW-2 и SW-4

На коммутаторах SW-2 MAC-адрес компьютера PC-A изучается на порту Gi0/2, а на коммутаторе SW-4 – на порту Gi0/1, соответственно, появляются и записи в таблицах MAC-адресов.

Коммутаторы SW-2 и SW-4 при отправке кадров руководствуются абсолютно теми же правилами, что и SW-1, то есть копируют кадр во все порты, кроме порта получения.

А вот с получением кадров от SW-4 и SW-2 на SW-3 начинается самое интересное.

Допустим, первым поступает кадр со стороны SW-4, и его MAC-адрес источника изучается на порту Gi0/2. Тогда копии кадра будут высланы из портов Gi0/1 и Gi0/3 (с последнего кадр наконец-то попадет к адресату, но его приключения на этом не закончатся).

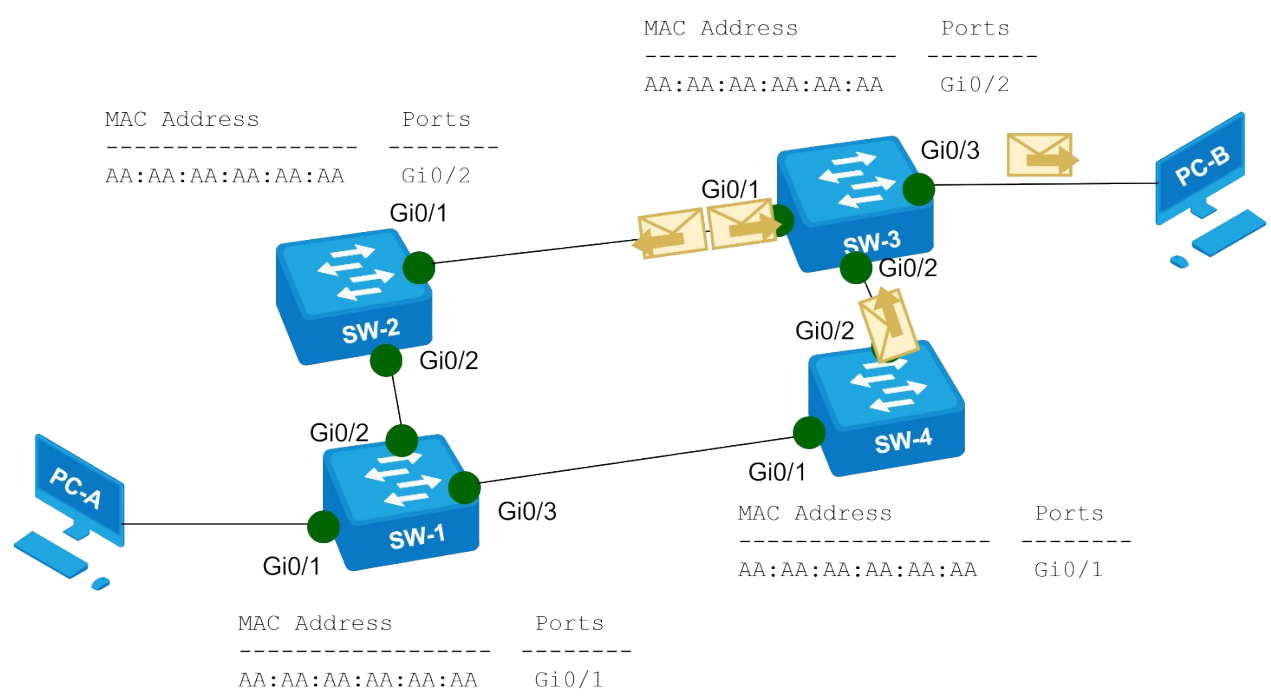


Рисунок 8.2.1.3. – Кадр приходит на SW-3 с двух разных портов

Тут же коммутатор SW-3 получает еще одну копию кадра, которая все это время «шла» с SW-2. MAC-адрес источника переизучается на порту Gi0/1, а копии кадра отправляются в остальные активные порты. В это же время копия того же кадра, высланная с коммутатора SW-3, уже доходит до SW-2 и переписывает ему таблицу MAC-адресов.

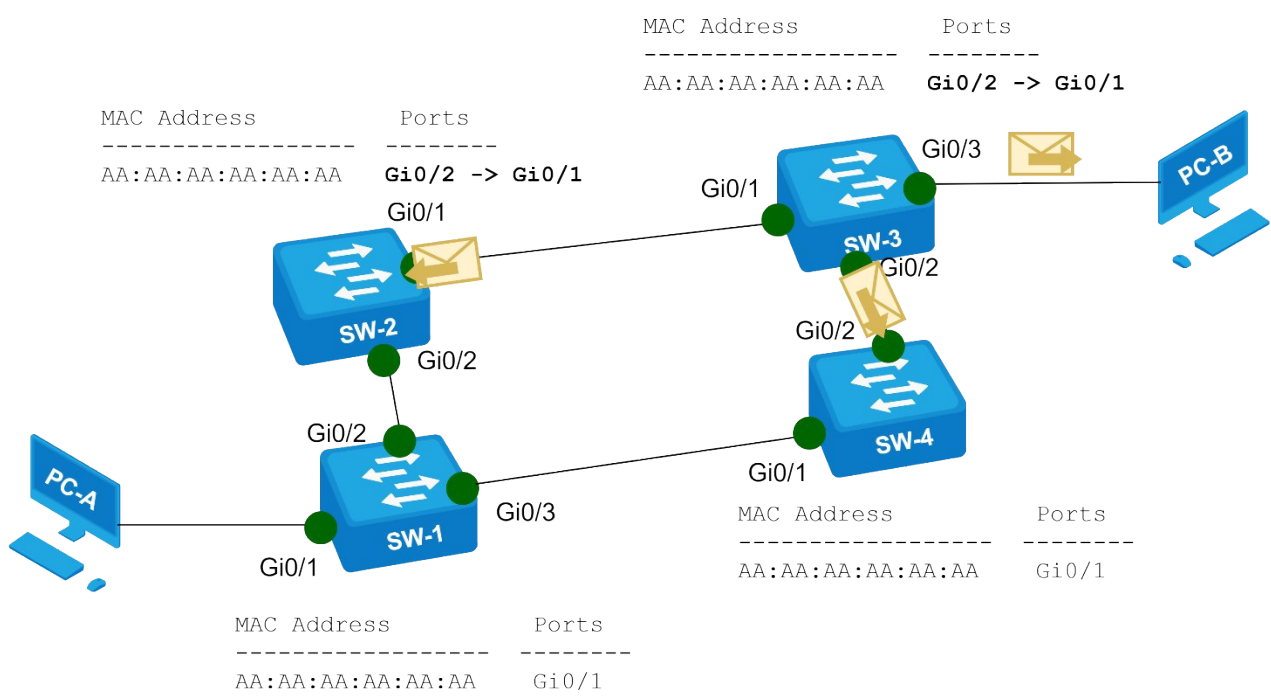


Рисунок 8.2.1.4. – Порты в таблице коммутации изменились

Заметьте, по сети циркулируют уже две копии кадра, которые и будут продолжать это занятие, так как механизма самоуничтожения, подобного TTL в IPv4, на уровне L2 не

предусмотрено.

На следующей итерации кадры с двух сторон поступают на коммутатор SW-1:

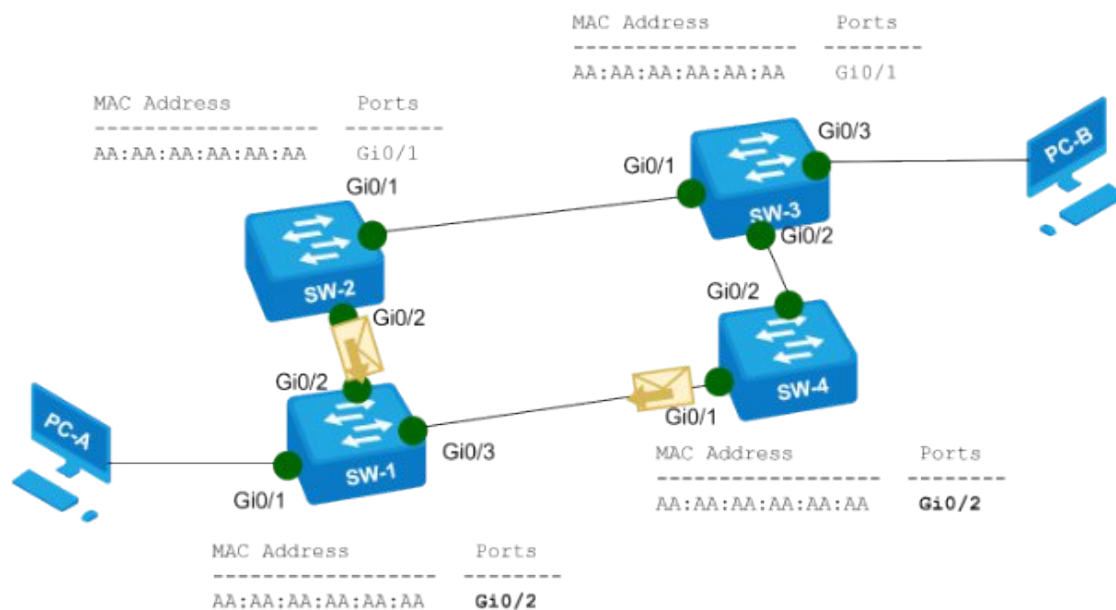


Рисунок 8.2.1.5. – Кадры приходят на SW-1

Здесь происходит очередной раунд размножения кадров. К этому времени вполне возможно, компьютер PC-B начинает передавать ответные кадры, которые еще более занимают полосу пропускания.

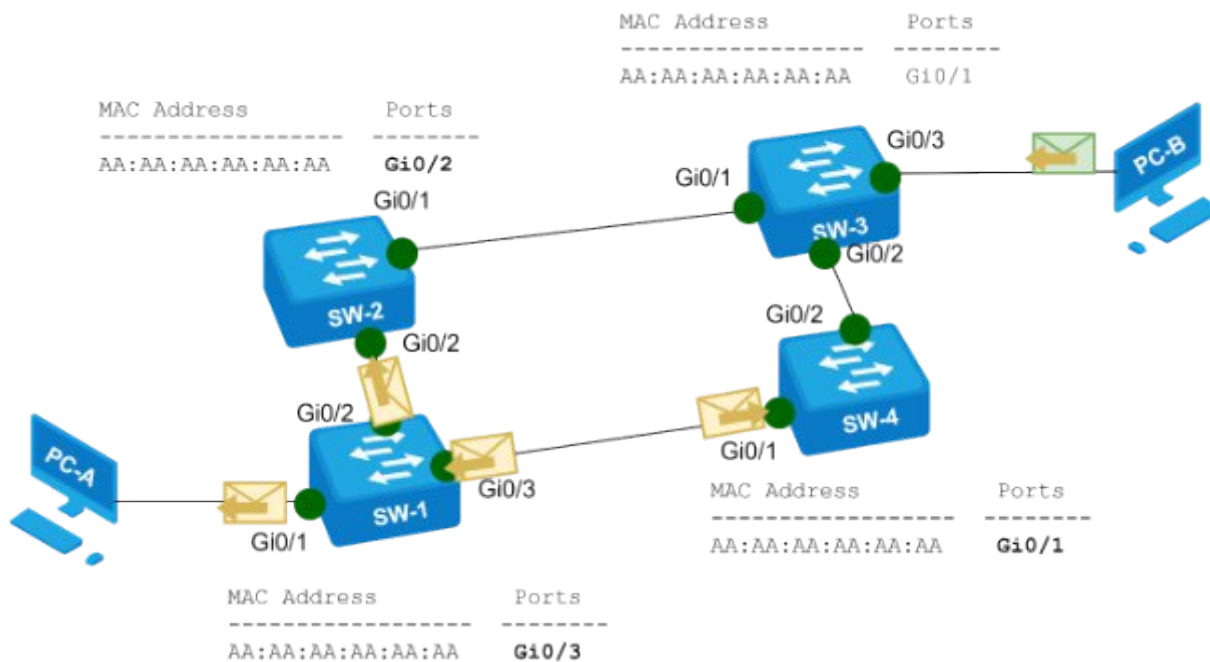


Рисунок 8.2.1.6. – Очередное размножение кадров

И да, если вы считали, что компьютер PC-B сильно удивился, получив подряд два пакета от компьютера PC-A, то только представьте, как удивится компьютер PC-A, начав получать пакеты от себя самого.

Все перечисленные выше события произошли менее чем за секунду, причем в совершенно «стерильных» условиях. Экстраполировав их на период времени хотя бы в минуту, можно совершенно справедливо прийти к выводу, что излишний трафик в такой сети будет лавинообразно нарастать, пока целиком не займет полосу пропускания всех каналов, после чего сеть станет неработоспособной. Подобное явление называется штормом (в нашем случае – одноадресным, ибо широковещательный шторм, например, от единственного высланного в сеть пакета протокола ARP, приводит к неработоспособности сети на порядок быстрее). Причиной возникновения штормов в нашем случае является переизучение (*re-learning*) MAC-адресов на разных портах коммутатора, приводящее к нестабильности таблицы MAC-адресов.

8.2.2. Предотвращение петель на уровне L2 и протокол STP

Давайте предположим, что в нашей топологии произошло изменение: на одном из портов отключена возможность получать и пересылать кадры.

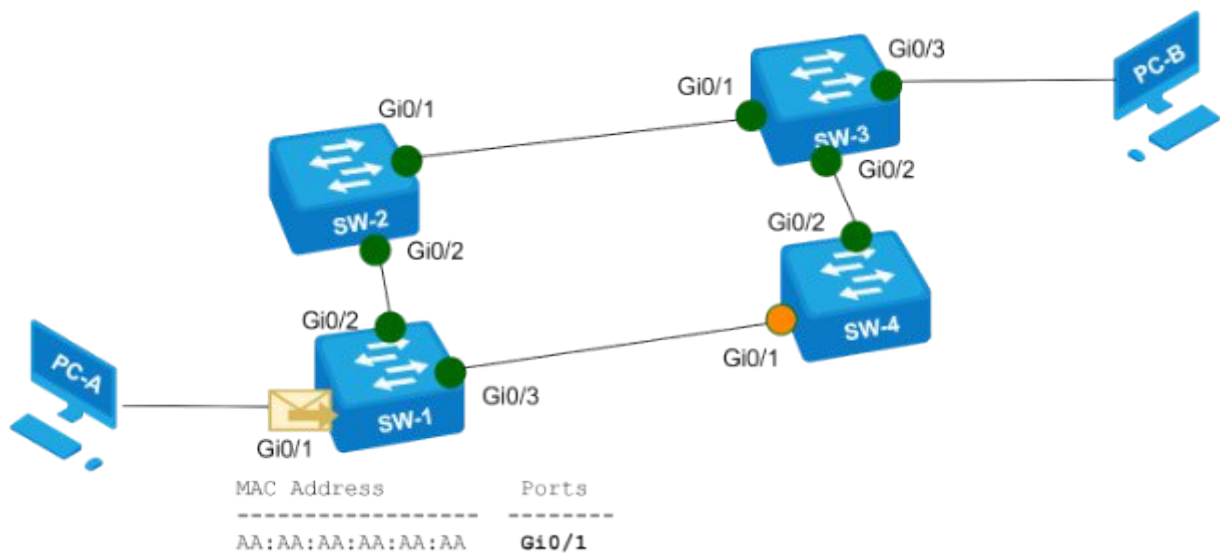


Рисунок 8.2.2.1. – Кадр приходит на SW-1

На первом этапе все происходит точно так же, как и в первом примере: PC-A отправляет кадр по направлению к PC-B, коммутатор SW-1 изучает MAC-адрес источника на порту Gi0/1 и затем копирует кадр в порты Gi0/1 и Gi0/3. Но на втором этапе все кардинально меняется.

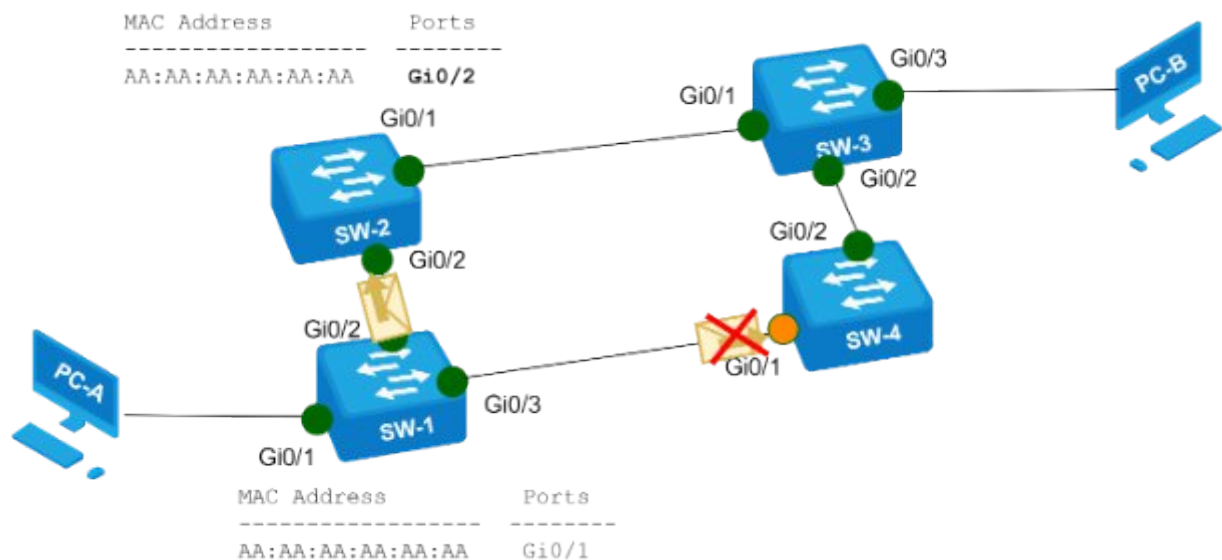


Рисунок 8.2.2.2. – Кадр рассылается на интерфейсы и приходит только на SW-2, т.к. его порт отключен

Кадр, высланный из порта Gi0/3, отбрасывается на порту коммутатора SW-4 и в дальнейшем процессе передачи не участвует. Соответственно, лавинообразного увеличения трафика не происходит.

Этот способ предотвращения образования петель был предложен Радией Перлман (Radia Perlman) в 1985 году. В то время она работала в компании Digital Equipment Corporation (DEC), в которой незадолго до того было изобретено устройство, позволявшее разделять коллизионные домены – **мост (bridge)**. Перлман довольно быстро поняла, что именно нужно сделать: разорвать «кольцо» и добиться того, чтобы сеть представляла собой древовидный граф с одним общим «корнем», через который и будет проходить трафик, во избежание петель.

Следующим шагом была разработка алгоритма определения мостами портов для последующей блокировки.

8.2.3. Выборы корневого моста

Для построения древовидной структуры необходимо определить корневой мост. Для этого мосты организуют «выборы» (*election*) корневого моста (*root bridge*). Для этого каждый из мостов самостоятельно формирует себе идентификатор моста (*BridgeID*). Идентификатор состоит из трех логических частей:

1. Приоритет, 4 бита;
2. Расширенный ID системы (VLAN или экземпляра), 12 бит;
3. MAC-адрес коммутатора, 48 бит.

Итого: 64 бита или 8 байтов.

Приоритет моста задается крайними левыми 4 битами, поэтому он принимает значение от 0 (все нули во всех 16 полях) до 65535 (все единицы в 16 полях, т.е. $2^{16}-1$).

Приоритет				Расширенный ID системы											
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Рисунок 8.2.3.1. – Состав поля приоритета моста (Bridge Priority)

По умолчанию приоритет равен 32768 (значение показано на рисунке выше). Шаг изменения приоритета равен 4096 (2^{12}).

После формирования идентификатора мост формирует протокольный блок приоритета моста (*Bridge Protocol Data Unit, BPDU*) и отправляет его из всех своих активных портов на multicast-адрес 01:80:C2:00:00:00.

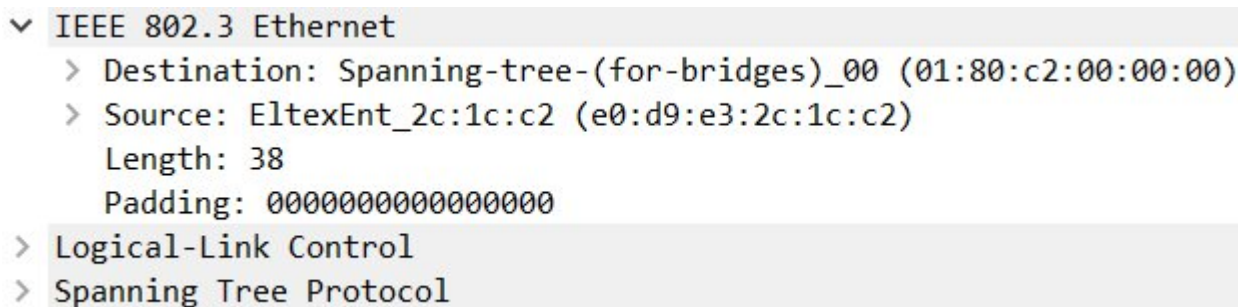


Рисунок 8.2.3.2. – BPDU протокола STP в Wireshark

Кроме идентификатора, BPDU содержит такие параметры, как:

- Идентификатор версии протокола STP (*Protocol Version Identifier*);
- Тип BPDU (*BPDU Type*);
- Идентификатор моста (*Bridge Identifier*);
- Идентификатор порта, с которого был выслан BPDU (*Port Identifier*);
- Идентификатор корневого моста (*Root Identifier*);
- Стоимость пути до корневого моста (*Root Path Cost*);
- Таймеры:
 - Расстояние до «корня» (*Message Age*): каждый последующий коммутатор при пересылке BPDU округляет этот параметр до целой секунды;
 - **Максимальный возраст (Max Age)** – максимальное время жизни BPDU (в секундах), после которого мост считается не участвующим в топологии. Этот же таймер служит для отсчета времени от последнего полученного BPDU для перевода порта из состояния *Blocking* в *Listening*;
 - *Частота отправки Hello BPDU (Hello time)*: по умолчанию BPDU отправляются каждые 2 секунды;
 - **Задержка отправки (Forward Delay)** – промежуток, в течение которого мост удаляет записи о топологии STP при поступлении TCN BPDU. Он же служит для переключения порта из состояния *Listening* в *Learning*, а из *Learning* в *Forwarding*.

```

    Spanning Tree Protocol
    Protocol Identifier: Spanning Tree Protocol (0x0000)
    Protocol Version Identifier: Spanning Tree (0)
    BPDU Type: Configuration (0x00)
    > BPDU flags: 0x00
    > Root Identifier: 32768 / 0 / e0:d9:e3:2c:1c:c0
    Root Path Cost: 0
    > Bridge Identifier: 32768 / 0 / e0:d9:e3:2c:1c:c0
    Port identifier: 0x8032
    Message Age: 0
    Max Age: 20
    Hello Time: 2
    Forward Delay: 15
  
```

Рисунок 8.2.3.3. — Поля BPDU протокола STP в Wireshark

В начале процесса каждый мост считает себя корневым (*Root bridge*), поэтому в отправляемых им BPDU идентификаторы моста и корня совпадают, как, например, на рисунке выше.

Рассмотрим ситуацию: есть четыре коммутатора, которые соединены между собой в некое подобие локальной сети. Коммутатор SW-1 соединен с SW-2 и SW-4 каналами в 10 Гб/с, а SW-3 соединен с ними каналами в 1 Гб/с. Все коммутаторы имеют одинаковый приоритет, и на первом этапе выборов все считают себя корневыми, поэтому с заметным энтузиазмом рассылают BPDU со своими «реквизитами» из всех своих активных портов (мы отметили BPDU номером каждого коммутатора, чтобы было понятно, что вообще происходит).

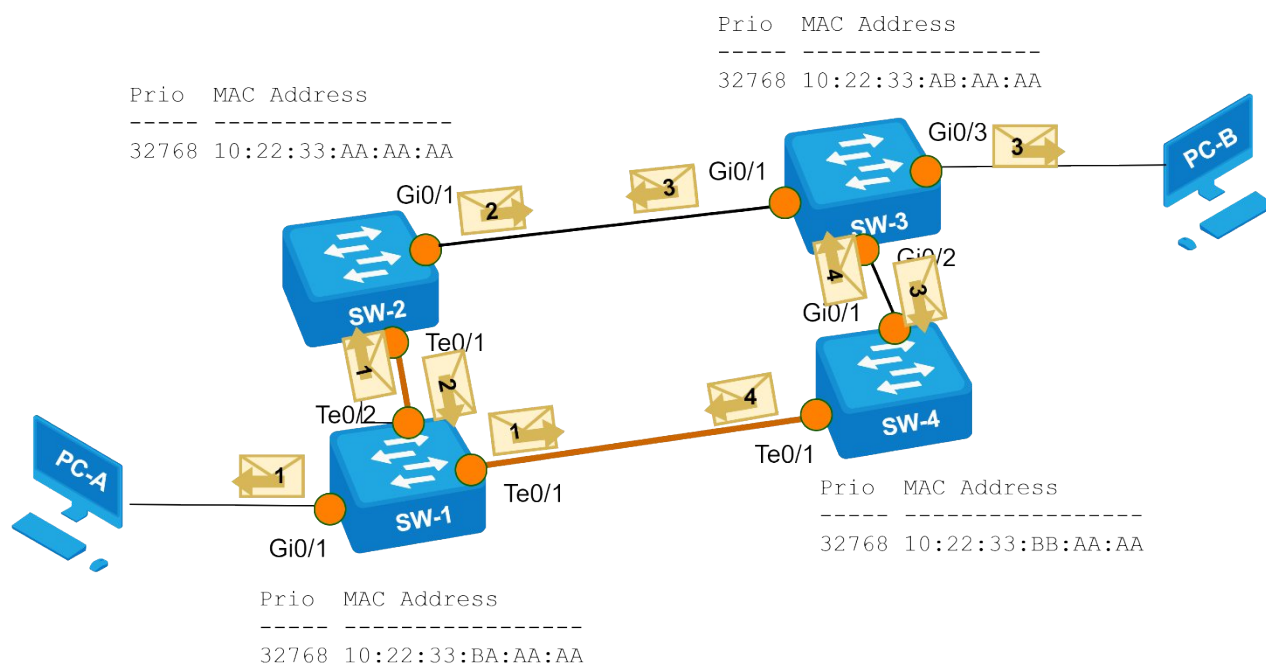


Рисунок 8.2.3.4. — Выборы корневого коммутатора

В нашем примере, очевидно, корневым портом становится SW-2, так как у него наименьший MAC-адрес при равных с другими коммутаторами значении приоритета.

Если мост получает BPDU с BPID меньшим, чем у него самого, то он начинает рассылку BPDU с двумя изменениями:

- В поле *Root Identifier* он записывает меньший идентификатор из полученного BPDU;
- В поле *Root Path Cost* он записывает «стоимость» порта, на который получено сообщение;
- Увеличивает значение Message Age на 1.

▼ Spanning Tree Protocol

```

Protocol Identifier: Spanning Tree Protocol (0x0000)
Protocol Version Identifier: Spanning Tree (0)
BPDU Type: Configuration (0x00)
> BPDU flags: 0x00
> Root Identifier: 32768 / 0 / e0:d9:e3:2c:1c:c0
Root Path Cost: 20000
> Bridge Identifier: 32768 / 0 / e4:5a:d4:8c:cd:d0
Port identifier: 0x8036
Message Age: 1
Max Age: 20
Hello Time: 2
Forward Delay: 15
    
```

Рисунок 8.2.3.5. – Поля BPDU некорневого моста в Wireshark

Выглядит это примерно так:

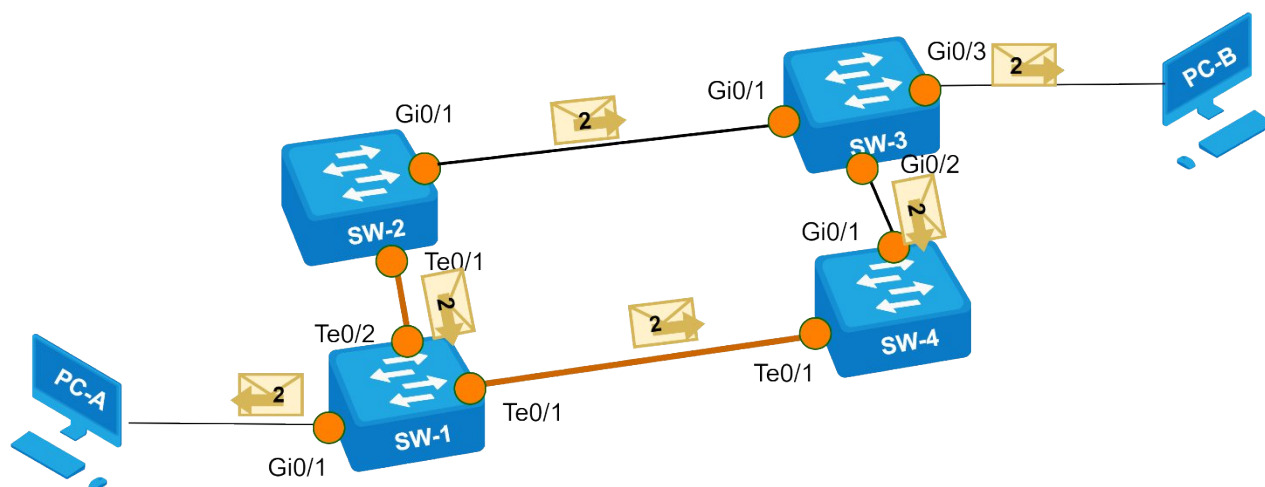


Рисунок 8.2.3.6. – Рассылка BPDU в сети

Сам порт, на который коммутатор получает сообщение от корневого моста, становится «корневым портом» (*Root Port*).

Следующий коммутатор также добавит значение стоимости канала к значению

Root Path Cost и отправит измененный BPDU из всех портов, кроме корневого.

Стоимости портов приведены в Таблице ниже.

Таблица 8.2.3.1. Значения стоимости портов в протоколе STP

№	Пропускная способность	Стоимость 802.1D (Short)	Стоимость 802.1 (Long)
1	100 Мб/с	19	200000
2	1 Гб/с	4	20000
3	10 Гб/с	2	2000
4	40 Гб/с	-	500

То есть значения полей BPDU в нашем примере будут следующими:

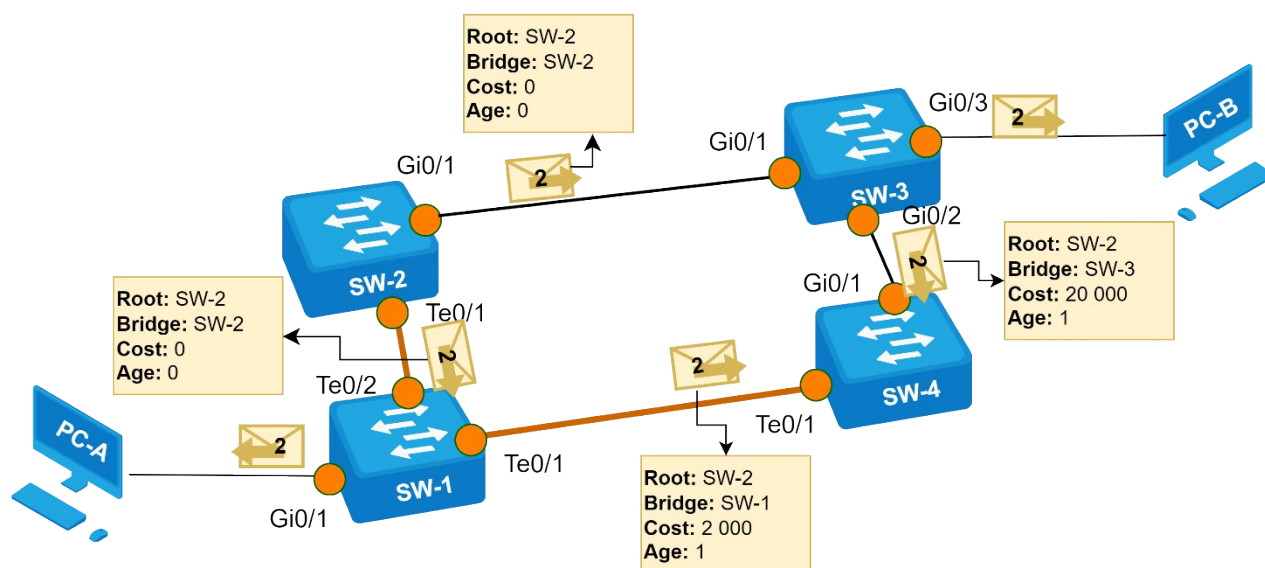


Рисунок 8.2.3.7. Рассылка BPDU в сети

Эти значения впоследствии будут использоваться мостами для того, чтобы решить, какие из портов необходимо блокировать, а каким разрешить пересылку трафика.

8.2.4. Роли портов в протоколе STP

Мы уже говорили о том, что порт, на который коммутатор получает BPDU от корневого моста, становится **корневым портом** (*Root Port*). Его «партнер» по каналу становится **назначенным портом** (*Designated Port*). Логично будет предположить, что все порты корневого моста будут назначенными.

Давайте еще раз вернемся к нашей топологии и отметим, какие из портов будут корневыми, а какие назначенными.

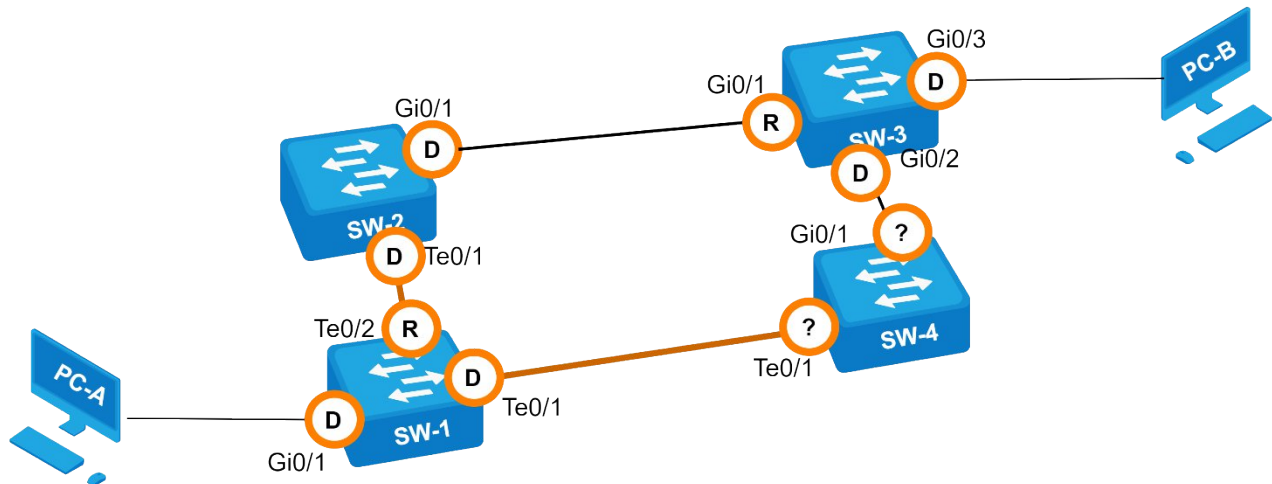


Рисунок 8.2.4.1. –Роли портов в STP после выбора корневого моста

То есть корневыми портами будут те, что «смотрят» в сторону корневого моста. В нашем случае это порты Gi0/1 коммутатора SW-3 и Te0/2 коммутатора SW-1. Назначенными портами, кроме всех портов корневого SW-2, будут порты Te0/1 коммутатора SW-1 и Gi0/2 SW-3.

Посмотрим, как выглядит ситуация с ролями портов на корневом и некорневом мостах в CLI. Состояния портов можно проверить командой `show spanning-tree active`.

```
MES (config)#show spanning-tree active
```

```
Spanning tree enabled mode STP
```

```
-----часть вывода пропущена-----
```

```
Interfaces
```

Name	State	Prio.Nbr	Cost	Sts	Role	PortFast	Type
gil/0/1	enabled	128.49	20000	Frw	Desg	Yes	P2P (STP)
gil/0/5	enabled	128.53	20000	Frw	Desg	Yes	P2P (STP)

На некорневом коммутаторе картина будет следующей:

```
MES (config)#show spanning-tree active
```

```
-----часть вывода пропущена-----
```

```
Interfaces
```

Name	State	Prio.Nbr	Cost	Sts	Role	PortFast	Type
gil/0/3	enabled	128.51	20000	Frw	Root	No	P2P (STP)
gil/0/5	enabled	128.53	20000	Frw	Desg	Yes	P2P (STP)

Теперь давайте попытаемся ответить на вопрос: а что же будет происходить с портами коммутатора SW-4, который получает BPDU от корневого моста аж на двух портах сразу? Что тут сказать, ему предстоит крайне нелегкий выбор: один из портов нужно будет оставить в режиме пересылки кадров, а вот другой перевести в режим блокирования, чтобы разорвать петлю.

Как будет происходить выбор порта для отключения?

Для начала коммутатор сравнит значения стоимости путей, которые соседи сообщили ему в своих BPDU. Рассмотрим их подробнее:

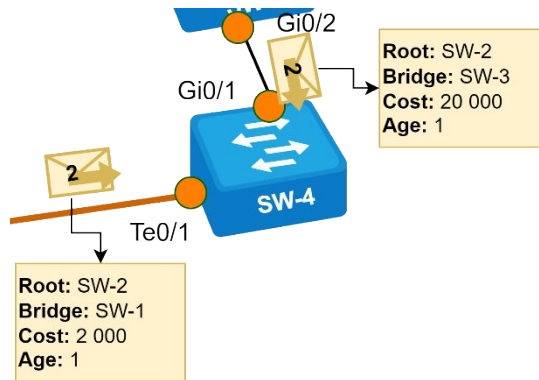


Рисунок 8.2.4.2. – Сравнение BPDU на двух портах коммутатора

Коммутатор понимает, что стоимость пути от SW-1 до корня равна 2.000, а от SW-3 – 20.000. Чтобы рассчитать полную стоимость пути до корня, SW-4 достаточно прибавить к этим значениям стоимость каналов, которые ведут к его соседям, а он их прекрасно знает - это значение его собственных портов.

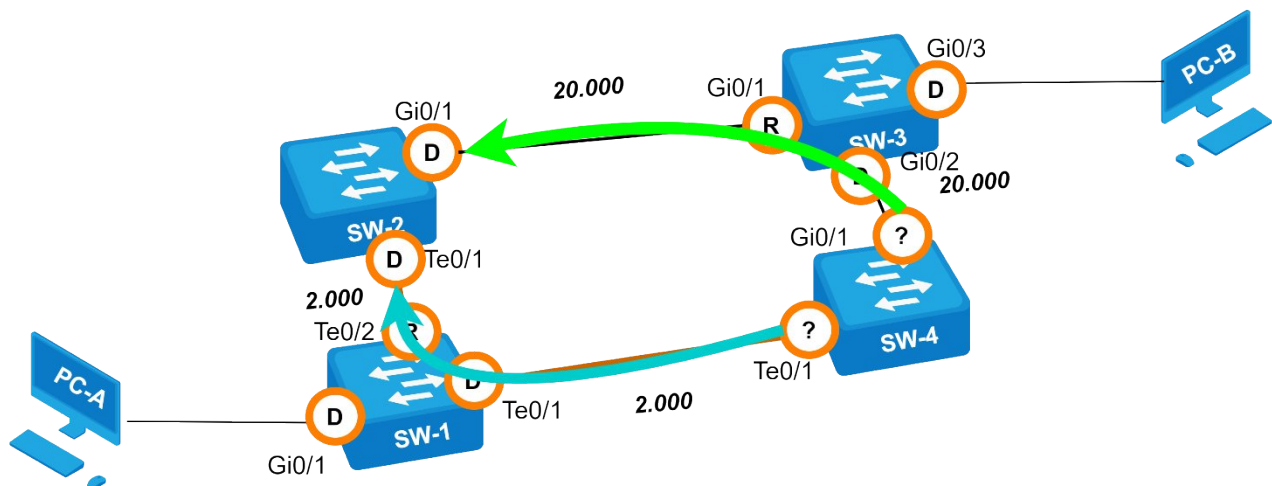


Рисунок 8.2.4.3. – Стоимость путей до корневого коммутатора

Значит, стоимость пути через SW-1 составит 2.000 (от SW-1 до корня) + 2.000 (стоимость порта Te0/1) = 4.000.

Стоимость пути через SW-3 составит 20.000 (от SW-3 до корня) + 20.000 (стоимость порта Gi0/1) = 40.000.

Мы видим, что стоимость пути через коммутатор SW-3 несколько больше стоимости пути через SW-1, а это значит, что если мы будем передавать кадр через SW-3, то он будет идти до корня нашего «дерева» медленнее, чем если мы его отправим через SW-1. Стало быть, *при данном соотношении приоритетов коммутаторов и скоростей каналов* в наших интересах отправлять все кадры с коммутатора SW-4 в

сторону корня через порт Te-0/1.

Решение о выборе корневого порта принимается коммутатором в следующем порядке:

1. Сравниваются стоимости пути до корневого моста (Root Path Cost);
2. Сравниваются приоритеты коммутаторов;
3. Сравниваются приоритеты портов на самом коммутаторе.

То есть, если бы у нас были равны стоимости путей, мы сравнили бы приоритеты коммутаторов-соседей. Это уже практически верный вариант, так как при уникальности MAC-адресов, которые являются частью BPID, вероятность равенства сочетания *приоритет+MAC* довольно низка.

Третий этап ставит окончательную точку в этом детективе: в этом значении учитывается еще и порядковый номер порта на самом коммутаторе, то есть оно точно будет уникальным.

«Побеждает» всегда меньшее значение.

Так как у нас решающим значением оказалась стоимость пути до корневого коммутатора, SW-4 присваивает порту Te0/1 роль Root Port, а порту Gi0/1 – Non-Designated (неназначенный). Если к порту подключено окончательное устройство, порт также получает роль назначенного.

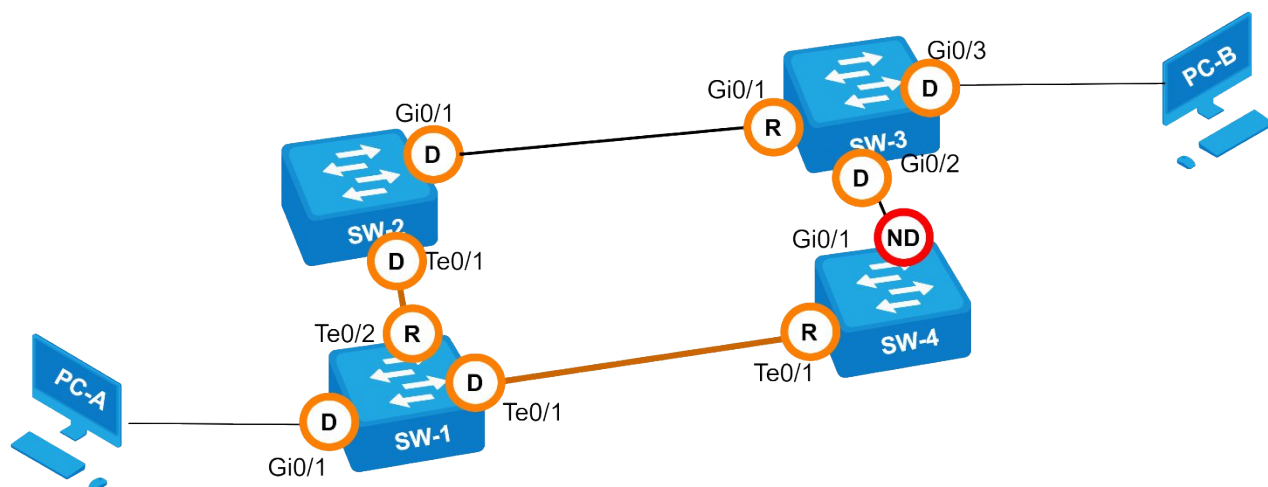


Рисунок 8.2.4.4. – Роли портов в установившейся топологии STP

У каждой роли порта существуют собственные функции:

- Через назначенные порты (*Designated*) отправляются BPDU от корневого моста;
- Через корневые порты (*Root*) принимаются BPDU от корневого моста, и отправляются сообщения об изменении топологии (*Topology Change Notification – TCN*);
- Через неназначенные порты (*Non-Designated*) данные не передаются, BPDU не передаются, но принимаются.

8.2.5. Состояния портов в STP

На рисунках выше все порты у нас обозначались оранжевым, кроме порта в роли Non-Designated: он обозначался красным, то есть не передавал трафик данных. Порт,

обозначенный оранжевым, находится в состоянии прослушивания (*Listening*), что обычно происходит при включении протокола STP на порту коммутатора. Кроме состояний *Listening*, в STP предусмотрены еще состояния изучения (*Learning*), которое в дальнейшем будет обозначено желтым цветом, и пересылки (*Forwarding*), которое будет обозначено зеленым цветом. В состоянии *Forwarding* обычно находятся корневые (*Root*) и назначенные (*Designated*) порты. В прошлом разделе эти порты обозначены оранжевым, так как за время происходящих событий они бы просто не успели достичь состояния *Forwarding*, для чего им потребовалось бы 30 секунд. Более подробно процесс перехода портов из одного состояния в другое рассмотрим ниже.

Порт не должен создавать петлю. Поэтому авторами протокола STP были предусмотрены два сценария перехода между состояниями *Forwarding* и *Blocking*:

1. Из *Forwarding* в *Blocking* напрямую, так как в этом случае передача данных прекращается, и петля создаться не может;
2. Из *Blocking* в *Forwarding*. Такой переход может создать петлю, поэтому он осуществляется не напрямую, а поэтапно через два других состояния: *listening* (прослушивание) и *learning* (изучение).

Давайте рассмотрим ситуацию, когда к уже сошедшейся сети подключается дополнительный коммутатор: в уже знакомой нам топологии к коммутаторам SW-3 и SW-4 подключается коммутатор SW-5.

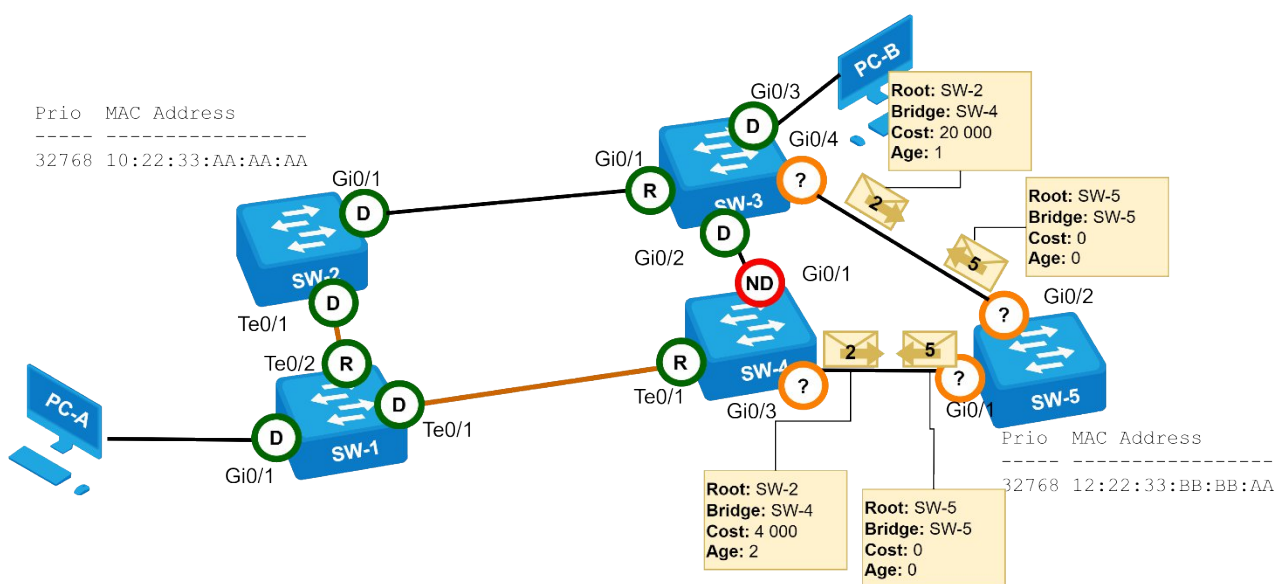


Рисунок 8.2.5.1. – Подключение нового коммутатора к сети с STP и обмен BPDU

При включении вновь задействованные порты на всех трех коммутаторах сразу же попадают в состояние *Listening* (прослушивание). В этом состоянии порты могут принимать и передавать BPDU от соседей, но блокируют все остальные кадры. Самое главное назначение состояния *Listening* – дать возможность коммутатору отправить с данного порта hello-BPDU с информацией о себе и принять аналогичное BPDU от соседа.

В нашем случае коммутатор SW-5 высылает с порта Gi0/1 BPDU, в котором объявляет себя корневым коммутатором.

Root ID: 32768, 12:22:33:BB:BB:AA

Bridge ID: 32768, 12:22:33:BB:BB:AA

Root Path Cost: 0

Message Age: 0

Вероятность его становления корневым мостом равна нулю, ибо его BID больше, чем у существующего «корня». Но он-то сам об этом еще не знает, и узнает он о существовании другого корневого моста только из BPDU, которое он получит от SW-4 и SW-3.

В BPDU от SW-4 будет следующая информация:

Root ID: 32768, 10:22:33:AA:AA:AA (BID SW-2);

Bridge ID: 32768, 10:22:33:BB:BB:AA (BID SW-4);

Root Path Cost: 4.000 (2.000, полученные от SW-1, плюс стоимость собственного порта 2.000);

Message Age: 2 (второй коммутатор, пересылающий BPDU, после корневого).

В BPDU от SW-3:

Root ID: 32768, 10:22:33:AA:AA:AA (BID SW-2);

Bridge ID: 32768, 10:22:33:AB:AA:AA (BID SW-3);

Root Path Cost: 20.000 (0, полученные от SW-2, плюс стоимость собственного порта 20.000);

Message Age: 1 (первый коммутатор, пересылающий BPDU, после корневого).

Итак, коммутаторы обменялись BPDU, и SW-5 прекращает рассылку BPDU. Просчитав стоимость пути до корневого моста с портов Gi0/2 и Gi0/1, коммутатор получает следующие цифры:

- стоимость до «корня» через Gi0/1 – 24.000;
- стоимость до «корня» через Gi0/2 – 40.000.

Корневым портом станет Gi0/1, а Gi0/2 примет роль Non-Designated и перейдет в состояние Blocking.

Казалось бы, вопрос решен: остальные порты можно переводить в состояние Forwarding и называть корневыми и назначенными. Но протокол STP выбирает осторожность и действует строго по расписанию, а именно:

1. дожидается окончания состояния *Listening* согласно таймеру Forward Delay;
2. переходит в состояние *Learning* (изучение), где пробудет еще 15 секунд согласно тому же таймеру. Порты в состоянии Learning на схеме обозначены желтым цветом.

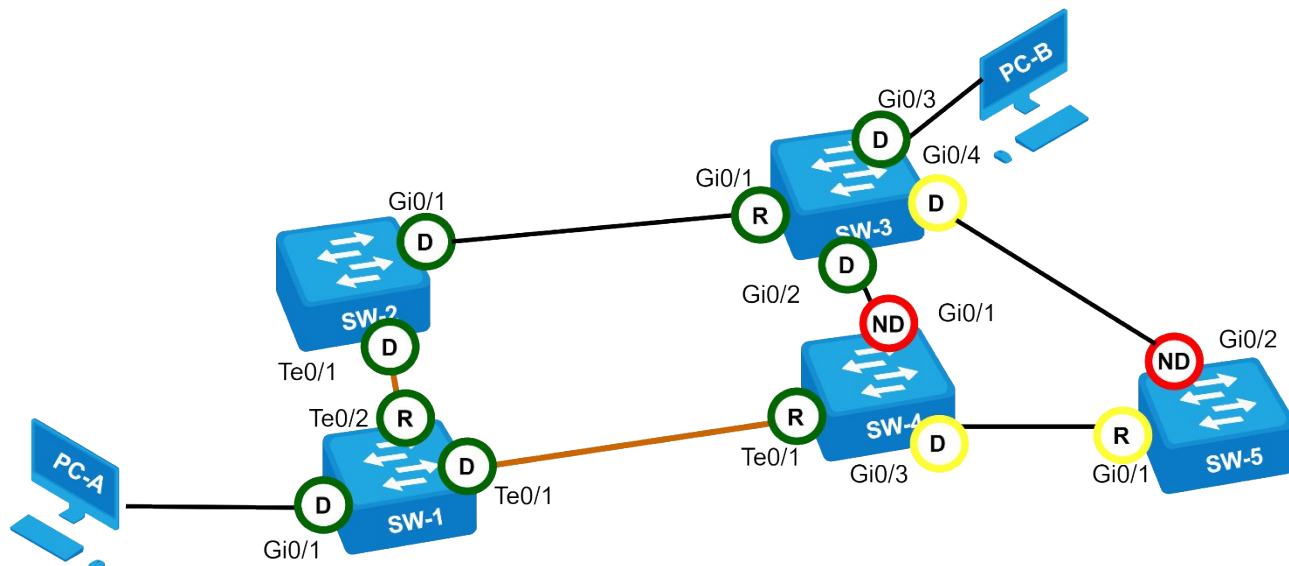


Рисунок 8.2.5.2. – Порты в состоянии Learning

В состоянии Learning порт не только прослушивает, отправляет BPDU, но и получает другие кадры, заполняет таблицу MAC-адресов согласно полям Source MAC Address. Это делается для того, чтобы вовремя заметить петли, например, если коммутатор вдруг получит на какой-нибудь из своих портов BPDU, который он отправил с другого своего же порта. Так как для отсчета времени нахождения в состоянии Learning используется тот же таймер, что и для состояния Listening, пребывание в Learning продолжается 15 секунд.

После этого порт считается безопасным с точки зрения потенциального образования петель и переходит в состояние Forwarding, то есть теперь он может отправлять и получать любой трафик.

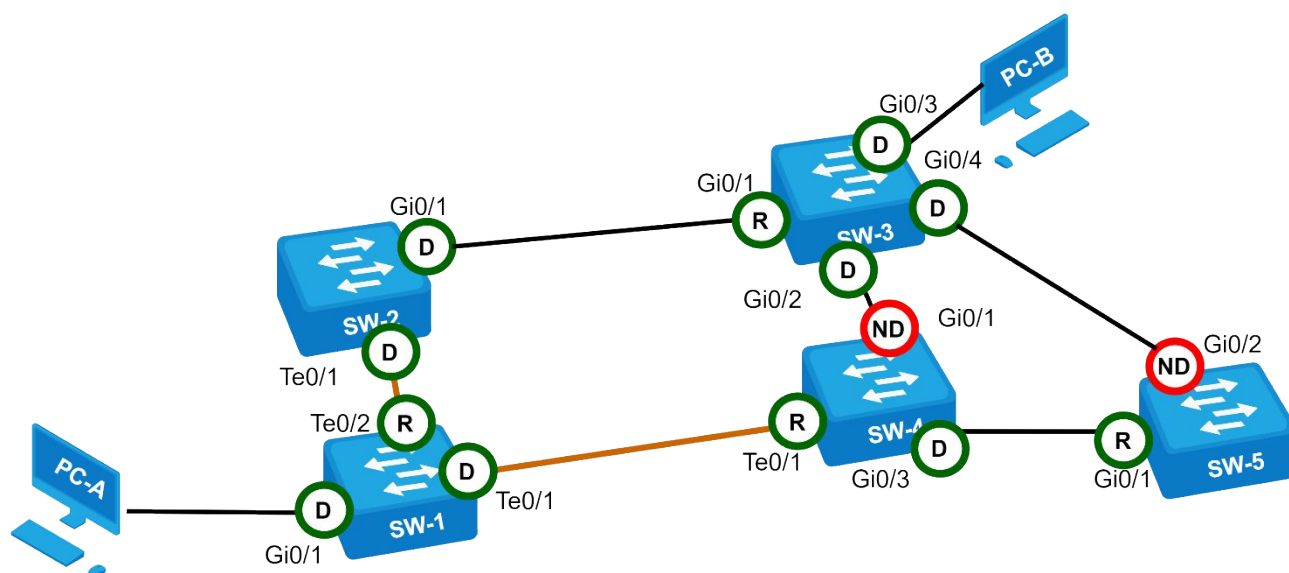


Рисунок 8.2.5.3. – Переход портов в состояние Forwarding

Сравнение состояний протокола STP представлено в Таблице ниже.

Таблица 8.2.5.1. – Состояния портов в протоколе STP

	Принимает BPDU	Отправляет BPDU	Изучает MAC-адреса	Принимает данные	Отправляет данные
--	----------------	-----------------	--------------------	------------------	-------------------

Blocking	Да	Нет	Нет	Нет	Нет
Listening	Да	Да	Нет	Нет	Нет
Learning	Да	Да	Да	Нет	Нет
Forwarding	Да	Да	Да	Да	Да

Кроме четырех вышеперечисленных состояний, существует еще одно – *Disabled* (Отключен). Это состояние, в которое попадает порт-участник процесса STP, если:

- на этом порту отключен протокол STP;
- порт отключен администратором (*Down Adm*);
- порт на другом конце канала отключен администратором, или отсутствует кабельное соединение (*Down nc*).

MES#show spanning-tree

-----часть вывода пропущена-----

Interfaces

Name	State	Prio.Nbr	Cost	Sts	Role	PortFast	Type
gil/0/1	enabled	128.49	2000000	Dsbl	Dsbl	No	-
gil/0/3	enabled	128.51	20000	Frw	Root	No	P2P (STP)

MES#show interfaces status

Port	Type	Duplex	Speed	Neg	Flow ctrl	Link State
gil/0/1	1G-Copper	--	--	--	--	Down (nc)
gil/0/2	1G-Copper	--	--	--	--	Down (nc)
gil/0/3	1G-Copper	Full	1000	Enabled	Off	Up

-----часть текста пропущена-----

Рисунок 3.1.5.4. – Состояние порта Disabled и его возможная причина

8.2.6. Изменение топологии STP

Мы рассмотрели и поняли, как ведут себя порты в процессе работы протокола STP, и что происходит, например, при подключении нового коммутатора к имеющейся сети.

Давайте рассмотрим, что произойдет, если в топологии STP произойдут изменения, например, если произойдет отключение корневого порта из-за повреждения сегмента, соединяющего коммутатор с «корнем» топологии. В нашем примере теряется связность между портом Gi0/1 коммутатора SW-5 и портом Gi0/3 коммутатора SW-4.

После потери работоспособности сегмента порт Gi0/1 переходит в статус Down (*nc*), следовательно, в состояние STP Disabled (обозначено черным цветом). Другой же порт коммутатора, Gi0/2, становится корневым портом, но ждет еще положенные 20 секунд, чтобы убедиться в «смерти» порта Gi0/1. Все это время он находится в состоянии Blocking.

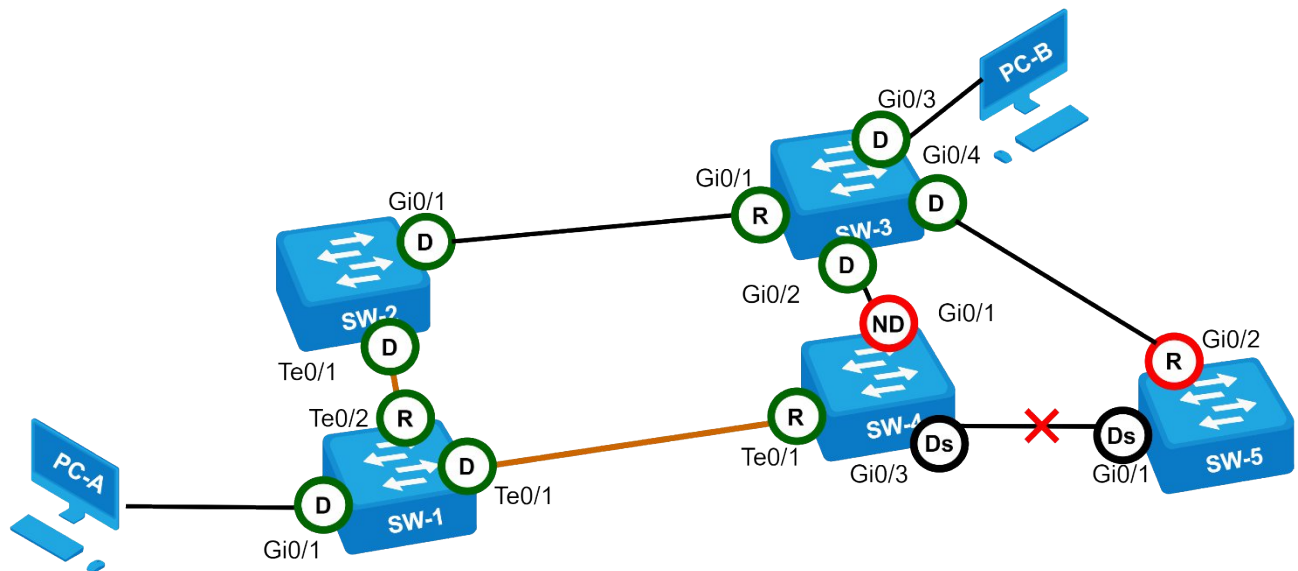


Рисунок 8.2.6.1. – Потеря работоспособности сегмента и переход портов в состояние Disabled

Не забываем о том факте, что все это время порт Gi0/2 получал BPDU через порт Gi0/4 коммутатора SW-3, так как состояние Blocking этому никак не мешает. Теперь же, когда порт Gi0/2 перешел в состояние Listening, разрешающее отправку BPDU, SW-5 сразу же отправляет через него сообщение об изменении топологии вышестоящему коммутатору. Такое сообщение называется **Topology Change Notification (TCN)**. На рисунке оно обозначено черным цветом, так как именно так обычно отмечается в WireShark.

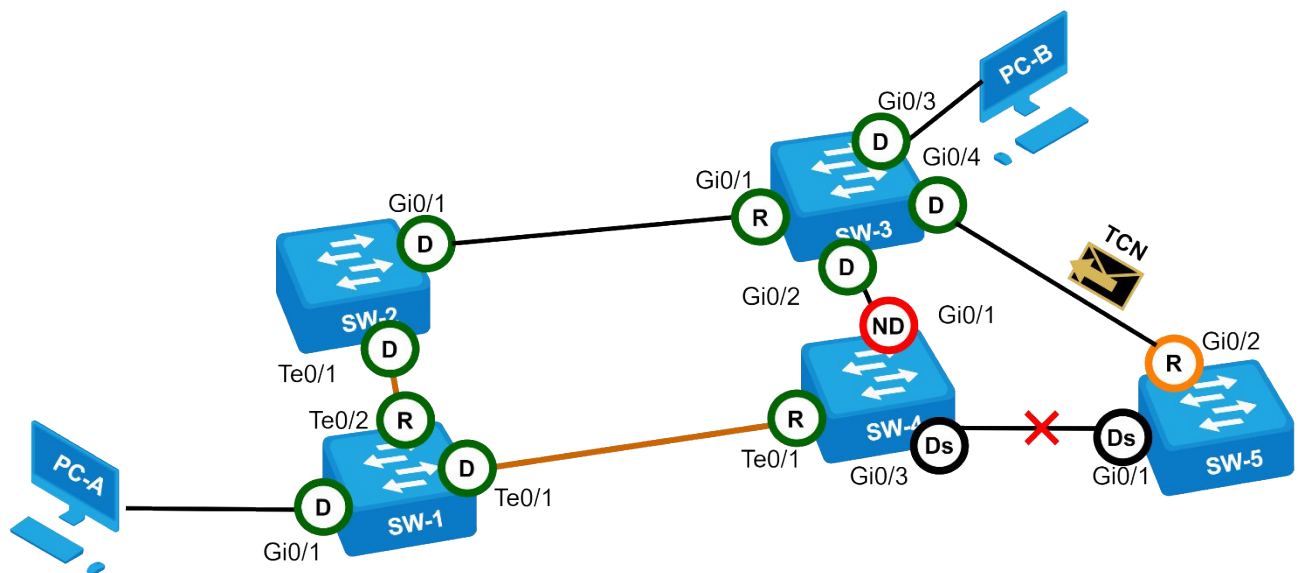


Рисунок 8.2.6.2. – Отправка TCN в сети

Сообщения TCN – второй вид BPDU, существующий в протоколе STP. В отличие от конфигурационных BPDU, они отправляются некорневыми коммутаторам в сторону корневого, то есть от нижестоящих к вышестоящим. Структура сообщения показана на рисунке ниже.

[illegible]

Рисунок – 8.2.6.3. Topology Change Notification

Получив такое сообщение, «вышестоящий» коммутатор делает следующее:

1. уравнивает таймер «старения» таблицы коммутации с таймером Forward Delay из настроек протокола STP (обычно это 15 секунд);
2. отправляет подтверждение принятия сообщения об изменении топологии (TCN Acknowledgement) с Designated-порта в сторону коммутатора, отправившего TCN;
3. отправляет TCN со своего корневого порта в сторону корневого коммутатора.

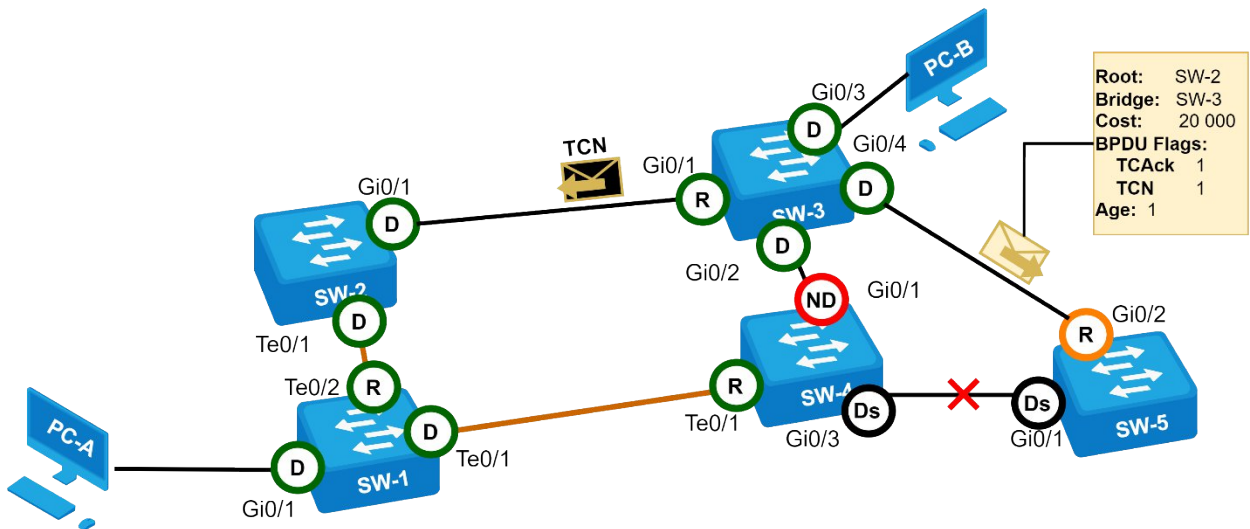


Рисунок 8.2.6.4. – Рассылка TCN в сети

Подтверждение изменения топологии (Topology Change Acknowledgement), отправляемое с SW-3 на SW-5, – это конфигурационное BPDU, отличающееся от обычного только наличием двух «флагов» в поле BPDU Flags⁴. Оно подтверждает коммутатору-отправителю, что его сообщение получено, поэтому незачем засорять сеть излишними TCN, можно спокойно дожидаться перехода порта Gi0/2 в состояние Forwarding и продолжения работы. Выглядит подтверждение следующим образом:

⁴ В этом поле целых 8 бит, но в «каноническом» STP используются только два: первый и последний. Остальные биты используются в протоколе RSTP, который мы рассмотрим отдельно.

```

v Spanning Tree Protocol
  Protocol Identifier: Spanning Tree Protocol (0x0000)
  Protocol Version Identifier: Spanning Tree (0)
  BPDU Type: Configuration (0x00)
v BPDU flags: 0x81, Topology Change Acknowledgment, Topology Change
  1... .... = Topology Change Acknowledgment: Yes
  .... ...1 = Topology Change: Yes
> Root Identifier: 32768 / 0 / a8:f9:4b:30:fc:c0
  Root Path Cost: 20000
> Bridge Identifier: 32768 / 0 / e0:d9:e3:2c:1c:c0
  Port identifier: 0x8031
  Message Age: 1
  Max Age: 20
  Hello Time: 2
  Forward Delay: 15
  
```

Рисунок 8.2.6.5.— Topology Change Acknowledgement

Такая же последовательность действий повторится и при получении TCN корневым коммутатором за одним исключением: после отправки TC Acknowledgement он начнет отправлять BPDU с одним флагом, а точнее, с флагом Topology Change (TC). В отличие от TCA, они отправляются не разово, а каждые 2 секунды в течение 60-70 секунд. Такая серия сообщений необходима, чтобы BPDU с правильной топологией (а с точки зрения корневого коммутатора с его MAC-адресом и стоимостью пути до его интерфейса) дошли до самых дальних уголков, а точнее, ветвей нашей сети. В Wireshark такие BPDU называются TC + Root Config. BPDU.

```

v Spanning Tree Protocol
  Protocol Identifier: Spanning Tree Protocol (0x0000)
  Protocol Version Identifier: Spanning Tree (0)
  BPDU Type: Configuration (0x00)
v BPDU flags: 0x01, Topology Change
  0... .... = Topology Change Acknowledgment: No
  .... ...1 = Topology Change: Yes
> Root Identifier: 32768 / 0 / a8:f9:4b:30:fc:c0
  Root Path Cost: 20000
> Bridge Identifier: 32768 / 0 / e0:d9:e3:2c:1c:c0
  Port identifier: 0x8031
  Message Age: 1
  Max Age: 20
  Hello Time: 2
  Forward Delay: 15
  
```

Рисунок 8.2.6.6. – BPDU типа Topology Change (TC)

Через 30 секунд после отправки первого TCN порт Gi0/2 завершает прохождение состояний Listening и Learning и переходит в состояние Forwarding. Топология сошлась.

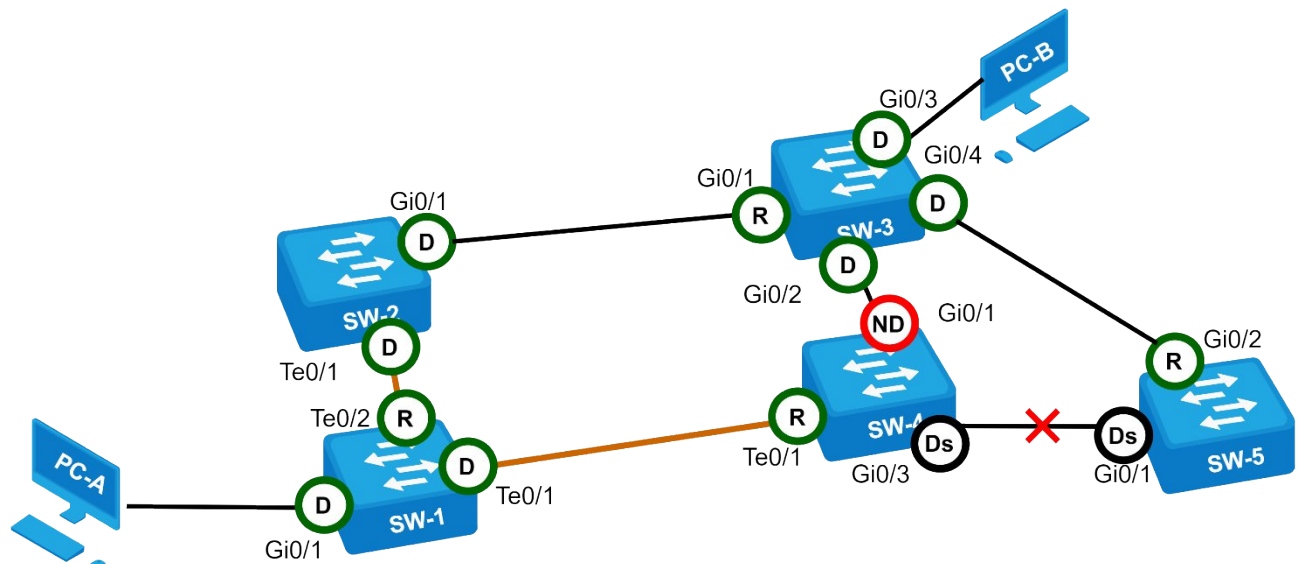


Рисунок 8.2.6.7. – Порт gi0/2 перешёл в состояние Forwarding

На восстановление связности между коммутатором SW-5 и остальной сетью у нас ушло 50 секунд, что, возможно, было приемлемо в 1985 году, но совершенно неприемлемо в наши дни. Вследствие этого была разработана «быстрая версия» данного протокола — **Rapid Spanning Tree Protocol (RSTP)**, речь о которой пойдет в следующем разделе.

8.2.7. Настройка протокола STP на коммутаторах MES

По умолчанию протокол STP глобально включен на всех коммутаторах Eltex MES.

На коммутаторах серии MES 23xx/33xx/35xx/5324 по умолчанию включен протокол версии RSTP.

На коммутаторах серии MES 14xx/24xx по умолчанию включен протокол версии MSTP.

Включение и отключение протокола STP.

Отключить протокол STP глобально можно командой:

```
MES (config) #no spanning-tree
```

Если протокол STP отключен, включить его глобально можно командой:

```
MES (config) #spanning-tree
```

Выбор режима «классического» STP:

```
MES2308 (config) #spanning-tree mode stp
```

Включить или отключить протокол STP на отдельных интерфейсах можно командами:

Отключить:

```
MES (config-if) #spanning-tree disable
```

Включить:

```
MES (config-if) #no spanning-tree disable
```

Настроить приоритет коммутатора (Bridge Priority):

```
MES (config) # spanning-tree priority <prior_val>
```

Prior_val может принимать значение от 0 до 61440 с шагом 4096. По умолчанию 32768.

Выбрать метод исчисления стоимости пути можно следующими командами:

```
MES (config) # spanning-tree pathcost method <long|short>
```

- **short** – значение стоимости в диапазоне [1..65535];
- **long** – значение стоимости в диапазоне [1..200000000].

Административно установить значение стоимости для порта:

```
MES (config-if) # spanning-tree cost <Portcost>
```

Portcost может принимать значения от 0 до 200 000 000.

Настроить таймеры STP:

Таймер отправки конфигурационных BPDU (hello-timer):

```
MES (config) # spanning-tree hello-time <1-10 sec>
```

По умолчанию 2 секунды.

Таймер максимального времени ожидания BPDU от корневого коммутатора (Max Age):

```
MES (config) # spanning-tree max-age <sec>
```

По умолчанию 20 секунд

Таймер переключения из состояния Listening в Learning и из Learning в Forwarding (Forward Delay):

```
MES (config) # spanning-tree forward-time <4-30 sec>
```

По умолчанию 15 секунд.

При отключении протокола STP может возникнуть необходимость заблокировать получение и отправку BPDU от других коммутаторов. Для этого на порту вводится команда:

```
MES (config-if) spanning-tree bpdu filtering
```

8.2.9. Принцип работы RSTP

Основной задачей протокола RSTP является обеспечение более быстрой сходимости по сравнению с «каноническим» протоколом STP. А также RSTP поддерживает STP для совместимости с устаревшим оборудованием. Поскольку в названии RSTP буква «R» означает Rapid, т.е. «быстрый», то порты RSTP, не находящиеся в режиме сбрасывания (*Discarding*), мгновенно переводятся в режим пересылки (*Forwarding*).

Для этого был разработан несколько иной механизм сходимости, было уменьшено количество состояний портов, а также добавлены 2 дополнительные роли порта.

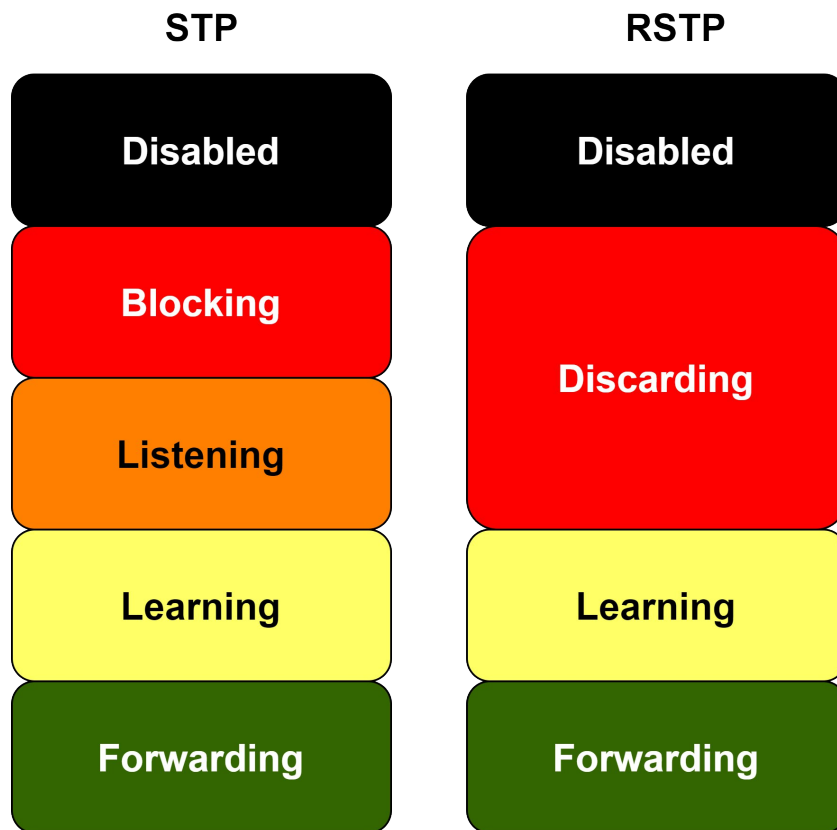


Рисунок 8.2.9.1. – Сравнение состояний STP и RSTP

В протоколе RSTP два состояния портов (Blocking и Listening) объединены в состояние Discarding (*Отбрасывание*). В этом состоянии порт продолжает получать BPDU, но не изучает MAC-адреса и не пересылает трафик.

Состояние Learning продолжается всего 10 секунд, чего, как мы увидим из изменившегося механизма сходимости, вполне достаточно для достижения беспетлевой конфигурации сети.

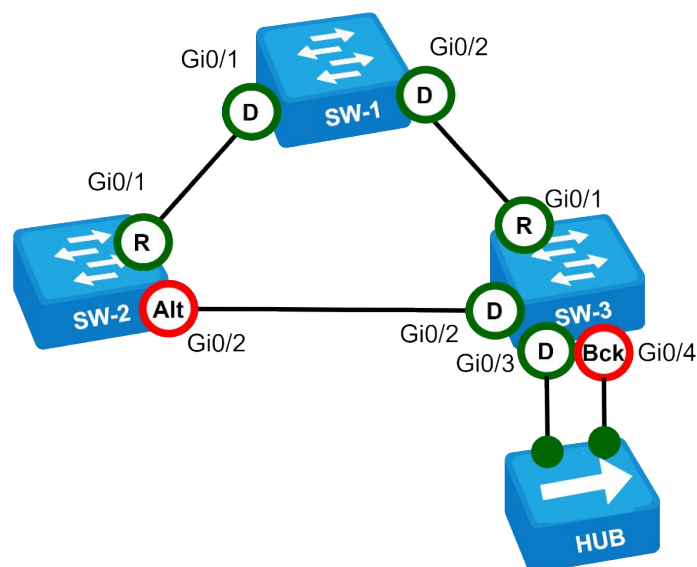


Рисунок 8.2.9.2. – Роли портов в RSTP

Еще одно нововведение – это дополнительные роли портов: *Alternate* (альтернативный) и *Backup* (запасной).

Alternate – порт, который, как следует из названия, является альтернативным путем до корневого коммутатора. На данном порту принимаются BPDU от корневого коммутатора с худшей стоимостью (cost) чем у текущего Root-порта. В роли Alternate порт отбрасывает любой трафик за исключением BPDU. При отсутствии BPDU на текущем Root-порту в течение 3 Hello-интервалов, Alternate-порт становится Root-портом.

Backup – порт, который «заместит» текущий Designated-порт в случае отключения линка на нем. Коммутатор назначает Backup-портом тот порт, который получает BPDU от самого коммутатора. Это может быть в двух случаях:

1. петля на коммутаторе (в нашей схеме между портами Gi0/3 и Gi0/4);
2. за портом находится устройство, не поддерживающее протокол STP (это может быть концентратор (хаб) или коммутатор с отключенным STP).

Во всех прочих случаях оба порта на коммутаторе SW-3 находились бы в состоянии Designated, а вот на ответном коммутаторе один из портов был бы Root, а другой – Alternate.

Разница между состояниями Discarding и Blocking невелика: порт не передает трафик от конечных устройств и других коммутаторов и продолжает принимать BPDU.

А вот роли портов отличаются, можно сказать, идеологически. *Alternate* принимает роль Root-порта при отказе «вышестоящего» коммутатора, но не через 20 секунд (Max Response Time, как в классическом STP), а через 6, так как для понимания отсутствия вышестоящего коммутатора ему достаточно пропустить 3 BPDU. *Backup* принимает роль *Designated* порта также через 3 пропущенных BPDU.

С состояниями порта все несколько иначе. Из состояния Discarding порт сразу переходит в Learning, в котором находится 10 секунд. За это время происходит несколько интересных событий. Рассмотрим их на нашем примере.

Допустим, по какой-либо причине у нас перестал функционировать канал между коммутаторами SW-1 и SW-3.

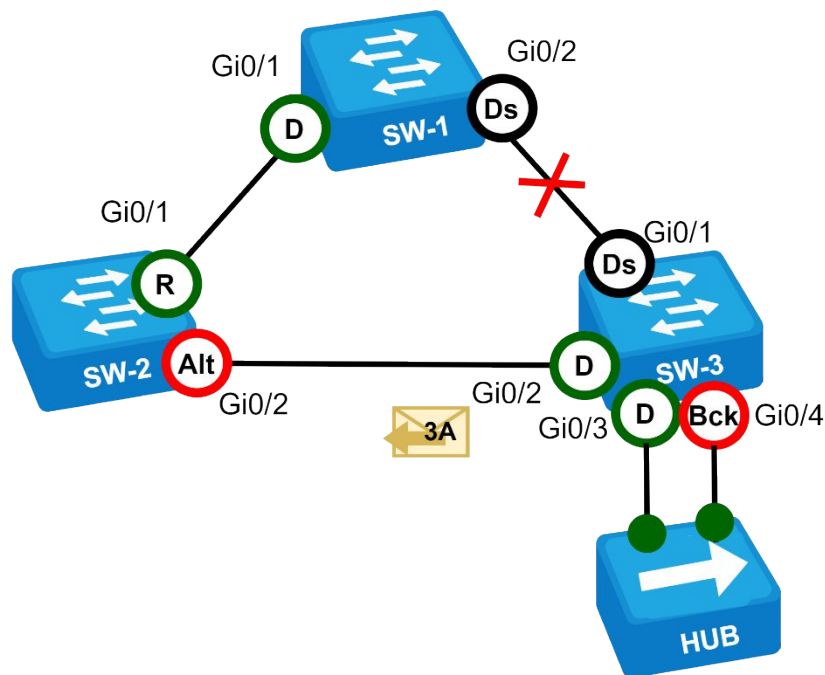


Рисунок 8.2.9.3. – Поведение RSTP при обрыве связи с корневым мостом

Этап №1. Не получив подряд 3 BPDU от SW-1 на порту Gi0/1, коммутатор начинает считать себя корневым. Об этой новости он оповещает всех соседей, так как за портами Gi0/3 и Gi0/4 нет соседей, поддерживающих протокол RSTP, мы сконцентрируемся на общении с коммутатором SW-2. SW-3 отправляет BPDU, в котором объявляет о том, что он теперь является корневым мостом (на схеме мы его обозначили 3A, то есть BPDU от коммутатора 3 с флагом Agreement).

```
> Source: EltexEnt_1f:f4:82 (e4:5a:d4:1f:f4:82)
  Length: 39
  Padding: 0000000000000000
> Logical-Link Control
▼ Spanning Tree Protocol
  Protocol Identifier: Spanning Tree Protocol (0x0000)
  Protocol Version Identifier: Rapid Spanning Tree (2)
  BPDU Type: Rapid/Multiple Spanning Tree (0x02)
▼ BPDU flags: 0x7c, Agreement, Forwarding, Learning, Port Role: Designated
  0... .... = Topology Change Acknowledgment: No
  .1.. .... = Agreement: Yes
  ..1. .... = Forwarding: Yes
  ...1 .... = Learning: Yes
  .... 11.. = Port Role: Designated (3)
  .... ..0. = Proposal: No
  .... ...0 = Topology Change: No
> Root Identifier: 32768 / 0 / e4:5a:d4:1f:f4:80
  Root Path Cost: 0
> Bridge Identifier: 32768 / 0 / e4:5a:d4:1f:f4:80
```

Рисунок 8.2.9.4. – BPDU SW-3, объявление себя корневым мостом

Обратите внимание, расширилось поле BPDU flags: в STP использовалось только два флага, а теперь все восемь.

Флаг Agreement (*согласие*) означает, что это BPDU отправлено коммутатором в

порядке информирования соседей. Совпадение полей Bridge Identified и Root Identified показывает, что коммутатор-отправитель считает себя корневым мостом.

Этап №2. Получив BPDU от коммутатора SW-3 о том, что тот считает себя корневым мостом, коммутатор SW-2 приходит в некоторое удивление, так как он точно знает, что за его портом Gi0/1 существует настоящий корневой мост SW-1, от которого он регулярно получает BPDU. При этом приоритет SW-1 лучше, чем у SW-3. Поэтому коммутатор SW-2 сразу же переводит порт Gi0/2 из состояния Discarding в состояние Learning, а из роли Alternate в роль Designated и пересылает SW-3 BPDU от SW-1, пометив его флагом Proposal (предложение), как бы предлагая ему стать частью устоявшейся топологии.

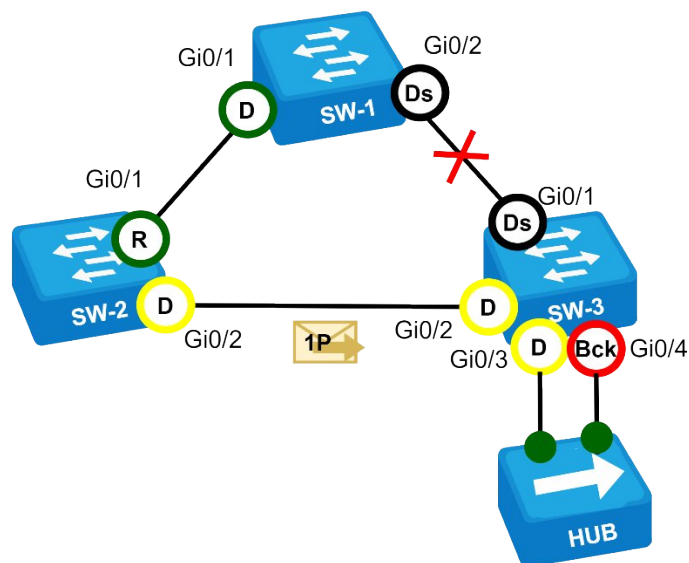


Рисунок 8.2.9.5. – BPDUs Proposal

```
> Source: EltexEnt_3d:7f:42 (e4:5a:d4:3d:7f:42)
  Length: 39
  Padding: 0000000000000000
Logical-Link Control
Spanning Tree Protocol
  Protocol Identifier: Spanning Tree Protocol (0x0000)
  Protocol Version Identifier: Rapid Spanning Tree (2)
  BPDU Type: Rapid/Multiple Spanning Tree (0x02)
  BPDUs flags: 0x0e, Port Role: Designated, Proposal
    0... .... = Topology Change Acknowledgment: No
    .0.. .... = Agreement: No
    ..0. .... = Forwarding: No
    ...0 .... = Learning: No
    .... 11.. = Port Role: Designated (3)
    .... ..1. = Proposal: Yes
    .... ...0 = Topology Change: No
  > Root Identifier: 32768 / 0 / a8:f9:4b:30:fc:c0
  Root Path Cost: 20000
  > Bridge Identifier: 32768 / 0 / e4:5a:d4:3d:7f:40
```

Рисунок 8.2.9.6. – BPDUs Proposal

Этап №3. Получив Proposal от SW-2, коммутатор SW-3 понимает, что в сети существует коммутатор с меньшим BID, находится он за портом Gi0/2, и «добраться» до него можно через соседа SW-2. Поэтому SW-3 быстро переводит свои порты Gi0/2 и Gi0/3 в состояние Discarding, затем Learning и назначает им роли Root и Designated соответственно. После этого он оповещает вышестоящий коммутатор SW-2 о подчинении, отправляя BPDU с собственным идентификатором Bridge ID, идентификатором SW-1 в качестве Root ID и стоимостью пути от SW-3 до SW-1 через SW-2. В сообщение проставляется флаг Agreement, что указывает на ответ на ранее отправленный Proposal.

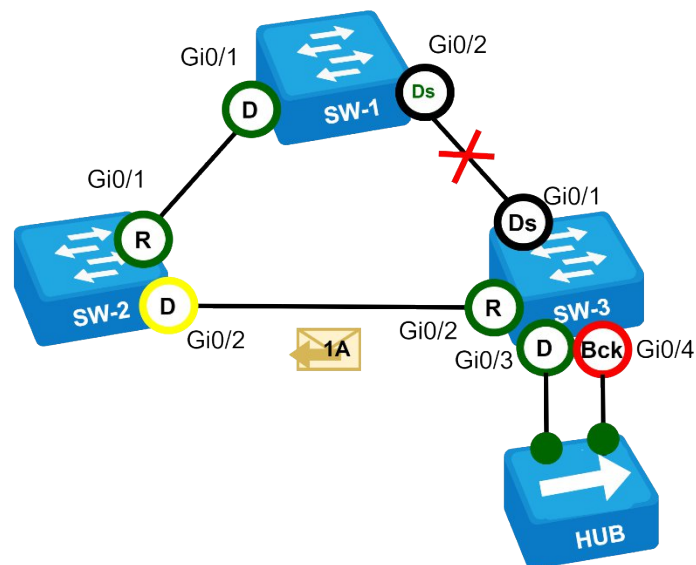


Рисунок 8.2.9.7. – BPDU Agreement

```
> Source: EltexEnt_1f:f4:82 (e4:5a:d4:1f:f4:82)
  Length: 39
  Padding: 0000000000000000
> Logical-Link Control
> Spanning Tree Protocol
  Protocol Identifier: Spanning Tree Protocol (0x0000)
  Protocol Version Identifier: Rapid Spanning Tree (2)
  BPDU Type: Rapid/Multiple Spanning Tree (0x02)
> BPDU flags: 0x78, Agreement, Forwarding, Learning, Port Role: Root
  0... .... = Topology Change Acknowledgment: No
  .1.. .... = Agreement: Yes
  ..1. .... = Forwarding: Yes
  ...1 .... = Learning: Yes
  .... 10.. = Port Role: Root (2)
  .... ..0. = Proposal: No
  .... ...0 = Topology Change: No
> Root Identifier: 32768 / 0 / a8:f9:4b:30:fc:c0
  Root Path Cost: 40000
> Bridge Identifier: 32768 / 0 / e4:5a:d4:1f:f4:80
  Port identifier: 0x8002
```

Рисунок 8.2.9.8. – BPDU подтверждения новой топологии RSTP

Отметим, что в ответном Agreement-сообщении порт Gi0/2 коммутатора SW-3 представляется уже не Designated, а Root-портом. Таким образом он дает сигнал коммутатору SW-2, что можно не ждать 10 секунд, а переключать порт Gi0/2 из состояния Learning в состояние Forwarding.

Этап 4. Коммутатор SW-2 переводит порт Gi0/2 в состояние Forwarding и отправляет BPDU с флагом Topology Change из Designated-портов и Root-порта, таким образом оповещая все коммутаторы о новой топологии. Каждый коммутатор в топологии, получив это сообщение, устанавливает таймер tcWhile (время рассылки сообщений с флагом TopologyChange) на Hello Time + 1 секунда (в нашем примере это 2+1=3 секунды) и в течение этого времени рассылает сообщения TC. По получении такого сообщения любой коммутатор с включенным RSTP сбрасывает изученные MAC-адреса на всех портах, кроме порта, на котором получено сообщение Topology Change, а также рассылает это сообщение дальше со своих Designated-портов.

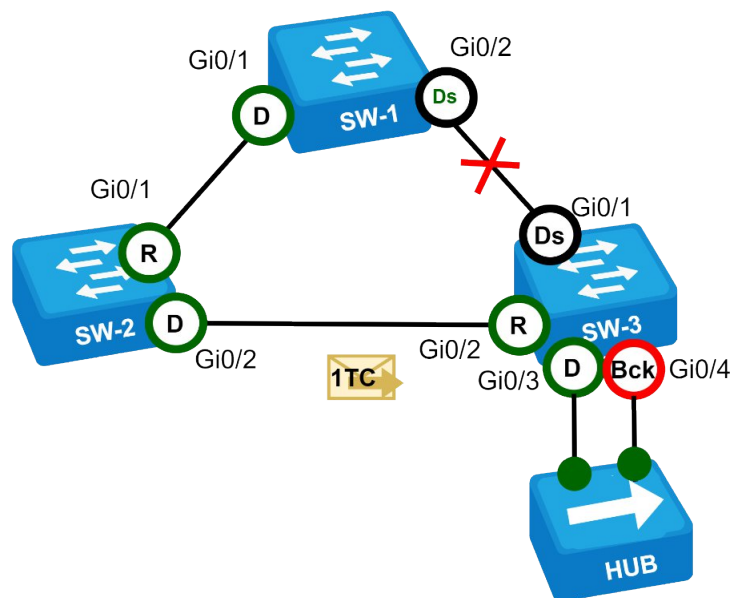


Рисунок 8.2.9.9. – Финальное схождение топологии RSTP

```
> Source: EltexEnt_3d:7f:42 (e4:5a:d4:3d:7f:42)
  Length: 39
  Padding: 0000000000000000
> Logical-Link Control
  > Spanning Tree Protocol
    Protocol Identifier: Spanning Tree Protocol (0x0000)
    Protocol Version Identifier: Rapid Spanning Tree (2)
    BPDU Type: Rapid/Multiple Spanning Tree (0x02)
    > BPDU flags: 0x3d, Forwarding, Learning, Port Role: Designated, Topology Change
      0... .. = Topology Change Acknowledgment: No
      .0.. .. = Agreement: No
      ..1. .... = Forwarding: Yes
      ...1 .... = Learning: Yes
      .... 11.. = Port Role: Designated (3)
      .... ..0. = Proposal: No
      .... ...1 = Topology Change: Yes
    > Root Identifier: 32768 / 0 / a8:f9:4b:30:fc:c0
    Root Path Cost: 20000
    > Bridge Identifier: 32768 / 0 / e4:5a:d4:3d:7f:40
    Port identifier: 0x8002
```

Рисунок 8.2.9.10. – Сообщение Topology Change

позволило избавиться от отдельного BPDU для сообщения изменения топологии, а это дало возможность каждому коммутатору рассылать информацию об изменении топологии (а не только корневому коммутатору, как это было в «классическом» STP).

8.2.10. Граничные (edge) и неграничные (non-edge) порты

У протокола RSTP есть еще один важный инструмент ускорения сходимости. Как мы уже знаем, при включении любой порт сначала попадает в состояние Discarding, а затем через Learning включается в передачу данных, то есть в состояние Forwarding. Это сделано для того, чтобы избежать случайных петель, ведь коммутатор не знает, есть ли за очередным его портом другой коммутатор или концентратор.

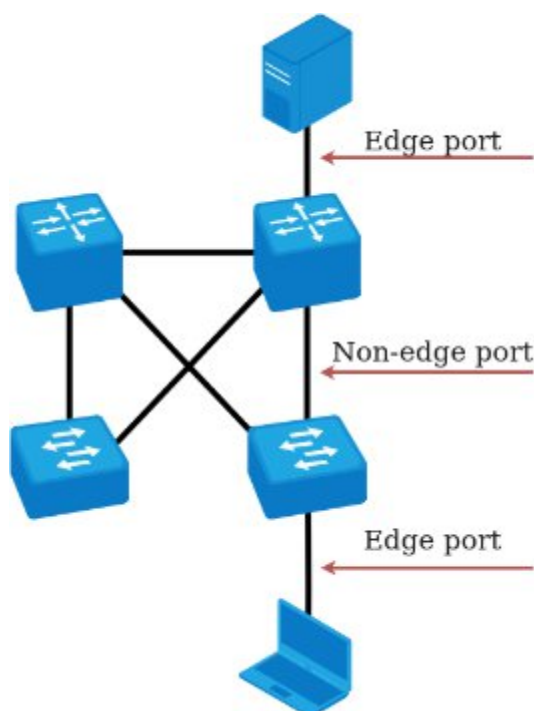


Рисунок 8.2.10.1. — Сообщение Topology Change

Но что если мы знаем, что на коммутаторе определенная группа портов никогда не будет использоваться для подключения к другим коммутаторам, и в них будут подключаться только оконечные устройства? Это бы сильно облегчило жизнь, ведь тогда можно было бы сразу включать эти порты в состояние Forwarding и не заставлять оконечные устройства ждать 10 секунд, пока им разрешать взаимодействие с сетью.

В протоколе RSTP все порты делятся на две категории:

1. граничные (*edge*) порты, которые предназначены исключительно для оконечных устройств;
2. неграничные (*non-edge*) порты, к которым могут подключаться сетевые устройства, то есть «потенциально петлеопасные».

Если изначально обозначить все граничные порты, то мы убьем двух зайцев: «разгрузим» вычислительные мощности коммутатора для более полезных заданий и добьемся более быстрого включения оконечных устройства.

Концепция граничных портов появилась при разработке протокола RSTP, но теперь

она используется и в классическом STP, и в более позднем MSTP.

"А что произойдет," - спросите вы, - "если неосознательный пользователь или администратор подключит сетевое устройство с включенным STP к граничному порту?"

В этом случае, конечно же, последствия будут прискорбными: либо образуется петля трафика, и, как следствие, шторм, либо будет перестроена вся топология STP, что также может привести к ее неоптимальному функционированию: будут заблокированы не те порты, которые изначально предполагались, и трафик «потечет» неоптимально.

Команда `portfast` приводит к немедленному переключению порта доступа в состояние пересылки (forwarding) протокола связующего дерева.

```
MES (config-if)#spanning-tree portfast
```

Warning: portfast should only be enabled on ports connected to a single host.

Connecting hubs, concentrators, switches, bridges, etc... to this interface when portfast is enabled, can cause temporary bridging loops.

Use with CAUTION

Рис. 8.2.10.2. – Предупреждение о небезопасности команды `portfast`

Отметим еще, что неграничные порты могут быть двух категорий:

1. **P2P (point-to-point)** – это порты, работающие в режиме полного дуплекса (*full duplex*).

2. **Shared** – это порты, работающие в режиме полудуплекса (*half-duplex*). Вероятность встретить их на рабочей сети стремится к нулю, но если вы увидели их в диагностике, скорее всего, на данном коммутаторе STP отключен.

```
Spanning tree Protocol has been disabled
Bridge Id      Priority 32768
              Address 00:00:00:00:00:00
              Max age is 20 sec 0 cs, forward delay is 15 sec 0 cs
              Hello Time is 2 sec 0 cs
              Dynamic Path Cost is Disabled
              Dynamic Path Cost Lag-Speed Change is Disabled

Name          Role      State      Cost      Prio    Type
----          -
fa 0/1        Disabled Unknown    200000    128     SharedLan
fa 0/2        Disabled Unknown    200000    128     SharedLan
```

Рис. 8.2.10.3. – Индикация отключения протокола STP на коммутаторе

Порядок настройки граничных портов будет рассмотрен далее.

8.2.11. Настройка протокола RSTP на коммутаторах MES

Включение и отключение протокола STP.

Отключить протокол STP глобально можно командой:

```
MES (config)#no spanning-tree
```

Если протокол STP отключен, включить его глобально можно командой:

```
MES (config) #spanning-tree
```

Выбрать режим RSTP можно командой:

```
MES (config)#spanning-tree mode rstp
```

Включить или отключить протокол STP на отдельных интерфейсах можно командами:

Отключить:

```
MES (config-if) #spanning-tree disable
```

Включить:

```
MES (config-if) #no spanning-tree disable
```

Настроить приоритет коммутатора (Bridge Priority):

```
MES (config) # spanning-tree priority <prior_val>
```

Prior_val может принимать значение от 0 до 61440 с шагом 4096. По умолчанию 32768.

Выбрать метод исчисления стоимости пути можно следующими командами:

```
MES (config) #spanning-tree pathcost method <long|short>
```

- **long** – значение стоимости в диапазоне [1..200000000];
- **short** – значение стоимости в диапазоне [1..65535].

Административно установить значение стоимости для порта:

```
MES (config-if) spanning-tree cost Portcost
```

Portcost может принимать значения от 0 до 200 000 000.

Настроить таймеры STP:

Таймер отправки конфигурационных BPDU (hello-timer):

```
MES (config) # spanning-tree hello-time <1-10 sec>
```

По умолчанию 2 секунды.

Таймер максимального времени ожидания BPDU от корневого коммутатора (Max Age):

```
MES (config) # spanning-tree max-age <sec>
```

По умолчанию 20 секунд.

Таймер переключения из состояния Listening в Learning и из Learning в Forwarding (Forward Delay):

```
MES (config) # spanning-tree forward-time <4-30 sec>
```

По умолчанию 15 секунд.

При отключении протокола STP может возникнуть необходимость заблокировать получение и отправку BPDU от других коммутаторов. Для этого на порту вводится команда:

```
MES (config-if) #spanning-tree bpdu filtering
```

Включить режим граничного порта на интерфейсе можно командой:

```
MES(config-if)#spanning-tree portfast
```

Отключить режим граничного порта на интерфейсе можно командой:

```
MES(config-if)#no spanning-tree portfast
```

8.2.12. Multiple Spanning Tree Protocol (MSTP)

Протокол Multiple STP является наиболее современной реализацией STP.

У протоколов STP и RSTP есть один ключевой недостаток: они не поддерживают работу с VLAN, а значит, при использовании этого инструмента сегментации сетей нас могут ожидать весьма непредсказуемые результаты. Если в сети используется значительное количество VLAN, то и нагрузка на коммутаторы кратно возрастает, что не может положительно повлиять на работоспособность сети.

Решить вопрос с интеграцией VLAN в протокол STP пытались некоторые вендоры самостоятельно, например, компания Cisco Systems, которая разработала протокол Per Vlan Spanning Tree (PVST+). В этом протоколе для каждой VLAN запускался свой собственный экземпляр (*instance*) протокола STP, следовательно, можно было каждый коммутатор сделать корневым в определенной VLAN. Это позволило распределить нагрузку по нескольким коммутаторам. Однако основная проблема осталась: в большой сети количество VLAN кратно превышает количество коммутаторов, поэтому с ростом количества VLAN производительность сети также заметно ухудшается. Главным же недостатком протокола PVST+ и его дальнейшего развития (RPVST+) долгое время являлось то, что они оставались проприетарными, то есть поддерживались только на оборудовании Cisco Systems.

Учитывая эти недостатки, был разработан стандартный протокол MSTP (***Multiple Spanning Tree Protocol*** – *протокол множественных расширяющихся деревьев*), который в настоящее время используется на оборудовании большинства производителей.

MSTP не «разменивается» на строительство деревьев в каждой из VLAN. Вместо этого он строит единое дерево для группы VLAN в пределах одного экземпляра (*instance*). Максимально допустимое количество экземпляров MSTP на одном коммутаторе – 64.

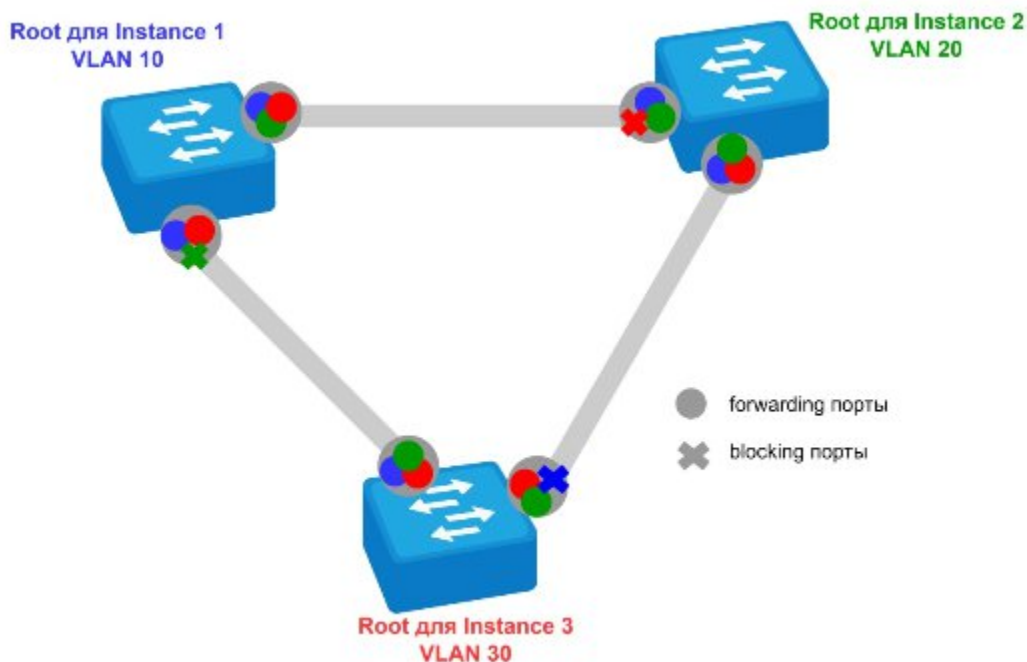


Рисунок 8.2.12.1. – Принцип работы протокола MSTP

Несколько коммутаторов, имеющих одинаковую конфигурацию деревьев (VLAN в экземплярах), объединяются в один регион (region). Еще одной особенностью протокола MSTP является то, что для всех экземпляров он использует единое сообщение BPDU, что изрядно экономит трафик (PVST+, к примеру, нещадно гоняет по сети отдельные BPDU для каждой VLAN, причем каждые 2 секунды).

Для правильной работы в пределах одного региона на коммутаторах должны совпадать:

1. Имя региона (*name*);
2. Экземпляры MSTP (*instance*);
3. Номер версии конфигурации (*revision*).

По умолчанию все VLAN привязаны к экземпляру под номером 0 (Instance 0). Он также называется «**внутренним расширяющимся деревом**» – *Internal Spanning Tree (IST)*.

Для настройки протокола MSTP на коммутаторах MES применяются следующие команды.

8.2.13. Настройка протокола MSTP на коммутаторах MES

Включение протокола MSTP:

```
MES (config) #spanning-tree mode mst
```

Настройка региона, экземпляров и версии конфигурации:

```
MES (config) #spanning-tree mst configuration
```

```
MES (config-mst) #name RegionName
```

```
MES (config-mst) #instance InstNumber vlan vlan-range
MES (config-mst) #revision RevNumber
MES (config) #[exit | abort]
```

- *RegionName* – имя региона;
- *InstNum* – номер экземпляра;
- *vlan-range* – диапазон VLAN, привязанных к данному экземпляру;
- *exit* – выход из режима конфигурации региона MSTP с сохранением настроенных параметров;
- *abort* – выход из режима конфигурации региона MSTP без сохранения настроенных параметров.

Также можно явно задать приоритет данного коммутатора для конкретного экземпляра, чтобы он стал корневым мостом по умолчанию.

```
MES (config) #spanning-tree mst InstNum priority 0
```

Рассмотрим пример конфигурации:

```
MES (config) #spanning-tree mode mst
MES (config) #spanning-tree mst configuration
MES (config-mst) #name testRegion
MES (config-mst) #instance 1 vlan 10-20
MES (config-mst) #instance 2 vlan 21-30
MES (config-mst) #instance 3 vlan 31-40
MES (config-mst) #revision 1
MES (config) #exit
MES (config) #spanning-tree mst 1 priority 0
```

Для проверки конфигурации протокола MSTP можно использовать команды `show`, которые могут незначительно отличаться по синтаксису в зависимости от моделей коммутаторов.

MES#show spanning-tree mst-configuration

```
Current MST configuration
Name: e4:5a:d4:39:bc:e0
Revision: 0
Instance      State                Vlans Mapped
-----
0             enabled             1, 72-73, 253-254, 337, 420, 560, 700-712
```

8.3. Агрегация каналов – Link Aggregation

8.3.1. Принципы работы агрегации каналов

Так как по каналам между коммутаторами чаще всего передается трафик

нескольких VLAN, то и «потеря» такого канала может быть гораздо более чувствительной, чем любого другого.

Очевидное решение – добавить еще один линк между коммутаторами.

Посмотрим, что будет происходить на коммутаторах при подключении дополнительного канала.

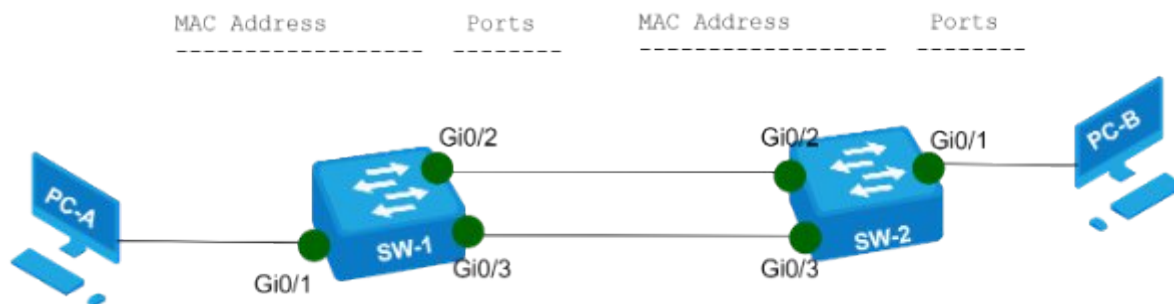


Рисунок 8.3.1.1. – Таблица коммутации пуста

Для понимания процесса предположим, что на обоих коммутаторах таблица коммутации (*mac address table*) является «необученной», то есть MAC-адреса компьютеров PC A и PC B ей неизвестны. Это совершенно обычная ситуация, если компьютеры не передают трафик через свои порты в течение какого-то времени.

PC A решил передать какой-то трафик PC B.

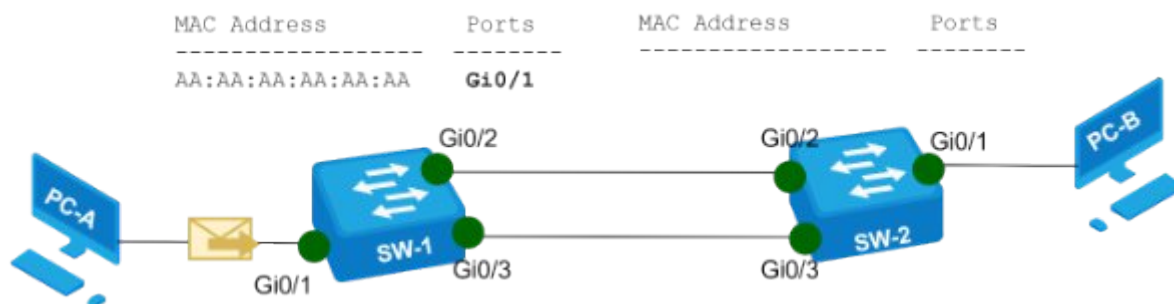


Рисунок 8.3.1.2. – Изучение MAC-адреса AA:AA:AA:AA:AA:AA на порту Gi0/1

При прохождении порта Gi 0/1 Коммутатор SW1 изучит его MAC-адрес AA:AA:AA:AA:AA:AA и занесет его в таблицу коммутации.

Далее коммутатор должен принять решение, что делать с кадром.

Вспомним основы работы коммутатора:

1. При поступлении кадра на порт коммутатор записывает MAC-адрес источника в таблицу коммутации (изучает MAC-адрес);
2. Если MAC-адрес назначения изучен в таблице коммутации за каким-то интерфейсом, кадр копируется в этот интерфейс;
3. Если MAC-адрес назначения не изучен ни одним интерфейсом, то коммутатор копирует его во все интерфейсы, кроме того, на который кадр был получен⁵. Такой метод рассылки

⁵ Действительно, как же он туда вышлет кадр, если на этом порту только что изучился MAC-адрес

называется «**одноадресная рассылка на неизвестный адрес**» (*unknown unicast*).

Следовательно, получив кадр от PC A, коммутатор скопирует его в порты Gi0/2 и Gi0/3.

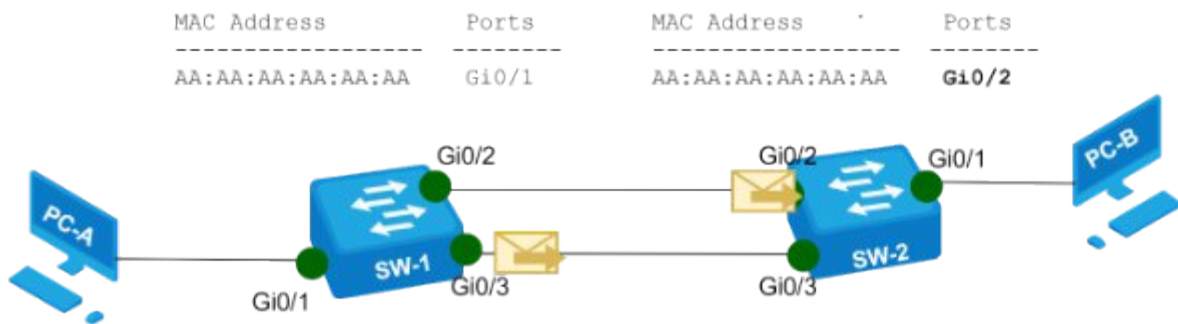


Рисунок 8.3.1.3. – Изучение MAC-адреса AA:AA:AA:AA:AA:AA на порту Gi0/2

Так как кадры будут отправлены с минимальной, но задержкой, на порты коммутатора SW-2 они также придут одновременно. Предположим, что первым кадр придет на порт Gi0/2. Следовательно, SW-2 изучит MAC-адрес отправителя на порту Gi0/2 и скопирует его в порты Gi0/3 и Gi0/1 (во все порты, кроме того, откуда пришел кадр).

В это время до коммутатора SW-2 доходит кадр с порта Gi0/3. И тут в действие вступает четвертое правило работы коммутатора:

- При изучении MAC-адреса, уже существующего в таблице коммутации на другом порту, коммутатор переписывает таблицу коммутации.

Следовательно, на коммутаторе SW-2 происходит переизучение (re-learning) MAC-адреса AA:AA:AA:AA:AA:AA на порту Gi0/3, и он копирует кадр в порты Gi0/2 и Gi0/1.

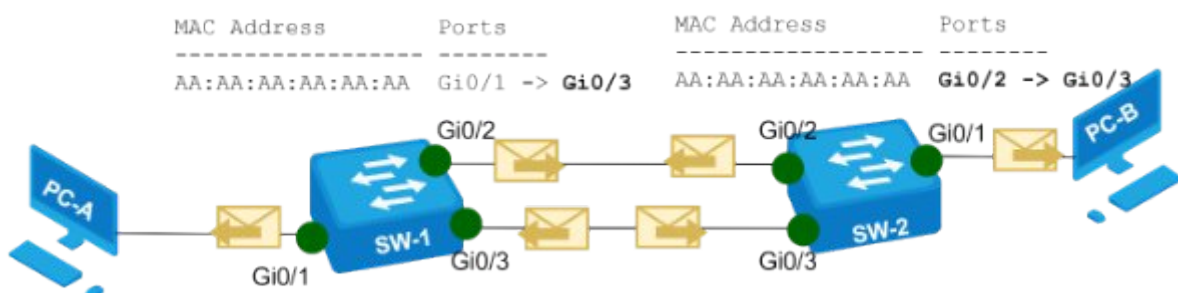


Рисунок 8.3.1.4. – Изменение портов в таблице коммутации Gi0/1 -> Gi0/3, Gi0/2 -> Gi0/3

А что в это время происходит на коммутаторе SW-1? Если помним, то SW-2 выслал ему наш же кадр с порта Gi0/3, а значит, он будет получен на порту Gi0/3. На нем же он будет переизучен и скопирован в порты Gi0/2 и Gi0/1.

Мы не зря постоянно используем слово «скопировал» по отношению к кадру. Дело в том, что на данный момент по сети циркулирует уже 4 копии этого кадра вместо одной. И, как вы понимаете, процесс этот не закончится никогда, так как у кадра Ethernet нет механизма предотвращения зацикливания трафика, подобного TTL в IP-пакете. А нас

еще ожидает «поступление» ответного кадра от компьютера PC-B, который постигнет та же участь. Следовательно, налицо цепная реакция: с каждой пересылкой количество кадров увеличивается, и коммутатор будет их размножать, пока не будет занята вся полоса пропускания. Такое явление называется **штормом**⁶.

Что произойдет, если мы подключим между коммутаторами еще один канал? Ничего особенного, просто трафик начнет размножаться вдвое быстрее.

Как же достичь цели резервирования канала между коммутаторами, при этом не получив шторм?

Для этого используется технология объединения каналов (*link aggregation*, не путать с «уровнем агрегации», который по-английски называется *distribution layer*). Работает она следующим образом:

На коммутаторе SW-1 мы объединяем порты Gi0/2 и Gi0/3 в группу каналов (*channel group*) и привязываем к ней виртуальный интерфейс (Port-Channel). Таким образом мы приказываем коммутатору сделать следующее:

1. Получаемые кадры на любом из физических интерфейсов, включенных в канальную группу, изучаются на соответствующем виртуальном интерфейсе Port Channel;
2. Отправляемые кадры сначала копируются в виртуальный интерфейс, а далее отправляются по любому из свободных физических интерфейсов, при этом возможна балансировка нагрузки (*load balancing*).

Как же отличается ситуация с агрегацией канала от описанной выше?

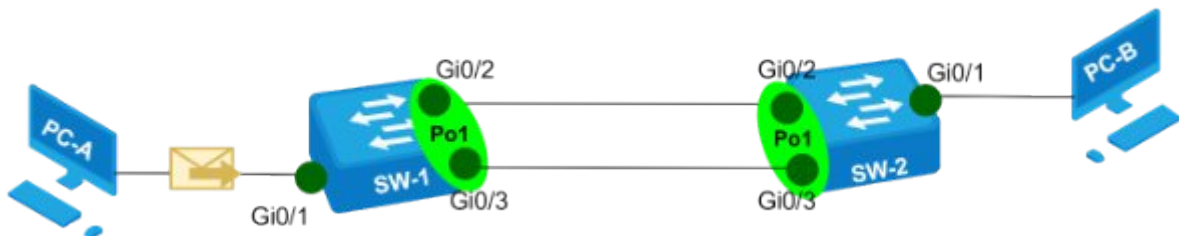


Рисунок 8.3.1.5. — PC-A отправляет кадр на PC-B

При поступлении кадра от PC-A на коммутатор SW-1 его MAC-адрес изучается на порту Gi0/1.

Теперь коммутатор должен скопировать кадр во все свободные порты, но теперь для коммутатора существует только один свободный порт – PortChannel 1. В него и копируется кадр. Он может дальше пересылаться в любой из двух портов: Gi0/2 или Gi0/3, в зависимости от того, какой из них будет в этот момент не занят. Допустим, в порт Gi0/2.

⁶ В нашем случае причиной шторма является одноадресный трафик, поэтому шторм тоже называется одноадресным. Кроме него еще выделяют широковещательный и многоадресный штормы.

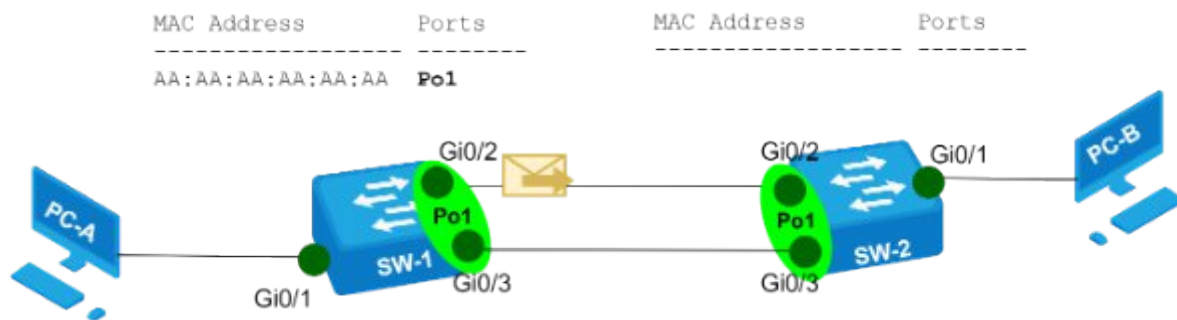


Рисунок 8.3.1.6. — Изучение MAC-адреса AA:AA:AA:AA:AA:AA на port-channel 1

На коммутаторе SW-2 кадр будет получен на физическом интерфейсе Gi0/2, но изучен он будет на интерфейсе Port Channel 1 (номера виртуальных интерфейсов на разных коммутаторах могут не совпадать, но мы для ясности будем использовать одинаковую нумерацию). Следовательно, далее коммутатор отправит кадр в порт Gi0/1, не создавая лишних копий и не засоряя полосу пропускания.

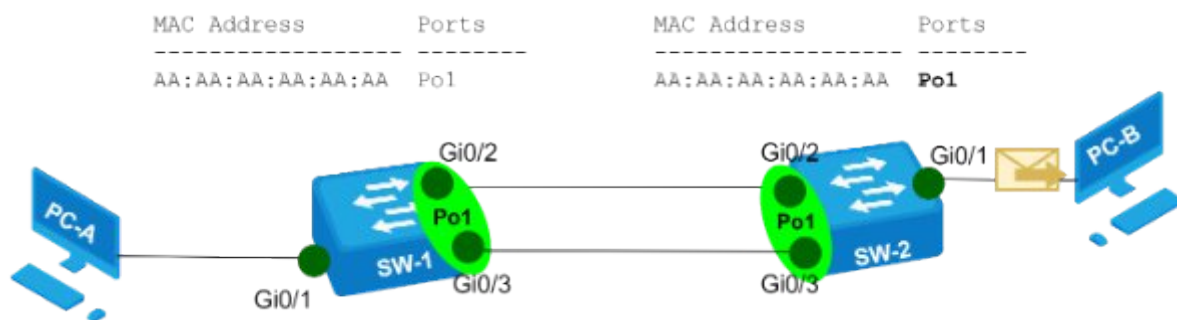


Рисунок 8.3.1.7. — Изучение MAC-адреса AA:AA:AA:AA:AA:AA на интерфейсе Port-Channel 1

От компьютера PC-B по направлению к PC-A кадр будет передан в обратном порядке без создания излишних копий.

Пример изучения MAC-адресов на интерфейсе Po1 указан в примере ниже:

```
MES# show mac-address-table
```

Vlan	Mac Address	Type	ConnectionId	Ports
----	-----	----	-----	-----
1	e0:d9:e3:c2:37:00	Learnt		po 1

8.3.2. Статическая агрегация каналов

В одну канальную группу можно объединить до 8 физических портов. При этом у портов должны совпадать следующие параметры:

1. Скорость;
2. Дуплекс/полудуплекс;
3. Режим работы порта (*access/trunk* и т. д.)⁷.

⁷Крайне желательно, чтобы на порту вообще не было настроек режима или VLAN.

Для исключения несовпадения параметров на портах, которые включаются в одну группу каналов, желательно очистить настройки командой **default interface**. После ввода команды устройство «попросит» вас подтвердить сброс настроек.

```
MES(config)# default interface GigabitEthernet0/1
Configuration for these interfaces will be set to default.
Are you sure you want to proceed? (Y/N) [N] Y
```

На коммутаторах Eltex MES настройка агрегации канала производится командой:

```
MES(config)# interface Gi0/x
MES(config-if)# channel-group N mode on
```

Где **N** – номер группы каналов. Всего таких групп каналов может быть до 48.

После ввода данной команды виртуальный интерфейс Port Channel N переходит в состояние UP, о чем устройство сообщает администратору сообщением протокола SysLog:

```
%TRUNK-I-PORTADDED: Port gil/0/x added to Po N
%LINK-I-Up: Po N
```

Рис. 8.3.2.1. – Пример включения интерфейса Port Channel N на коммутаторе MES

При такой настройке группа агрегации каналов включается администратором и происходит безусловно, интерфейсы коммутаторов-соседей не обмениваются никакими сообщениями, поэтому такой способ иногда называется **статической агрегацией каналов** (*static link aggregation, static LAG*). Статическая агрегация каналов удобна, когда в сети присутствует оборудование нескольких производителей, поддерживающее несовместимые протоколы динамической агрегации, о которой мы поговорим в следующем разделе.

В нашем примере необходимо будет произвести следующую настройку интерфейсов:

```
MES(config)# interface range gi0/2-3
MES(config-if)# channel-group 1 mode on
05-Jan-2023 11:53:44 %TRUNK-I-PORTADDED: Port gil/0/2 added to Po1
05-Jan-2023 11:53:44 %TRUNK-I-PORTADDED: Port gil/0/3 added to Po1
05-Jan-2023 11:53:44 %LINK-I-Up: Po1
```

Однако необходимо обратить внимание на то, что интерфейс Port Channel и может находиться в состоянии UP, это еще не значит, что на ответном коммутаторе канал сконфигурирован правильно. Например, при проверке канала командой **show interface port-channel** можно увидеть, что сам интерфейс находится в состоянии UP, и оба включенных в соответствующую группу физических интерфейса также находятся в активном состоянии, при этом на соседнем коммутаторе соответствующие интерфейсы не включены в группу.

Так, например, ситуация выглядит на коммутаторах MES:

```
MES# show interface Port-Channel 1
```

Port is up (connected)

Interface index is 1000

Hardware is aggregated ethernet interface(s), MAC address is e0:d9:e3:c2:37:01

Interface MTU is 1500

Link is up for 0 days, 0 hours, 3 minutes and 19 seconds

Link aggregation type is static LAG

No. of members in this port-channel: 2 (active 2, minimum required 1)

gigabitethernet1/0/2, full-duplex, 100Mbps (active)

gigabitethernet1/0/3, full-duplex, 100Mbps (active)

Обусловлено это как раз-таки «природой» статической агрегации: порты в канальной группе не «общаются» с «соседями», поэтому не могут ни «согласовать» свои действия в рамках агрегированного канала, ни даже просто выяснить, существует ли агрегированный канал по ту сторону кабеля.

8.3.3. Динамическая агрегация каналов. Протокол LACP

Статический LAG позволяет объединить до 8 каналов (линков). Однако если произойдет отключение одного из каналов, то доступно будет всего 7, что скажется на пропускной способности.

Есть ли способ сделать так, чтобы 8 каналов в одном агрегированном канале были доступны постоянно?

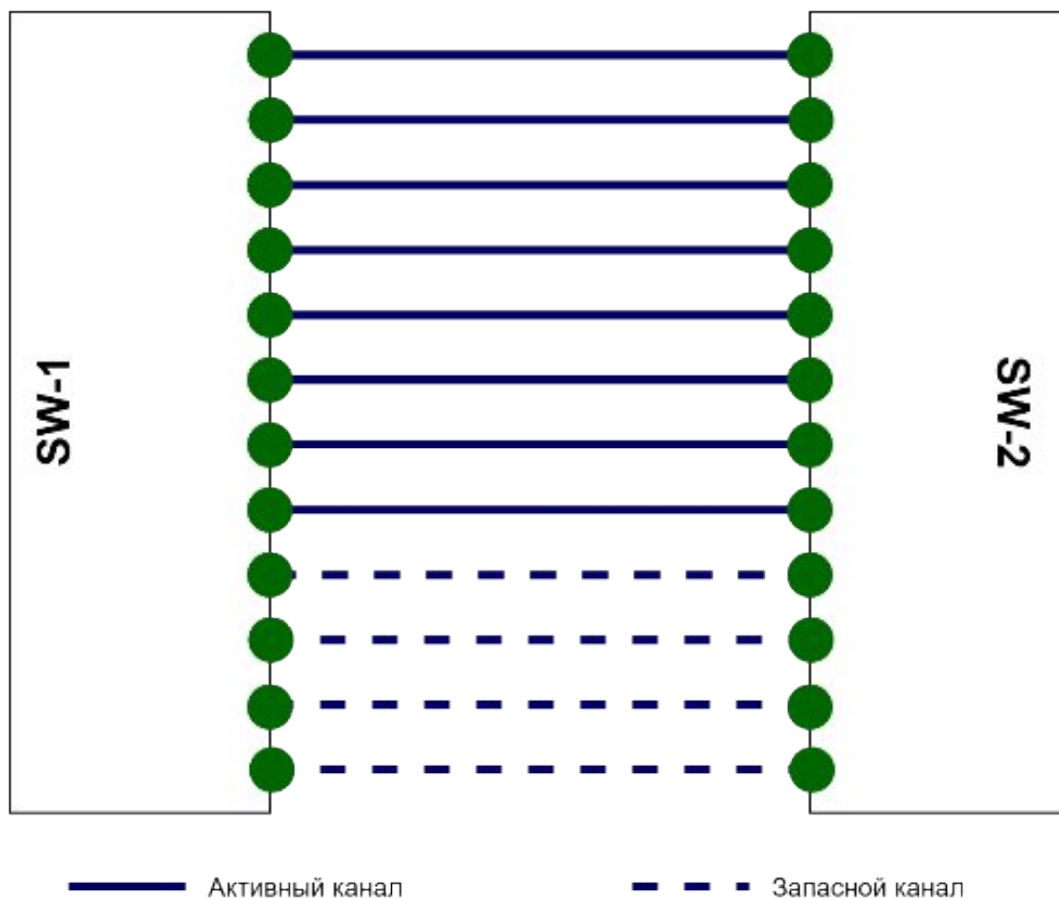


Рисунок 8.3.3.1. – Агрегация по протоколу LACP

Для этой цели используется протокол LACP (*Link Aggregation Control Protocol*), предназначенный для динамического резервирования каналов связи без образования петель коммутации.

LACP является частью стандарта IEEE802.3ad, который позволяет объединять несколько физических портов в единый логический канал. LACP позволяет сетевому устройству, такому как коммутатор, согласовывать автоматическое объединение каналов путём отправки пакетов LACPDU на устройство.

LACPDU отправляются коммутаторами на многоадресный MAC 01:80:C2:01:00:02. Так как MAC-адреса такого рода никогда не бывают адресами источника, они не изучаются в MAC-таблице за портами коммутатора, а значит, и кадры, содержащие их, не будут пересылаться дальше первого порта.

Протокол LACP должен быть включен на обеих сторонах канала.

После активации LACP коммутаторы начинают обмениваться пакетами LACPDU, основная цель которых – показать обоим участникам процесса, что второй участник также является членом соответствующей группы портов и готов к передаче данных, и что такая может осуществляться через определенный порт.

В LACP коммутаторы могут быть активными либо пассивными.

Активный коммутатор через определенные промежутки времени отправляет LACPDU на партнерские порты⁸ коммутатора-соседа и по полученным ответным LACPDU

⁸ В LACPDU коммутатор-отправитель называется actor, а его сосед по каналу – partner.

принимает решение относительно работоспособности порта.

Пассивный коммутатор ожидает получения LACPDU от активного коммутатора. Получение такого пакета означает для него, что партнер готов к работе, и своим ответным пакетом он подтверждает собственную готовность к работе в агрегированном канале.

Соответственно, если активный коммутатор отправляет LACPDU партнеру и не дожидается от него ответного LACPDU, он делает вывод, что тот к работе не готов, и порт в канальную группу не включается. Аналогично, если пассивный коммутатор не смог дожидаться LACPDU от активного, он не включает порт в агрегированный канал.

Стоит ли говорить о том, что если на двух сторонах одного канала оба коммутатора находятся в активном режиме, то канал обязательно «срастется».

А вот если один из портов сконфигурирован как статический, то независимо от того, находится ли на другой стороне канала активный или пассивный коммутатор, канал «собирается» не будет, так как от порта в режиме mode on никакие LACPDU просто-напросто не будут отправляться.

Таблица 8.3.3.1. — Совместимость режимов LAG

	Auto/Active	Passive	On
Auto/Active	Да	Да	Нет
Passive	Да	Нет	Нет
On	Нет	Нет	Да

Активный режим на коммутаторах серии MES 23xx/33xx/35xx/53xx называется *auto*, а на коммутаторах MES 14xx/24xx – *active*. Пассивный режим на обеих сериях называется *passive*.

Агрегация канала по протоколу LACP включается той же командой **channel-group**, что и для статической агрегации канала, но с указанием режима работы active (auto) или passive.

```
MES (config-if) # channel-group 1 mode auto
```

Пассивный режим на коммутаторах включается командой:

```
MES (config-if) # channel-group 1 mode passive
```

Передаваемые LACPDU содержат три основных раздела:

1. Версия протокола LACP;
2. Блок информации об отправителе (Actor Information);
3. Блок информации о получателе (Partner Information);

Блоки 2 и 3 идентичны по составу полей.

В разделе «Флаги» (Flags) передается информация о текущем состоянии отправителя, а именно:

1. **Активность (LACP Activity):** 0, если Passive, 1, если Active;
2. **Время ожидания (LACP Timeout):** 0, если выбран период Long (отправка LACPDU

раз в 30 секунд, отключение при отсутствии LACPDU через 90 секунд), 1, если Short (отправка каждую секунду, отключение через 3 секунды);

3. **Включение в канал (Aggregation)**: 0, если порт является индивидуальным каналом, 1, если включен в агрегированный канал;
4. **Синхронизация (Synchronization)**: 0, если порт не синхронизирован с системой, 1, если синхронизирован (может получить данные о приоритете системы, например);
5. **Получение (Collecting)**: 0, если порт не предназначен для получения данных в рамках агрегированного канала, 1, если может их получать;
6. **Передача (Distribution)**: 0, если на порту отключена передача данных по агрегированному каналу, 1, если не отключена;
7. **Сброс настроек (Defaulted)**: 0 – действуют настройки пользователя, 1 – порт сбрасывается в значения по умолчанию;
8. **Просрочено (Expired)**: 0, если LACPDU получено в пределах времени ожидания (timeout), 1, если после обозначенного промежутка времени.

Пример LACPDU приведен на рисунке ниже.

```

▼ Link Aggregation Control Protocol
  LACP Version: 0x01
  TLV Type: Actor Information (0x01)
  TLV Length: 0x14
  Actor System Priority: 1
  Actor System ID: EltexEnt_8c:cd:d0 (e4:5a:d4:8c:cd:d0)
  Actor Key: 1000
  Actor Port Priority: 1
  Actor Port: 55
  ▼ Actor State: 0x3d, LACP Activity, Aggregation, Synchronization, Collecting, Distributing
    .... ..1 = LACP Activity: Active
    .... ..0. = LACP Timeout: Long Timeout
    .... .1.. = Aggregation: Aggregatable
    .... 1... = Synchronization: In Sync
    ...1 .... = Collecting: Enabled
    ..1. .... = Distributing: Enabled
    .0.. .... = Defaulted: No
    0... .... = Expired: No
    [Actor State Flags: **DCSG*A]
    Reserved: 000000
    TLV Type: Partner Information (0x02)
    TLV Length: 0x14
    Partner System Priority: 1
    Partner System: EltexEnt_2c:1c:c0 (e0:d9:e3:2c:1c:c0)
    Partner Key: 1000
    Partner Port Priority: 1
    Partner Port: 55
    ▼ Partner State: 0x3d, LACP Activity, Aggregation, Synchronization, Collecting, Distributing
      .... ..1 = LACP Activity: Active
      .... ..0. = LACP Timeout: Long Timeout
      .... .1.. = Aggregation: Aggregatable
      .... 1... = Synchronization: In Sync
      ...1 .... = Collecting: Enabled
      ..1. .... = Distributing: Enabled
      .0.. .... = Defaulted: No
      0... .... = Expired: No
  
```

Рисунок 8.3.3.2. – Пример LACPDU от активного к активному коммутатору

После получения LACPDU от партнера коммутатор запоминает значения флагов поля Actor State, а также указывает их в поле флагов Partner State в ответном LACPDU. Флаги запоминаются до получения следующего LACPDU, чтобы сравнить текущее состояние с предыдущим.

```
Link Aggregation Control Protocol
LACP Version: 0x01
TLV Type: Actor Information (0x01)
TLV Length: 0x14
Actor System Priority: 1
Actor System ID: EltexEnt_8c:cd:d0 (e4:5a:d4:8c:cd:d0)
Actor Key: 1001
Actor Port Priority: 1
Actor Port: 52
▼ Actor State: 0x3c, Aggregation, Synchronization, Collecting, Distributing
    .... 0 = LACP Activity: Passive
    .... 0. = LACP Timeout: Long Timeout
    .... 1.. = Aggregation: Aggregatable
    .... 1... = Synchronization: In Sync
    ...1 .... = Collecting: Enabled
    ..1. .... = Distributing: Enabled
    .0.. .... = Defaulted: No
    0... .... = Expired: No
[Actor State Flags: **DCSG**]
Reserved: 000000
```

Рисунок 8.3.3.3. –Пример LLCPU от пассивного коммутатора

Еще одной функцией протокола LACP является определение активных портов в агрегированном канале. Как вы помните, только 8 портов могут участвовать в пересылке данных. Они называются активными портами (*active*), а остальные – запасными (*backup*).

Какие именно порты будут участвовать в пересылке данных?

Определение активных портов происходит в два этапа:

1. На первом этапе определяется коммутатор, который и будет выбирать активные порты;
2. На втором этапе ведущий коммутатор определяет 8 портов (если их больше восьми), которые будут активными в канале.

«Ведущий» коммутатор определяется суммой полей System Priority и System ID, последнее представляет собой MAC-адрес коммутатора.

Поле System Priority (Приоритет системы) может принимать значение от 0 до 65535. Для коммутаторов серии MES 23xx/33xx/35xx/53xx приоритет системы по умолчанию равен 1, для коммутаторов серии MES 14xx/24xx – 32768.

```

v Link Aggregation Control Protocol
  LACP Version: 0x01
  TLV Type: Actor Information (0x01)
  TLV Length: 0x14
  Actor System Priority: 1
  Actor System ID: EltexEnt_8c:cd:d0 (e4:5a:d4:8c:cd:d0)
  Actor Key: 1001
  Actor Port Priority: 1
  Actor Port: 52
  
```

Рисунок 8.3.3.4. – Поля Actor System Priority и Actor System ID в LLCPDU

Приоритет системы можно изменить командой:

```
MES(config)# lacp system-priority value
```

Узнать текущее значение приоритета системы можно с помощью команд

```

MES# show lacp Port-Channel 1
Port-Channel Po1
  Minimal links required: 1
  Port Type Gigabit Ethernet
  Attached Lag id:
  Actor
    System Priority:1
    MAC Address:    e0:d9:e3:c2:37:00
    Admin Key:      1000
    Oper Key:       1000
  Partner
    System Priority:32768
    MAC Address:    e4:5a:d4:8e:0d:40
    Oper Key:       1
  
```

Активные порты группы каналов выбираются согласно двум параметрам:

1. Приоритет порта (Port Priority);
2. Номер порта (Port Number).

```

v Link Aggregation Control Protocol
  LACP Version: 0x01
  TLV Type: Actor Information (0x01)
  TLV Length: 0x14
  Actor System Priority: 1
  Actor System ID: EltexEnt_8c:cd:d0 (e4:5a:d4:8c:cd:d0)
  Actor Key: 1001
  Actor Port Priority: 1
  Actor Port: 52
  
```

Рисунок 8.3.3.5. – Приоритет порта и номер порта в LLCPDU

Администратор может поменять приоритет порта командой:

```
MES(config-if) # lacp port-priority value
```

где *value* – значение от 1 до 65535.

При одинаковых приоритетах активными становятся 8 портов с наименьшим номером.

8.3.4. Настройка режимов работы канальных интерфейсов

При объединении нескольких портов в агрегированный канал настройки режима работы порта осуществляются на Port-Channel интерфейсе.

При конфигурации интерфейса Port-channel используются те же команды, что и для физического интерфейса. Например, если нужно настроить интерфейс Port-channel 1 (Po1) в режиме General и разрешить на нем прохождение VLAN 10 и 20, выполняются команды:

```
MES(config) # interface Port-Channel 1
```

```
MES(config-if) # switchport mode general
```

```
MES(config-if) # switchport trunk allowed vlan add 10,20
```

8.4. Избыточность на уровне L3

8.4.1. Необходимость резервирования шлюза по умолчанию

Шлюз по умолчанию играет в сети огромную роль: именно через него осуществляется пересылка пакетов от оконечного устройства в сети, отличные от собственной сети устройства. Более подробно механизм пересылки пакетов рассмотрен в главе 4.

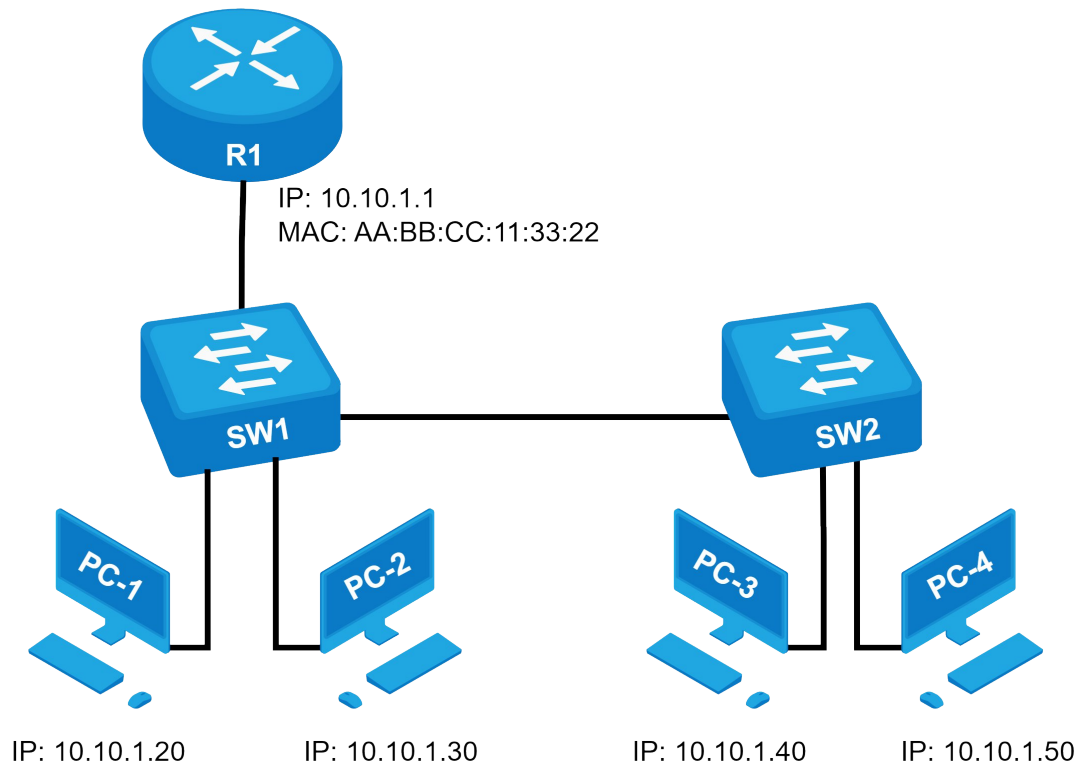


Рисунок 8.4.1.1. –Сеть с одним маршрутизатором, выполняющим роль шлюза по умолчанию

В сети на рисунке 8.4.1.1. присутствует единственный шлюз ко умолчанию: интерфейс маршрутизатора R1 с IP 10.10.1.1 и MAC AA:BB:CC:11:33:22.

Доменом отказа этого шлюза по умолчанию является вся локальная сеть, то есть при выходе его из строя ни один из компьютеров не сможет отправить ни один пакет за пределы сети.

Что произойдет, если в сеть добавить второй маршрутизатор с IP-адресом 10.10.1.1?

При установке IP-адреса на интерфейс маршрутизатор выполняет процедуру верификации IP-адреса: отправляет ARP-запрос на вновь устанавливаемый адрес. Если придет ответ, значит, адрес уже установлен на другом устройстве, и во избежание конфликта IP-адресов маршрутизатор выдаст ошибку «Невозможно установить данный IP-адрес». То есть «пассивный» вариант резервирования путем простого добавления еще одного L3-устройства невозможен.

На интерфейс маршрутизатора R2 придется установить IP-адрес, отличный от 10.10.1.1, например, 10.10.1.2.

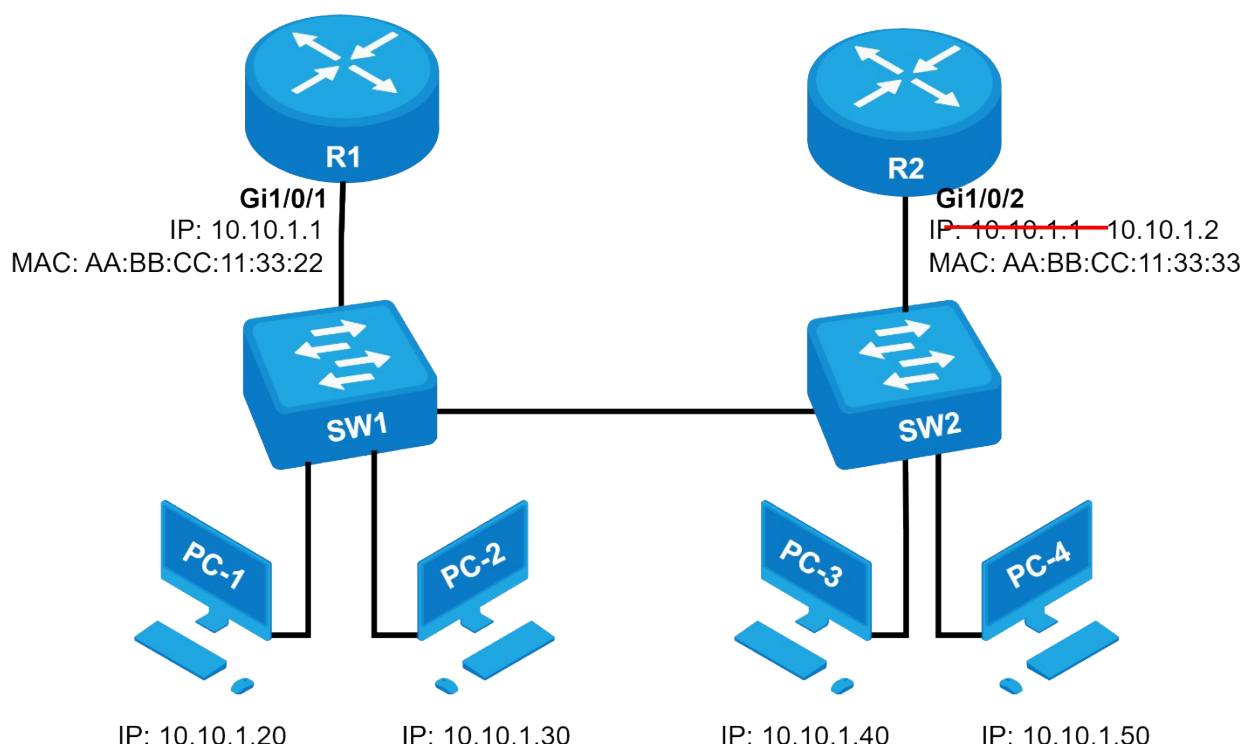


Рис. 8.4.1.2. – Попытка внедрения двух маршрутизаторов с одним IP-адресом в одну локальную сеть

Следовательно, у администратора сети остается два варианта:

1. При отказе маршрутизатора R1 изменить на R2 адрес интерфейса на 10.10.1.1;
2. Изменить на всех компьютерах адрес шлюза по умолчанию на 10.10.1.2.

Ни первый, ни второй варианты не смогут решить главный вопрос: обеспечение бесперебойной работы сети при отказе шлюза по умолчанию.

Для этой цели используются протоколы избыточности первого перехода (*First Hot Redundancy Router Protocols*). Их основная задача – отслеживать работоспособность основного маршрутизатора, выполняющего роль шлюза по умолчанию, и замещать его в случае его отказа.

В нашем курсе будет рассмотрен протокол VRRP. **VRRP (Virtual Router Redundancy Protocol)** – протокол резервирования с помощью виртуального маршрутизатора). Предназначен для увеличения доступности маршрутизаторов, выполняющих роль шлюза по умолчанию.

Данный протокол является стандартным и работает по одному и тому же алгоритму на устройствах различных производителей. Алгоритм работы протокола VRRP описан в [RFC 2338](#).

8.4.2. Принцип работы VRRP

8.4.2.1. Дублирование маршрутизаторов

Чтобы понять, как работает протокол VRRP, зададим себе вопрос: как компьютер определяет, какое устройство является шлюзом по умолчанию? Очевидно, что для него это устройство, которое отвечает своим MAC-адресом на ARP-запрос по адресу шлюза по умолчанию. В нашем случае, если компьютер отошлет ARP-запрос на адрес 10.10.1.1,

шлюзом по умолчанию для него будет любое устройство, которое ответит на него своим MAC-адресом. При этом MAC-адрес может быть любым, так как оконечное устройство не может проверить, принадлежит ли MAC-адрес реально существующему или виртуальному устройству.

Следовательно, если маршрутизатор R1 ответит на ARP-запрос оконечного устройства виртуальным MAC-адресом, оконечное устройство примет этот результат ARP-ответа и будет в дальнейшем использовать его в поле Destination MAC для формирования кадров. Если маршрутизатор R1 примет такой кадр и обработает его для отправки согласно таблице маршрутизации, то результат будет совершенно приемлемым как для оконечного устройства, так и для его пользователя.

Так как запрос ARP передается широковещательным кадром, его может получить не только маршрутизатор R1, но и маршрутизатор R2, который также может ответить на него виртуальным MAC-адресом, а затем получить и обработать кадр с данным MAC-адресом в поле Destination MAC. При этом MAC-адрес, которым отвечает маршрутизатор R2, может быть идентичным MAC-адресу, которым отвечает маршрутизатор R1.

Стало быть, в сети может существовать более одного маршрутизатора, способного ответить на ARP-запрос одинаковым MAC-адресом, пригодным для дальнейшей пересылки трафика.

Для того, чтобы отвечать на ARP-запросы по одному IP-адресу, несколько маршрутизаторов должны быть объединены в одну группу виртуального маршрутизатора (*Virtual Router, VR*) с собственным номером (*Virtual Router ID, VRID*). Все маршрутизаторы в группе с одним VRID могут отвечать на ARP-запросы по адресу виртуального маршрутизатора MAC-адресом 00:00:5E:00:01:XX, где **XX** – номер группы в шестнадцатеричном формате.

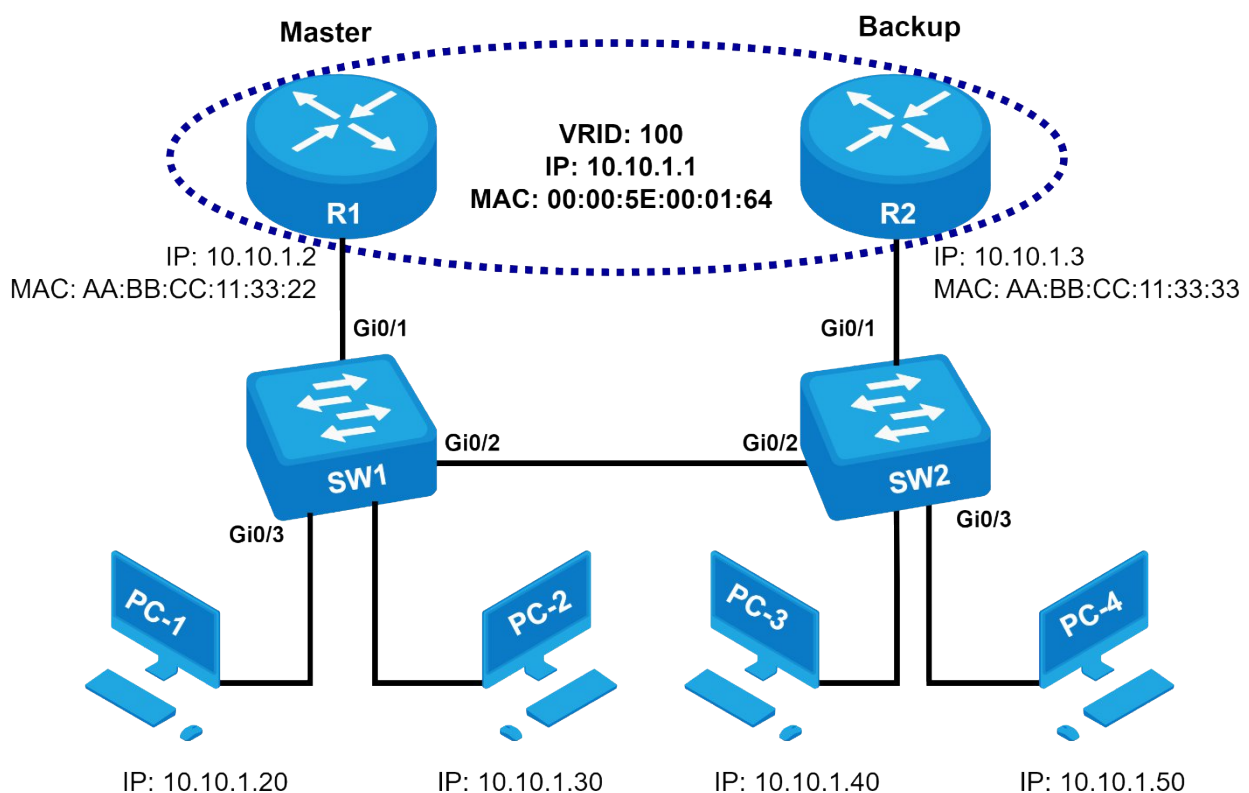


Рисунок 8.4.2.1.1. – Маршрутизаторы в одной группе виртуального маршрутизатора

8.4.2.2. Синхронизация маршрутизаторов

Следующая задача – обеспечить синхронизацию маршрутизаторов R1 и R2 для того, чтобы они не отвечали на ARP-запрос по адресу шлюза по умолчанию одновременно.

Синхронизация начинается с определения основного (*Master Router*) и запасного (*Backup Router*) маршрутизаторов. Маршрутизатор становится основным, если:

1. Он обладает более высоким приоритетом (полю Priority может быть присвоено значение от 1 до 254);
2. IP-адрес шлюза по умолчанию явно назначен на один из его интерфейсов (маршрутизатор является *владельцем* виртуального IP-адреса). В этом случае его приоритет принимает максимальное значение – 255.

В любой момент времени только один из физических маршрутизаторов выполняет маршрутизацию трафика, то есть становится основным маршрутизатором (*VRRP Master router*), остальные маршрутизаторы в группе становятся запасными (*VRRP Backup router*).

Master-маршрутизатор рассылает сообщения-анонсы (*VRRP Announcements*). Основная цель этих сообщений – дать понять запасным маршрутизаторам, что основной маршрутизатор находится в рабочем состоянии и может отвечать на ARP-запросы конечных устройств.

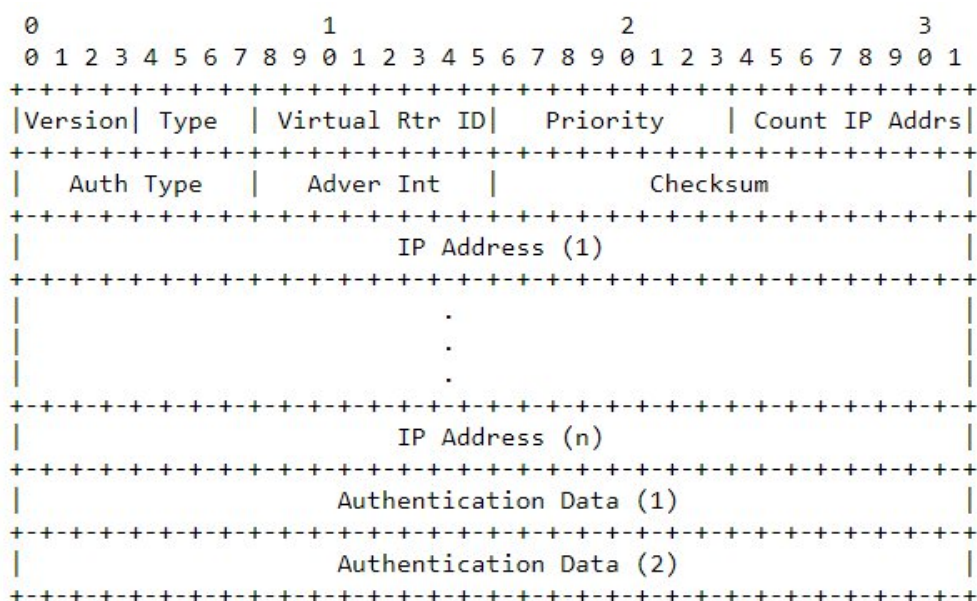


Рисунок 8.4.2.2.1. – Анонс VRRP (RFC 2338)

Анонсы VRRP посылаются на групповой IP-адрес 224.0.0.18 (соответствующий MAC-адрес 01:00:5E:00:00:12) с частотой 1 раз в секунду. Анонс отправляется на MAC-адрес 01:00:5E:00:00:18, соответствующий групповому IP-адресу 224.0.0.18, с MAC-адреса виртуального маршрутизатора 00:00:5E:00:01:64. Последний октет MAC-адреса соответствует идентификатору виртуального маршрутизатора (в нашем примере VRID=100).

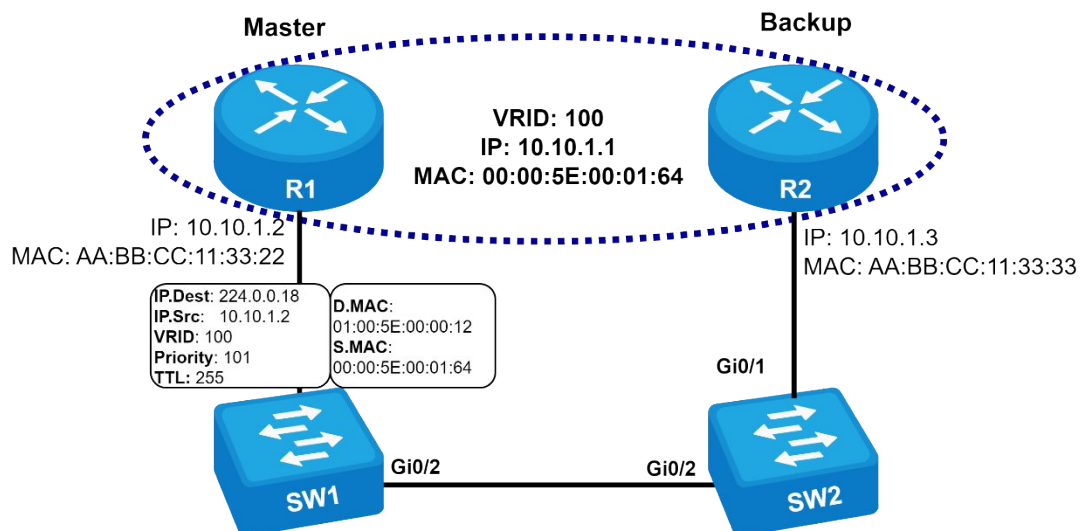


Рисунок 8.4.2.2.2. – Анонс VRRP, отправленный Master-маршрутизатором

Отличительной особенностью пакета анонса VRRP является значение поля Time to Live 255. Во-первых, оно не позволяет маршрутизаторам пересылать этот пакет дальше первого интерфейса в локальной сети, во-вторых, служит индикатором легитимности пакета, так как означает, что пакет был инициирован легальным маршрутизатором VRRP. При получении пакета VRRP с TTL, отличным от 255, согласно стандарту RFC 2338, маршрутизатор обязан отбросить данный пакет.

Так как анонс VRRP отсылается на групповой MAC-адрес (восьмой бит первого октета равен 1), он пересылается коммутаторами из всех портов, за исключением порта получения.

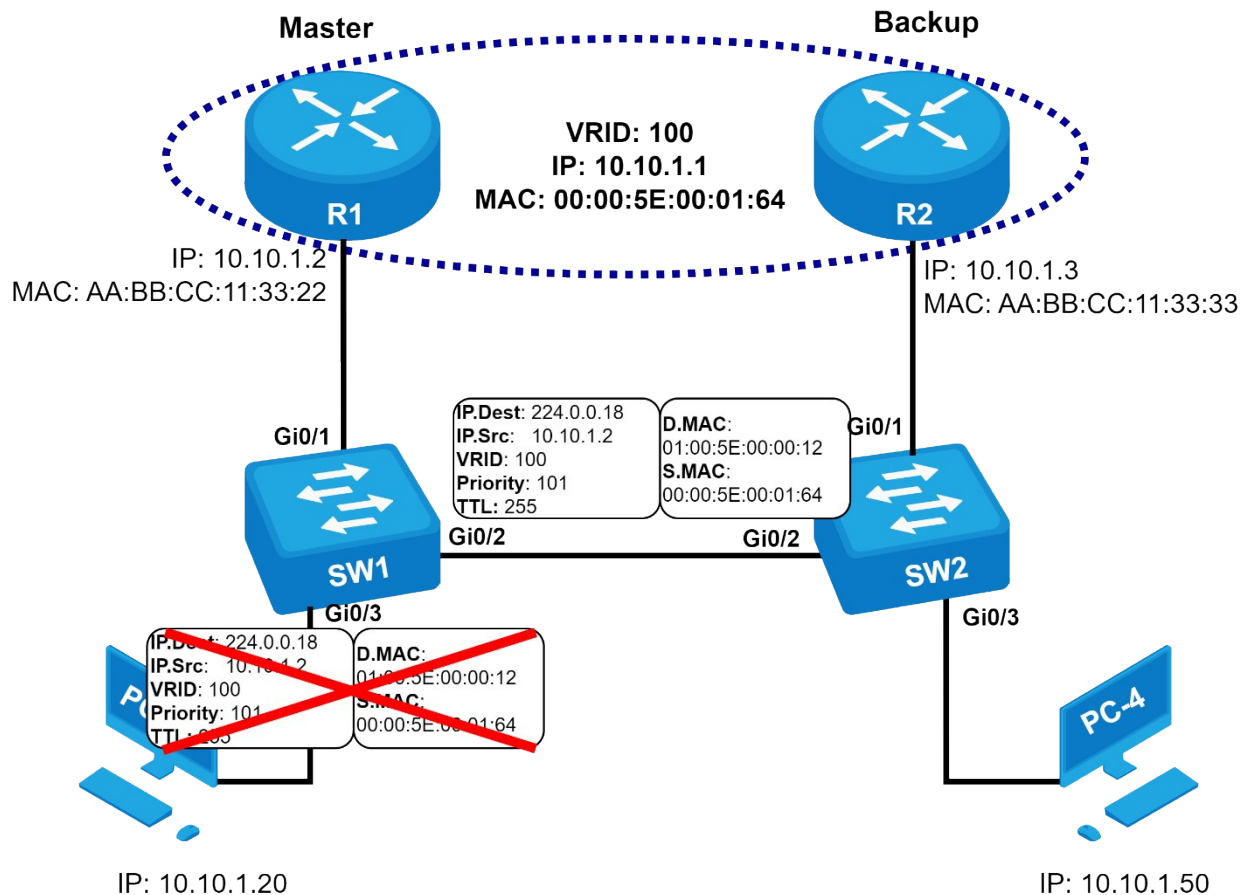


Рисунок 8.4.2.2.3. – Передача анонса VRRP коммутаторами в локальной сети

Компьютер PC-1, получив анонс VRRP, отбрасывает его, так как ни MAC-адрес, ни групповой IP-адрес не соответствуют его интерфейсам. А коммутатор SW-2, получив данный пакет на интерфейсе Gi0/2, рассылает его в порты Gi0/3 и Gi0/1.

Подключенный к порту Gi0/3 компьютер PC-4 отбрасывает пакет аналогично компьютеру PC-1.

Однако отправленный из порта Gi0/1 пакет принимается и обрабатывается маршрутизатором R2.

Из данных в пакете анонса VRRP маршрутизатор R2 получает следующую информацию:

1. Анонс поступил от маршрутизатора, состоящего с ним в одной группе виртуального маршрутизатора (VRID = 100);
2. У маршрутизатора, отправившего анонс, больший приоритет, а значит, он является основным (Master). Соответственно, маршрутизатор R2 принимает роль запасного (Backup);
3. Поле AdvertInt содержит интервал рассылки анонсов Master-маршрутизатором. Значение этого поля используется для вычисления времени перехода запасного маршрутизатора в состояние Master.

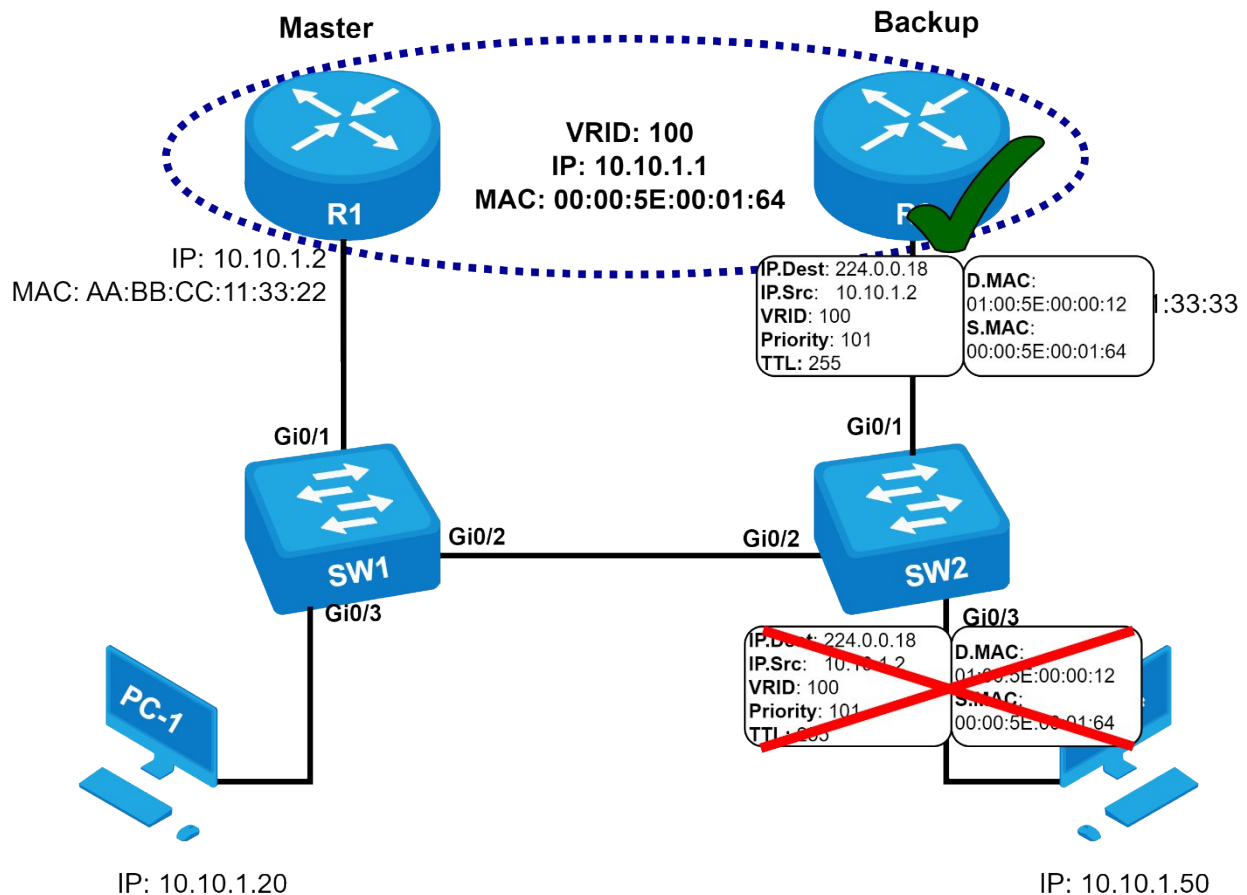


Рисунок 8.4.2.2.3. – Обработка анонса VRRP маршрутизатором R2

В анализаторе трафика Wireshark анонс VRRP выглядит следующим образом:

```
Ethernet II, Src: IETF-VRRP-VRID_64 (00:00:5e:00:01:64), Dst: IPv4mcast_12 (01:00:5e:00:00:12)
Internet Protocol Version 4, Src: 10.10.1.2, Dst: 224.0.0.18
Virtual Router Redundancy Protocol
  > Version 2, Packet type 1 (Advertisement)
    Virtual Rtr ID: 100
    Priority: 110 (Non-default backup priority)
    Addr Count: 1
    Auth Type: No Authentication (0)
    Adver Int: 1
    Checksum: 0x658e [correct]
    [Checksum Status: Good]
    IP Address: 10.10.1.1
```

Рисунок 8.4.2.2.4. – Анонс VRRP в Wire Shark

8.4.2.3. Состояния маршрутизаторов

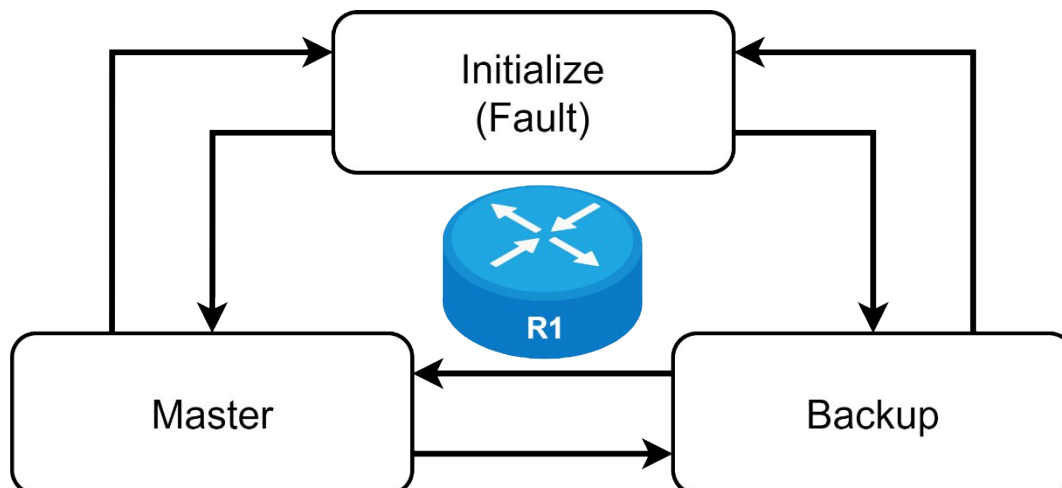


Рисунок 8.4.2.3.1. – Возможные состояния маршрутизатора VRRP

При включении протокола VRRP начальным состоянием маршрутизатора является состояние *Initialize* (инициализация), которое в маршрутизаторах Eltex ESR называется *Fault*, так как длительное пребывание в таком состоянии свидетельствует об ошибках в настройке протокола VRRP.

В состоянии *Initialize* маршрутизатор находится до тех пор, пока не проверит, установлен ли на его интерфейсе IP-адрес, совпадающий с адресом виртуального маршрутизатора (шлюза по умолчанию). Если это так, маршрутизатор переходит в состояние *Master*, а если нет – в состояние *Backup*.

Перейдя в состояние *Master*, маршрутизатор начинает рассылку анонсов VRRP и продолжает их либо до отключения протокола VRRP администратором, либо до получения анонса VRRP с большим, чем у него самого приоритетом или с равным, но большим IP-адресом отправителя.

При отключении протокола VRRP маршрутизатор отправляет анонс с приоритетом равным 0 и переходит в состояние *Initialize*.

При получении анонса с приоритетом большим, чем у него (или с равным, но от отправителя с большим IP-адресом), маршрутизатор переходит в состояние *Backup*.

Процесс перехода между состояниями процесса VRRP показан на рисунке 8.4.2.3.2.

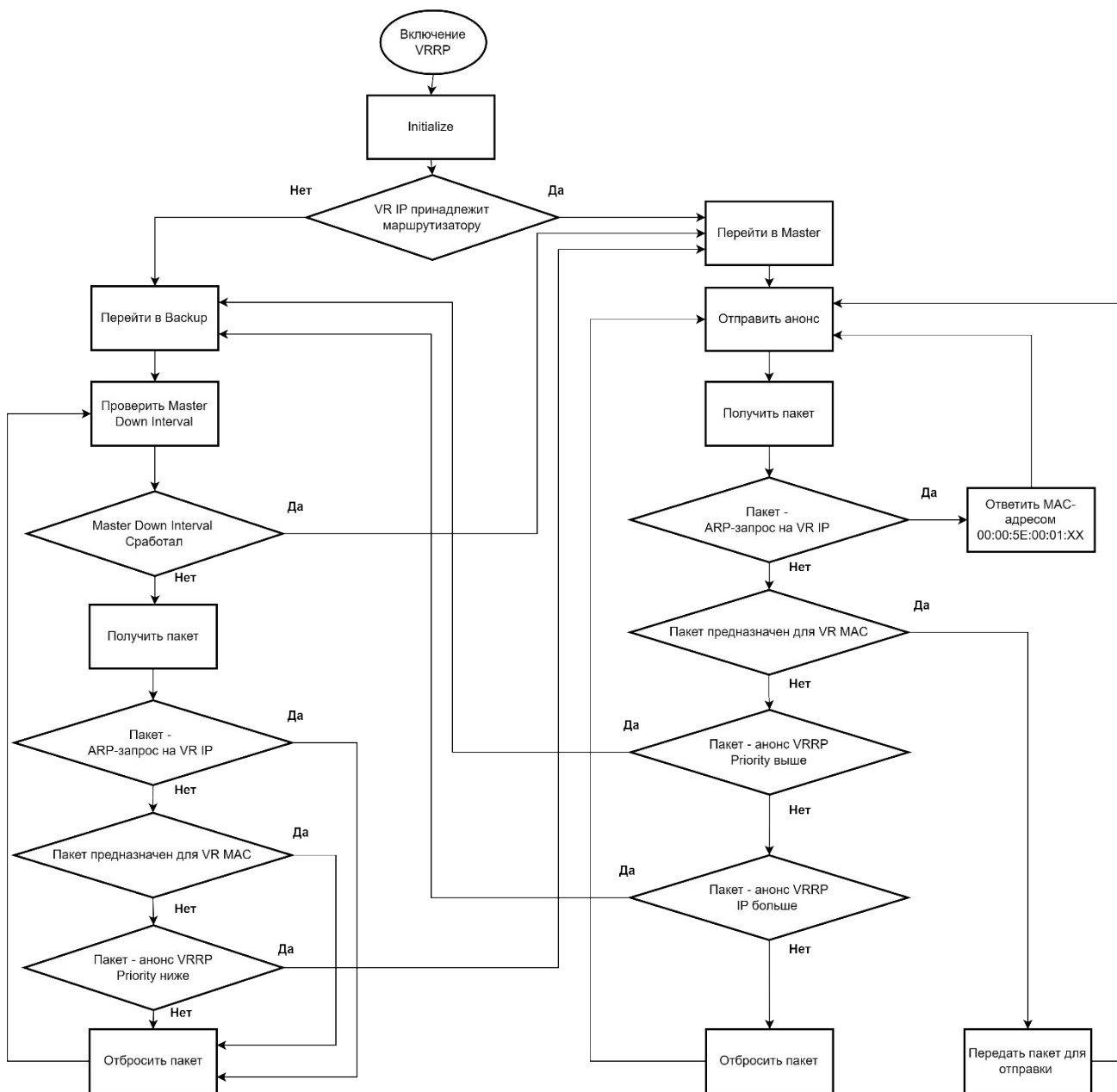


Рисунок 8.4.2.3.2. – Схема смены состояний протокола VRRP

Необходимо обратить внимание на параметр *Master Down Interval*. Это промежуток времени, по истечению которого Backup-маршрутизатор переходит в состояние Master. Он рассчитывается по формуле:

$$MDI = 3 \times AdverInt + ST$$

Где **AdverInt** – значение поля Advertising Interval (интервал анонсов). По умолчанию равно 1 секунде;

ST – SkewTime (время перекося). Рассчитывается по формуле:

$$ST = \frac{(256 - Pri) * AdverInt}{256}$$

Общий вид формулы расчета выглядит следующим образом:

$$MDI = 3 \times AdverInt + \frac{(256 - Pri) * AdverInt}{256}$$

Чем выше приоритет маршрутизатора, тем меньше время его ожидания перед переходом в состояние мастера. Чем выше числовое значение приоритета, тем больше шансов маршрутизатору стать новым мастером.

Значение поля *Priority* может меняться от 1 до 254. По умолчанию оно равно 100.

Пользователь не может задать «крайние» значения поля *Priority*:

- 255: поле принимает это значение, если физический маршрутизатор является владельцем IP-адреса виртуального маршрутизатора;
- 0: поле принимает это значение при отключении протокола VRRP на маршрутизаторе и его переходе в состояние Initialize.

8.4.3. Команды настройки VRRP

Настроить VRRP можно на интерфейсе, bridge-интерфейсе или туннеле. Для целей нашего курса будет рассматриваться настройка VRRP на интерфейсах.

Перед включением протокола VRRP нужно настроить IP-адрес и отключить или настроить Firewall.

```
esr(config)# interface gigabitethernet 1/0/x
esr(config-if-gi)# ip address <ipaddress/prefixlength>
esr(config-if-gi)# ip firewall disable
```

где *ipaddress* – это собственный адрес интерфейса, а *prefixlength* – длина префикса. IP-адрес также можно вводить в формате «IP-адрес маска_подсети».

Интерфейс подготовлен к конфигурации протокола VRRP. Приступаем к самой конфигурации.

Сконфигурируем адрес виртуального маршрутизатора:

```
esr(config-if-gi)# vrrp ip <ipaddress/prefixlength>
```

Затем укажем идентификатор виртуального маршрутизатора VRID:

```
esr(config-if-gi)# vrrp id yyy
```

где ууу – целое число от 1 до 255.

Установим приоритет маршрутизатора в процессе VRRP:

```
esr(config-if-gi)# vrrp priority zzz
```

где zzz – целое число от 1 до 254. Как уже отмечалось ранее, значение приоритета влияет на время перехода маршрутизатора из состояния *Backup* в состояние *Master*. При совпадении IP-адресов интерфейса и виртуального маршрутизатора приоритет автоматически устанавливается равным 255. При несовпадении этих параметров по умолчанию устанавливается приоритет 100.

Процесс VRRP на интерфейсе включается командой:

```
esr(config-if-gi) # vrrp
```

8.4.4. Приоритетное вытеснение (Preempt)

Приоритетное вытеснение (preempt) – это способность маршрутизатора VRRP, находящегося в состоянии Backup, перенимать роль Master у существующего «мастера». Для этого Backup-маршрутизатору необходимо иметь приоритет выше, чем у текущего Master-маршрутизатора.

Для чего может потребоваться такая способность? Самый очевидный пример – возвращение бывшего «мастера» после отключения или сбоя. До отключения у него уже был наивысший приоритет среди маршрутизаторов VRRP, иначе он бы не стал ведущим маршрутизатором, и теперь, включаясь после сбоя, он может «потребовать» передать ему «полномочия» рассылки анонсов. Однако при любой передаче «мастерства» может возникнуть вероятность потери пакетов, передаваемых через текущего «мастера». В связи с этим администратор может ограничить переходы мастерства, отключив возможность приоритетного вытеснения на Master-маршрутизаторе.

Возможность приоритетного вытеснения передается переменной `Preempt_State`, которая может принимать значение `True` или `False`. Если переменная имеет значение `True`, то маршрутизатор будет высылать собственный анонс, если у действующего «мастера» приоритет ниже. Если же данная переменная имеет значение `False`, то маршрутизатор будет высылать собственные анонсы, только если сработал `Master Down Interval`, то есть текущий «мастер» прекратил работу.

По умолчанию переменная `Preempt_State` имеет значение `True`, то есть приоритетное вытеснение включено. Выключить его, присвоив переменной `Preempt_State` значение `False`, можно командой:

```
ESR(config-if-gi) #vrrp preempt disable
```

Снова включить функцию приоритетного вытеснения после ее отключения можно командой:

```
ESR(config-if-gi) #no vrrp preempt disable
```

8.4.5. Диагностика VRRP

Для диагностики текущего состояния процессов VRRP на маршрутизаторе Eltex ESR необходимо использовать команду:

```
ESR#show vrrp
```

```
ESR# show vrrp
Virtual router    Virtual IP                Priority    Preemption    State
-----
100               10.10.1.1/32              110        Disabled      Master
200               10.10.2.1/32              10         Disabled      Backup
```

Рисунок 8.4.5.1. – Диагностика всех процессов VRRP на маршрутизаторе ESR

Для просмотра отдельного процесса VRRP на маршрутизаторе можно использовать команду `show vrrp` с указанием конкретного идентификатора виртуального маршрутизатора:

```
ESR#show vrrp vrid
```

```
ESR# show vrrp 100
VRRP ID:                100
VRRP Version:           2
Interface:              gi1/0/1
Priority:                110
State:                  Master
Track ID:               --
Source IP address:      10.10.1.2
Virtual IP address:     10.10.1.1/32
Virtual MAC address:    00:00:5e:00:01:64
Synchronization group ID: --
Preemption:             Disabled
Advertisement interval:  1 seconds
Gratuitous ARP delay:   5 seconds
Gratuitous ARP repeat:  5 times
Gratuitous ARP refresh: 0 seconds
Gratuitous ARP refresh repeat: 1 times
```

Рисунок 8.4.5.2. – Диагностика индивидуального виртуального маршрутизатора

Оценить правильность конфигурации протокола VRRP на конкретном интерфейсе можно с помощью проверки текущей конфигурации интерфейса командой `show running-config interfaces`:

```
ESR# show running-config interfaces gigabitethernet 1/0/1
interface gigabitethernet 1/0/1
 ip firewall disable
 ip address 10.10.1.2/24
 vrrp id 100
 vrrp ip 10.10.1.1/32
 vrrp priority 110
 vrrp preempt disable
 vrrp
 lldp transmit
 lldp receive
exit
```

Рисунок 8.4.5.3. – Проверка правильности текущей конфигурации VRRP на интерфейсе

8.4.6. Примеры настройки VRRP

Пример 1.

Администратору необходимо обеспечить резервирование шлюза по умолчанию с адресом 10.10.1.1 для сети 10.10.1.0/24. Для этого имеются два маршрутизатора: R1 и R2. На маршрутизаторе R1 в локальную сеть подключен интерфейс Gi1/0/1 с IP-адресом 10.10.1.2, а на маршрутизаторе R2 – интерфейс Gi1/0/2 с IP-адресом 10.10.1.3.

Маршрутизатор R1 должен всегда принимать роль ведущего маршрутизатора (Master), за исключением случаев отказа, перезагрузки или отключения протокола VRRP.

Кроме того, необходимо обеспечить доступность шлюза по умолчанию для

компьютеров PC-1 и PC-4, которые на уровне доступа подключены к VLAN 10.

Схема подключения дана на рисунке 8.4.6.1.

Рассмотрим план конфигурирования устройств:

1. Сконфигурировать VRRP на маршрутизаторе R1 (Master);
2. Сконфигурировать VRRP на маршрутизаторе R2 (Backup);
3. Сконфигурировать VLAN 10 на коммутаторах SW1 и SW2;
4. Разрешить VLAN 10 на транковом канале между SW1 и SW2.

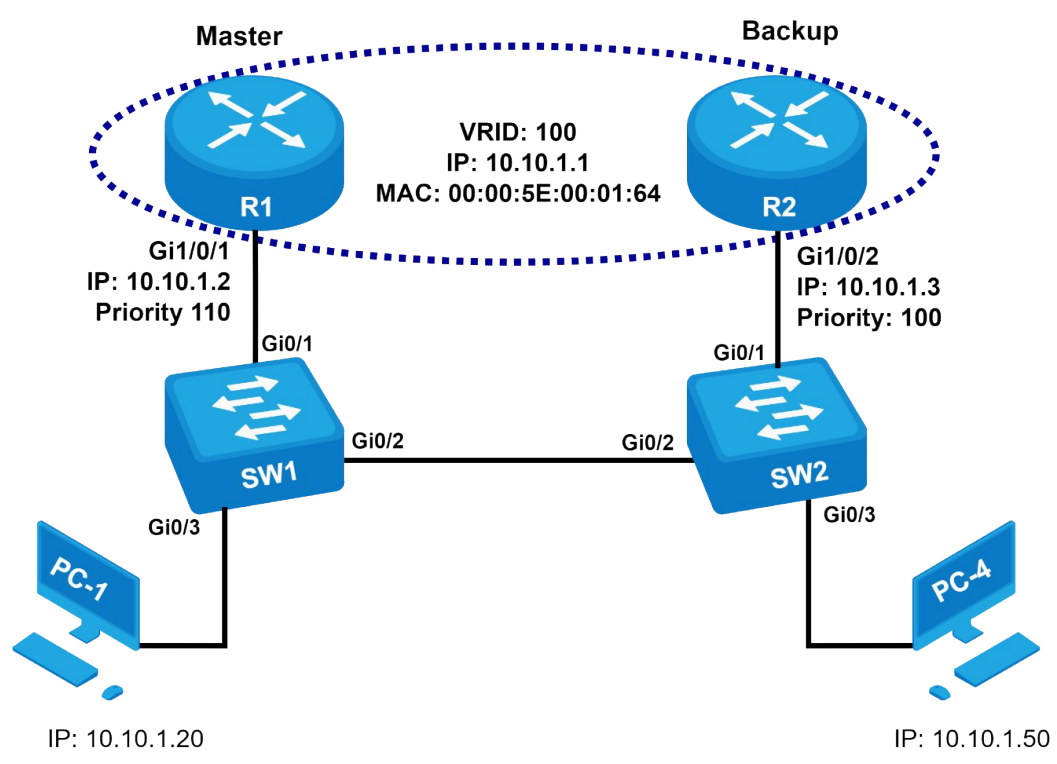


Рисунок 8.4.6.1. – Топология для настройки VRRP

Таблица 8.4.6.1. – Настройки маршрутизаторов

Маршрутизатор R1	Маршрутизатор R2
Вход в режим глобальной конфигурации	
R1# configure	R2# configure
Выбор интерфейса для конфигурации	
R1(config)# interface Gi1/0/1	R2(config)# interface Gi1/0/2
Настройка IP-адреса	
R1(config-if-gi)# ip address 10.10.1.2/24	R2(config-if-gi)# ip address 10.10.1.3/24
Отключение межсетевого экрана на интерфейсах маршрутизаторов	
R1(config-if-gi)# ip firewall disable	R2(config-if-gi)# ip firewall disable
Настройка идентификатора виртуального маршрутизатора	
R1(config-if-gi)# vrrp id 100	R2(config-if-gi)# vrrp id 100
Настройка виртуального IP-адреса	

R1(config-if-gi) # vrrp ip 10.10.1.1	R2(config-if-gi) # vrrp ip 10.10.1.1
Настройка приоритета маршрутизатора	
R1(config-if-gi) # vrrp priority 110	Так как приоритет 100 является значением по умолчанию, настройка не нужна
Настройка приоритетного вытеснения (Preempt)	
Так как приоритетное вытеснение включено по умолчанию, дополнительная настройка не нужна	Так как у маршрутизатора R2 меньший приоритет, он будет проигрывать R1, а значит, можно не отключать Preempt дополнительно.
Включение виртуального маршрутизатора на интерфейсе	
R1(config-if-gi) # vrrp	R2(config-if-gi) # vrrp
Применение и подтверждение изменений в конфигурации	
R1(config-if-gi) # do commit	R2(config-if-gi) # do commit
R1(config-if-gi) # do confirm	R2(config-if-gi) # do confirm

Таблица 8.4.6.2. – Настройки коммутаторов

Коммутатор SW1	Коммутатор SW2
Вход в режим глобальной конфигурации	
SW1# configure	SW2# configure
Выбор интерфейсов	
SW1(config)# interface range gi0/1,gi0/3	SW2(config)# interface range gi0/1,gi0/3
Конфигурация режима доступа и VLAN	
SW1(config-if-range)# switchport mode access	SW2(config-if-range)# switchport mode access
SW1(config-if-range)# switchport access vlan 10	SW2(config-if-range)# switchport access vlan 10
Выбор интерфейса для транкового канала и добавление VLAN 10 в список разрешенных	
SW1(config-if-range)# interface gi0/2	SW1(config-if-range)# interface gi0/2
SW1(config-if)# switchport mode trunk	SW1(config-if)# switchport mode trunk
SW1(config-if)# switchport trunk allowed vlan add 10	SW1(config-if)# switchport trunk allowed vlan add 10

Пример 2. Настройка резервирования шлюза по умолчанию для двух VLAN

Задача: настроить резервирование шлюза по умолчанию для VLAN 10 и 20 с адресацией 10.10.1.0/24 и 10.10.2.0/24 соответственно. Маршрутизатор R1 сделать Master для виртуального маршрутизатора с адресом 10.10.1.1 (VRID 100), а маршрутизатор R2 – для виртуального маршрутизатора с адресом 10.10.2.1 (VRID 200).

Обеспечить доступность шлюза по умолчанию по адресу 10.10.1.1 для компьютеров PC-1 и PC-4, поместив их в VLAN 10, а доступность шлюза по умолчанию по адресу 10.10.2.1 для компьютеров PC-2 и PC-3, подключив их в VLAN 20.

Схема подключения дана на рисунке 8.4.6.2.

Рассмотрим план конфигурирования устройств:

1. Сконфигурировать VRID 100 на подинтерфейсе Gi1/0/1.10 маршрутизатора R1 (Master);
2. Сконфигурировать VRID 200 на подинтерфейсе Gi1/0/1.20 маршрутизатора R1 (Backup);
3. Сконфигурировать VRID 100 на подинтерфейсе Gi1/0/1.10 маршрутизатора R2 (Backup);
4. Сконфигурировать VRID 200 на подинтерфейсе Gi1/0/1.20 маршрутизатора R2 (Master);
5. Сконфигурировать VLAN 10 и 20 на портах доступа коммутаторов SW1 и SW2;
6. Разрешить VLAN 10 и 20 на транковых каналах коммутаторов SW1 и SW2.

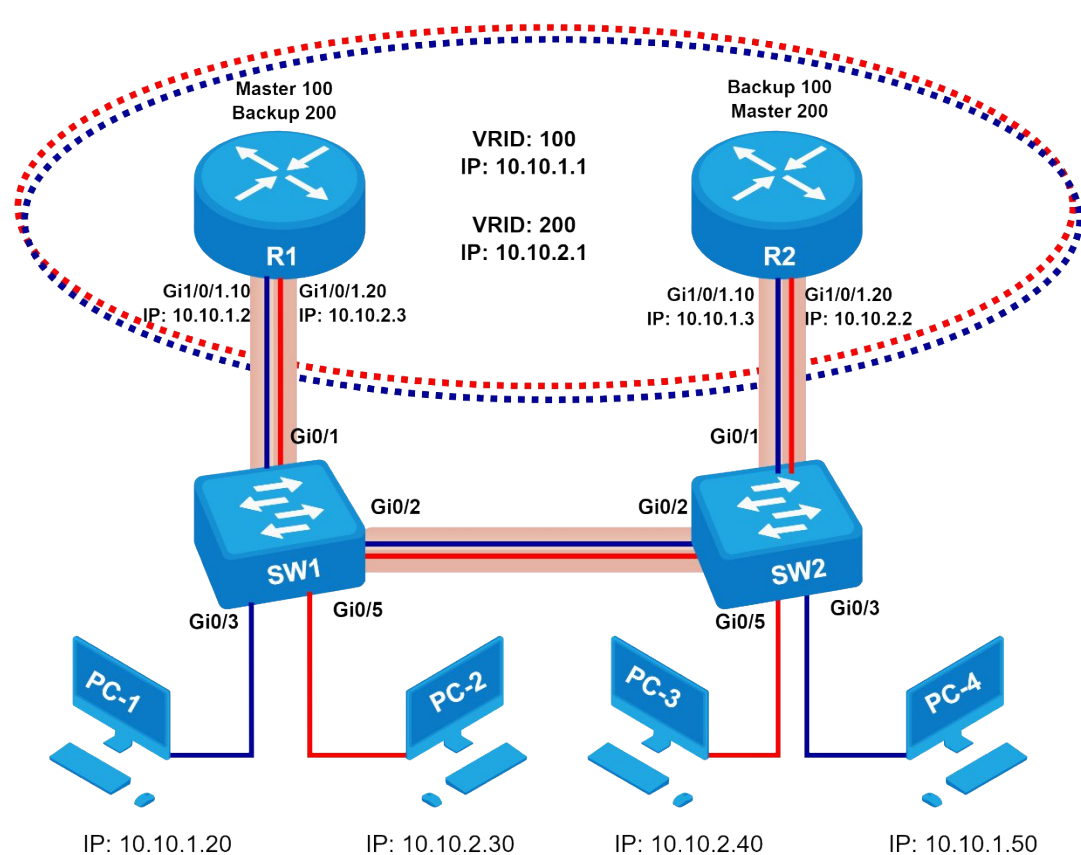


Рисунок 8.4.6.2. – Настройка VRRP с VLAN

Таблица 8.4.6.3. – Настройки маршрутизаторов

Маршрутизатор R1	Маршрутизатор R2
Вход в режим глобальной конфигурации	
R1# configure	R2# configure
Создание подинтерфейсов, конфигурация IP-адресов, МСЭ	
R1 (config) # interface Gi1/0/1.10	R2 (config) # interface Gi1/0/1.10
R1 (config-subif) # ip address 10.10.1.2/24	R2 (config-subif) # ip address 10.10.1.3/24
R1 (config-subif) # ip firewall disable	R2 (config-subif) # ip firewall disable
R1 (config-subif) # exit	R2 (config-subif) # exit
R1 (config) # interface Gi1/0/1.20	R2 (config) # interface Gi1/0/1.20

R1(config-subif) # ip address 10.10.2.3/24	R2(config-subif) # ip address 10.10.2.2/24
R1(config-subif) # ip firewall disable	R2(config-subif) # ip firewall disable
R1(config-subif) # exit	R2(config-subif) # exit

Таблица 8.4.6.3. – Настройки маршрутизаторов (продолжение)

Маршрутизатор R1	Маршрутизатор R2
Настройка виртуального маршрутизатора VRID 100	
R1(config) # interface gil/0/1.10	R2(config) # interface gil/0/1.10
R1(config-subif) vrrp id 100	R2(config-subif) vrrp id 100
R1(config-subif) vrrp ip 10.10.1.1	R2(config-subif) vrrp ip 10.10.1.1
R1(config-subif) vrrp priority 110	R2(config-subif) vrrp priority 10
R1(config-subif) vrrp	R2(config-subif) vrrp
R1(config-subif) exit	R2(config-subif) exit
Настройка виртуального маршрутизатора VRID 200	
R1(config) # interface gil/0/1.20	R2(config) # interface gil/0/1.20
R1(config-subif) vrrp id 200	R2(config-subif) vrrp id 200
R1(config-subif) vrrp ip 10.10.2.1	R2(config-subif) vrrp ip 10.10.2.1
R1(config-subif) vrrp priority 10	R2(config-subif) vrrp priority 110
R1(config-subif) vrrp	R2(config-subif) vrrp
R1(config-subif) exit	R2(config-subif) exit
Принятие и подтверждение настроек	
R1(config) # do commit	R2(config) # do commit
R1(config) # do confirm	R2(config) # do confirm

Таблица 8.4.6.4. – Настройки коммутаторов

Коммутатор SW1	Коммутатор SW2
Конфигурация портов доступа	
SW1# configure	SW2# configure
SW1(config) # interface Gi0/3	SW2(config) # interface Gi0/3
SW1(config-if) # switchport mode access	SW2(config-if) # switchport mode access
SW1(config-if) # switchport access vlan 10	SW2(config-if) # switchport access vlan 10
SW1(config-if) # interface Gi0/5	SW2(config-if) # interface Gi0/5
SW1(config-if) # switchport mode access	SW2(config-if) # switchport mode access
SW1(config-if) # switchport access vlan 20	SW2(config-if) # switchport access vlan 20
Конфигурация транковых портов	
SW1(config) # interface range Gi0/1-2	SW2(config) # interface range Gi0/1-2
SW1(config-if-range) # switchport mode	SW2(config-if-range) # switchport mode

<pre>trunk SW1(config-if-range)# switchport trunk allowed vlan add 10,20</pre>	<pre>trunk SW2(config-if-range)# switchport trunk allowed vlan add 10,20</pre>
--	--